

DBLog Framework Core: A Source-Side Contract for Bracketed State Capture Interleaved with a Commit-Ordered Log

Andreas Andreakis

September 7, 2026

Abstract

Machine-checked theorem core for Generalized DBLog. One source-side contract — coordinates denoting nested log prefixes with a membership oracle, half-open capture windows, per-unit refreshes with per-key existential (bracket-local) validity, and a canonical latest-wins replay — is formalized together with its contract-level virtual-cut theorem and the instance work connecting it to the published formal study of classic watermarked DBLog [2] (whose *DBLog_Virtual_Cuts* session [1] is imported as a separate session with a shared-session refactoring that preserves the published mathematical content) and to write-free capture in the style of read-only incremental snapshots. The protocol family under study is that of DBLog [3]: bracketed full-state capture interleaved with a commit-ordered change log.

Contents

1	Contract base: events, source states, coordinates, and windows	3
1.1	Events, logs, and source states	4
1.2	Coordinate spaces: prefix denotations	6
1.3	Half-open windows and the membership oracle	7
1.4	Per-key event selection over prefixes and windows	8
2	Capture plans, the contract, and the contract-level cut theorem	9
2.1	Units and plans	10
2.2	The contract	11
2.3	The contract-level cut theorem	12
2.4	The conservative trajectory after the latest high	14
2.5	Corollary 1: exhibited shared-witness strengthening (T3) . .	16

2.6	Corollary 2: continuation under extended observation	17
2.7	Corollary 3: mixing (heterogeneous composition)	18
3	Emitted streams and the window-discard equivalence	20
3.1	List auxiliaries	21
3.2	Stream replay and position normal forms	22
3.3	The window-discard emission	23
3.4	Per-key characterization of the emission	25
3.5	The window-discard equivalence theorem	29
3.6	Window-discard per-key monotonicity	33
4	Constructed witnesses and boundary fixtures	34
4.1	The witness log and coordinate space	35
4.2	Fixture 1: the torn read	35
4.3	Fixture 2: the tablesync shape (composition is sharp at T2) .	37
4.4	Fixture 3: amortized shared bracket with a shared witness (T3)	43
4.5	Fixture 4: a surviving refresh under window-discard	44
5	Classic DBLog satisfies the contract	46
5.1	History-to-log mapping	47
5.2	Sorted histories: the at-or-before filter is a prefix	48
5.3	Generic filter-by-position normal form	49
5.4	State correspondence: the contract fold meets <i>Src</i>	49
5.5	Derived units and the import theorem	50
5.6	Consistency with the classic model’s conclusion	54
6	A write-free instance: read-only incremental snapshots over executed-GTID sets	55
6.1	GTID-labeled histories and executed-set coordinates	56
6.2	The read-only incremental plan	59
6.3	Constructed witness (non-vacuity)	61
7	The buffered-range-merge discipline	63
7.1	List auxiliary	64
7.2	The merged buffer and the gated emission	64
7.3	Clause (b): the merged buffer is the state at the high edge . .	65
7.4	Per-key characterization of the gated emission	66
7.5	The range-merge equivalence theorem	71
7.6	Range-merge per-key monotonicity	74
7.7	Scalar offset coordinates (shared by the offset instances) . . .	75
7.8	Necessity of the withholding clause (a) — fixture	76

8	A parallel-chunks instance: Flink-style incremental snapshots	78
8.1	The parallel-chunks plan	79
8.2	Fidelity, cut, equivalence, and mixing at the instance	82
8.3	Constructed witness (non-vacuity)	84
9	Native-dump splice and LSN-positioned selects	89
9.1	The trusted point splice	91
9.2	The uncertainty-bracket degradation	94
9.3	Constructed witness: the trusted splice (non-vacuity)	97
9.4	Constructed witness: the degraded bracket (non-vacuity)	99

Vocabulary

The paper calls obligation O2 *bracket-local validity*, the values a refresh assigns *read results*, the buffer of the range-merge discipline the *merged result*, and the coordinate of Corollary 1 a *shared state witness*. The prose of this development uses those terms. Its formal identifiers keep the earlier names under which the proofs were checked, namely `O2_honest`, `honest`, `wit_honest`, `wf_read_honest`, `refresh_honest_at_rec`, `mixing_shared_honesty_lift`, `wit_sync_memberA_shared_honesty`, and `wit_sync_memberB_shared_honesty`. Read *honest* as *valid* wherever an identifier carries it.

```
theory Contract-Base
  imports Main HOL-Library.Sublist
begin
```

1 Contract base: events, source states, coordinates, and windows

Base substrate for the source-side capture contract: keyed events in a commit-ordered log, the source state as a per-key fold of the log, coordinate spaces as prefix denotations with the membership oracle, and half-open windows.

Representational notes — which contract clauses are carried by the representation itself rather than by named assumptions:

- S-LOG (commit-ordered keyed log): the log is a fixed finite list of events in commit order; “every committed change to a scoped key appears as exactly one event” is the modeling identification of committed changes with list occurrences. The event type denotes a post-commit occurrence: L has no pending, uncommitted, or aborted-transaction

component. In a transaction-aware instance, a committed transaction contributes its complete contiguous row-event block only after commit. A list prefix inside such a block is an intermediate replay prefix of already-committed events, not a source state during the commit. Transaction labels and the predicate that a prefix is a block boundary are deliberately instance-side; the unlabelled core needs neither for its per-key theorem. Positions, not event values, are the occurrence identity: a log may contain equal events at distinct positions, and the development reasons at position level throughout.

- S-IMG (image sufficiency): an event carries the full post-image of its key (*ev-img* e , *None* for a delete), and the source dynamics *apply-ev* makes that image the post-state by definition. Delta logs are outside this core (S-IMG); their re-read repair is a log normalization producing effective full-image events, not an in-core case.
- Coordinates denote nested prefixes: a coordinate space is a type $'c$ together with a denotation into prefixes of the log, represented here by prefix length. Linear nestedness of denoted prefixes is a theorem of this representation (*den-nested*), and the derived relation $\sqsubseteq$ agrees with prefix inclusion of denotations (*cle-iff-prefix*). No total order on the coordinate type is assumed; $\sqsubseteq$ is a preorder, and distinct coordinates may denote the same prefix. The operational-fidelity admissibility clause is an instance-level obligation: an instance must prove that its own operational window-membership decisions agree with its declared denotation (discharged per instance; the write-free instance's executed-GTID-set oracle-agreement lemma is the paradigm case).
- Half-open windows: the window of $lo \sqsubseteq hi$ is the log segment strictly after the events of *px* lo up to and including position *den* $hi - 1$: the list difference of the two denoted prefixes. Low edge excluded, high edge included, uniformly for every instance; an instance whose raw endpoint records use another convention declares its endpoint-to-prefix normalization before using windows.

1.1 Events, logs, and source states

datatype $('k, 'v)$ *event* = *Event* (*ev-key*: $'k$) (*ev-img*: $'v$ option)

type-synonym $('k, 'v)$ *state* = $'k \Rightarrow 'v$ option

definition *apply-ev* :: $('k, 'v)$ *state* $\Rightarrow$ $('k, 'v)$ *event* $\Rightarrow$ $('k, 'v)$ *state* **where**
apply-ev σ e = $\sigma(\text{ev-key } e := \text{ev-img } e)$

definition *state-after* :: $('k, 'v)$ *state* $\Rightarrow$ $('k, 'v)$ *event list* $\Rightarrow$ $('k, 'v)$ *state* **where**

$state_after\ \sigma 0\ P = foldl\ apply_ev\ \sigma 0\ P$

$state_after\ \sigma 0\ P$ is the contract's σ_P : the row-event replay state after the committed-log prefix P , over the initial state $\sigma 0$. In a transaction-aware instance it is also a committed database state when P ends at a transaction boundary.

definition $evs_for :: 'k \Rightarrow ('k, 'v)\ event\ list \Rightarrow ('k, 'v)\ event\ list$ **where**
 $evs_for\ k\ P = filter\ (\lambda e. ev_key\ e = k)\ P$

lemma evs_for_Nil [simp]: $evs_for\ k\ [] = []$
by (simp add: evs-for-def)

lemma evs_for_append [simp]:
 $evs_for\ k\ (P @ Q) = evs_for\ k\ P @ evs_for\ k\ Q$
by (simp add: evs-for-def)

lemma $state_after_Nil$ [simp]: $state_after\ \sigma 0\ [] = \sigma 0$
by (simp add: state-after-def)

lemma $state_after_append$:
 $state_after\ \sigma 0\ (P @ Q) = state_after\ (state_after\ \sigma 0\ P)\ Q$
by (simp add: state-after-def)

lemma $state_after_snoc$:
 $state_after\ \sigma 0\ (P @ [e]) = apply_ev\ (state_after\ \sigma 0\ P)\ e$
by (simp add: state-after-def)

The per-key characterization of the source state: the last event for the key decides, and absent any event the initial state persists. This is the shape every argument below reduces to.

lemma $state_after_key$:
 $state_after\ \sigma 0\ P\ k =$
 $(if\ evs_for\ k\ P = []\ then\ \sigma 0\ k\ else\ ev_img\ (last\ (evs_for\ k\ P)))$

proof (induction P rule: rev-induct)

case Nil

then show ?case **by** simp

next

case (snoc $e\ P$)

show ?case

proof (cases $ev_key\ e = k$)

case $True$

then show ?thesis

by (simp add: state-after-snoc apply-ev-def evs-for-def)

next

case $False$

then show ?thesis

using snoc.IH **by** (simp add: state-after-snoc apply-ev-def evs-for-def)

qed

qed

lemma *state-after-no-events*:

evs-for k $P = [] \implies \text{state-after } \sigma 0 P k = \sigma 0 k$
by (*simp add: state-after-key*)

lemma *state-after-last-event*:

evs-for k $P \neq [] \implies \text{state-after } \sigma 0 P k = \text{ev-img } (\text{last } (\text{evs-for } k P))$
by (*simp add: state-after-key*)

lemma *state-after-append-no-k*:

evs-for k $Q = [] \implies \text{state-after } \sigma 0 (P @ Q) k = \text{state-after } \sigma 0 P k$
by (*simp add: state-after-append state-after-no-events*)

1.2 Coordinate spaces: prefix denotations

A coordinate space over a log L : an arbitrary coordinate type $'c$ with a denotation *den* giving, for each coordinate, the length of the log prefix it denotes. Prefixes of one list biject with prefix lengths, so this representation is exactly “denotations are prefixes of the log”; the bound *den-le-len* says a coordinate never denotes beyond the committed history under discussion.

locale *coordinate-space* =

fixes $L :: ('k, 'v) \text{ event list}$
and $\text{den} :: 'c \Rightarrow \text{nat}$
assumes *den-le-len*: $\bigwedge c. \text{den } c \leq \text{length } L$

begin

definition *pfx* :: $'c \Rightarrow ('k, 'v) \text{ event list}$ **where**

pfx $c = \text{take } (\text{den } c) L$

definition *cle* :: $'c \Rightarrow 'c \Rightarrow \text{bool}$ (**infix** $\sqsubseteq$ 50) **where**

$c \sqsubseteq c' \longleftrightarrow \text{den } c \leq \text{den } c'$

lemma *cle-refl* [*simp*]: $c \sqsubseteq c$

by (*simp add: cle-def*)

lemma *cle-trans* [*trans*]: $c \sqsubseteq c' \implies c' \sqsubseteq c'' \implies c \sqsubseteq c''$

by (*simp add: cle-def*)

lemma *cle-total*: $c \sqsubseteq c' \vee c' \sqsubseteq c$

by (*simp add: cle-def nat-le-linear*)

lemma *pfx-length* [*simp*]: $\text{length } (\text{pfx } c) = \text{den } c$

by (*simp add: pfx-def den-le-len*)

lemma *take-prefix-take*: $m \leq n \implies \text{prefix } (\text{take } m \text{ xs}) (\text{take } n \text{ xs})$

by (*metis min.absorb1 take-is-prefix take-take*)

The nestedness clause is a theorem of the representation, and $\sqsubseteq$ agrees with prefix inclusion of the denotations — the representation is faithful to

the contract's $c \sqsubseteq c' \equiv \llbracket c \rrbracket \subseteq \llbracket c' \rrbracket$.

lemma *cle-iff-prefix*: $c \sqsubseteq c' \longleftrightarrow \text{prefix } (\text{pfx } c) (\text{pfx } c')$

proof

assume $c \sqsubseteq c'$

then show $\text{prefix } (\text{pfx } c) (\text{pfx } c')$

by (*simp add: cle-def pfx-def take-prefix-take*)

next

assume $\text{prefix } (\text{pfx } c) (\text{pfx } c')$

then have $\text{length } (\text{pfx } c) \leq \text{length } (\text{pfx } c')$

by (*rule prefix-length-le*)

then show $c \sqsubseteq c'$ **by** (*simp add: cle-def*)

qed

lemma *den-nested*: $\text{prefix } (\text{pfx } c) (\text{pfx } c') \vee \text{prefix } (\text{pfx } c') (\text{pfx } c)$

by (*metis cle-def cle-iff-prefix nat-le-linear*)

1.3 Half-open windows and the membership oracle

The window $(lo, hi]$ as a log segment: the list difference of the denoted prefixes, kept in commit order with occurrence (position) identity preserved.

definition *win* :: $'c \Rightarrow 'c \Rightarrow ('k, 'v) \text{ event list}$ **where**

$\text{win } lo \ hi = \text{drop } (\text{den } lo) (\text{take } (\text{den } hi) \ L)$

lemma *pfx-split*: $lo \sqsubseteq hi \implies \text{pfx } hi = \text{pfx } lo @ \text{win } lo \ hi$

by (*metis append-take-drop-id cle-def min.absorb1 pfx-def take-take win-def*)

lemma *win-length*: $lo \sqsubseteq hi \implies \text{length } (\text{win } lo \ hi) = \text{den } hi - \text{den } lo$

by (*metis add-diff-cancel-left' length-append pfx-length pfx-split*)

lemma *win-empty-iff*: $lo \sqsubseteq hi \implies (\text{win } lo \ hi = [] \longleftrightarrow \text{den } lo = \text{den } hi)$

by (*metis cle-def diff-is-0-eq le-antisym length-0-conv win-length*)

lemma *win-split*: $lo \sqsubseteq mid \implies mid \sqsubseteq hi \implies \text{win } lo \ hi = \text{win } lo \ mid @ \text{win } mid \ hi$

by (*metis append.assoc cle-trans pfx-split same-append-eq*)

lemma *win-nth*:

$lo \sqsubseteq hi \implies p < \text{den } hi - \text{den } lo \implies \text{win } lo \ hi ! p = L ! (\text{den } lo + p)$

by (*simp add: win-def cle-def den-le-len add commute*)

The membership oracle's set reading: under distinct occurrences the window is exactly the set difference of the denoted prefixes — the paper's " $e \in \llbracket c' \rrbracket \setminus \llbracket c \rrbracket$ ". The development itself reasons at position level (where no distinctness is needed); this lemma ties the representation to the paper-level set phrasing.

lemma *oracle-set-reading*:

assumes *distinct* L **and** $lo \sqsubseteq hi$

shows $\text{set } (\text{win } lo \ hi) = \text{set } (\text{pfx } hi) - \text{set } (\text{pfx } lo)$

proof –
 have *split*: $\text{pfx } hi = \text{pfx } lo @ \text{win } lo \ hi$
 by (rule *pfx-split*[*OF assms*(2)])
 have *distinct* (*pfx hi*)
 using *assms*(1) by (simp add: *pfx-def*)
 with *split* have *disj*: $\text{set } (\text{pfx } lo) \cap \text{set } (\text{win } lo \ hi) = \{\}$
 by (metis *distinct-append*)
 from *split* have $\text{set } (\text{pfx } hi) = \text{set } (\text{pfx } lo) \cup \text{set } (\text{win } lo \ hi)$
 by (metis *set-append*)
 with *disj* show ?thesis by auto
qed

1.4 Per-key event selection over prefixes and windows

lemma *evs-for-pfx-split*:
 $lo \sqsubseteq hi \implies \text{evs-for } k \ (\text{pfx } hi) = \text{evs-for } k \ (\text{pfx } lo) @ \text{evs-for } k \ (\text{win } lo \ hi)$
 by (metis *evs-for-append pfx-split*)

lemma *evs-for-win-split*:
 $lo \sqsubseteq mid \implies mid \sqsubseteq hi \implies$
 $\text{evs-for } k \ (\text{win } lo \ hi) = \text{evs-for } k \ (\text{win } lo \ mid) @ \text{evs-for } k \ (\text{win } mid \ hi)$
 by (metis *evs-for-append win-split*)

Window invariance: if no event for a key falls in a window, the source state on that key is the same at every coordinate of the window — the fact that lets a bracket-locally valid read result propagate to any other coordinate of its bracket (O2 together with O3).

lemma *win-invariant-state*:
 assumes *lo-c*: $lo \sqsubseteq c$ and *c-hi*: $c \sqsubseteq hi$
 and *quiet*: $\text{evs-for } k \ (\text{win } lo \ hi) = []$
 shows $\text{state-after } \sigma 0 \ (\text{pfx } c) \ k = \text{state-after } \sigma 0 \ (\text{pfx } lo) \ k$

proof –
 have $\text{evs-for } k \ (\text{win } lo \ hi) = \text{evs-for } k \ (\text{win } lo \ c) @ \text{evs-for } k \ (\text{win } c \ hi)$
 by (rule *evs-for-win-split*[*OF lo-c c-hi*])
 with *quiet* have $\text{evs-for } k \ (\text{win } lo \ c) = []$
 by simp
 moreover have $\text{pfx } c = \text{pfx } lo @ \text{win } lo \ c$
 by (rule *pfx-split*[*OF lo-c*])
 ultimately show ?thesis
 by (metis *state-after-append-no-k*)
qed

lemma *win-invariant-state2*:
 assumes $lo \sqsubseteq c$ and $c \sqsubseteq hi$ and $lo \sqsubseteq c'$ and $c' \sqsubseteq hi$
 and $\text{evs-for } k \ (\text{win } lo \ hi) = []$
 shows $\text{state-after } \sigma 0 \ (\text{pfx } c) \ k = \text{state-after } \sigma 0 \ (\text{pfx } c') \ k$
 using *assms win-invariant-state* by metis

end

end

```
theory Cut-Theorem
  imports Contract-Base
begin
```

2 Capture plans, the contract, and the contract-level cut theorem

Capture plans as finite families of units (domain, bracket, refresh), the plan obligations O1–O5, the canonical per-key replay, the contract-level virtual-cut theorem, and Corollaries 1–3.

Obligation placement in the mechanization:

- O1 (coverage/disjointness) and O2 (bracket-local per-key existential validity) are locale assumptions below. Operational DBLog instances additionally construct refreshes from committed reads: they do not expose in-progress writes. If transaction labels are retained, the operational O2 witness is therefore a transaction boundary, as encoded explicitly by theory *ReadOnly-Instance*. The transaction-free core records only the extensional per-key equality consumed by the proof; an interior replay prefix is not thereby licensed as a dirty-read state.
- O3 (domain completeness) is carried by representation plus O2: a refresh is a total function $(\text{'}k, \text{'}v)$ *state*, so every key of the unit yields exactly one value-or-absence read result (“exactly one” is structural), and O2 quantifies over ALL keys of the unit’s domain, so a scan that silently missed a key cannot discharge O2 unless that key was genuinely absent at an in-bracket coordinate — absence-by-omission is only valid when it is true absence.
- O4 (declared external assertions) has no core-level content: it disciplines HOW an instance may discharge bracket/validity premises (by named assumption rather than by proof) and surfaces in the instance theories, never here.
- O5 (representation agreement) is carried by typing in the core: refresh values and log images land in the same $\text{'}k$ and $\text{'}v$ by construction. Its operational half (dump-file rendering vs log rendering) is an instance-side obligation discharged when an instance constructs the typed refresh.
- S-OBS: the model’s log L is the committed history the capture consumes; the fold below is defined from L directly, so the core theorem

needs no separate observability premise. Observability becomes real at the discipline level (theory *Merge-Disciplines*): an emission is constructed from a consumed segment, and the consumption-start premise $s_0 \sqsubseteq lo_i$ appears there. Because *sink-at* never mentions the frontier parameter, the fold-level results below are frontier-parametric; the observation-frontier bound is stated at each corollary.

2.1 Units and plans

```
record ('k, 'v, 'c) cunit =
  u-dom :: 'k set
  u-lo :: 'c
  u-hi :: 'c
  u-refresh :: ('k, 'v) state
```

A capture plan: a coordinate space plus a finite family of units, a scope, and a frontier. The family is a list — an indexed family in the paper’s sense (the final partition the run produced); positions are the family’s index set. This locale fixes only the signature; the obligations live in *capture-contract* below. The plan is deliberately silent about how refreshes were obtained, how units were scheduled, and how brackets were realized: none of those occur in the signature at all.

```
locale capture-plan = coordinate-space L den
  for L :: ('k, 'v) event list and den :: 'c  $\Rightarrow$  nat +
  fixes  $\sigma 0$  :: ('k, 'v) state
  and units :: ('k, 'v, 'c) cunit list
  and scope :: 'k set
  and f :: 'c
begin
```

```
definition the-unit :: 'k  $\Rightarrow$  ('k, 'v, 'c) cunit where
  the-unit k = (THE u. u  $\in$  set units  $\wedge$  k  $\in$  u-dom u)
```

The canonical replay, per key: the last event for the key in the half-open interval from the covering unit’s low bracket edge to the queried coordinate wins; absent any such event, the unit’s read result stands. This fold is stated at any coordinate g (the emission truncated at g): refreshes are placed at lo_i and later events win.

```
definition sink-at :: 'c  $\Rightarrow$  ('k, 'v) state where
  sink-at g k =
    (if evs-for k (win (u-lo (the-unit k)) g) = []
     then u-refresh (the-unit k) k
     else ev-img (last (evs-for k (win (u-lo (the-unit k)) g)))))
```

```
end
```

2.2 The contract

locale *capture-contract* = *capture-plan* *L* *den* $\sigma 0$ *units* *scope* *f*
for *L* :: ('k, 'v) *event list* **and** *den* :: 'c $\Rightarrow$ *nat*
and $\sigma 0$:: ('k, 'v) *state*
and *units* :: ('k, 'v, 'c) *cunit list*
and *scope* :: 'k *set*
and *f* :: 'c +
assumes *O1-cover*: $(\bigcup u \in \text{set } \text{units}. u\text{-dom } u) = \text{scope}$
and *O1-disjoint*: $\llbracket i < \text{length } \text{units}; j < \text{length } \text{units}; i \neq j \rrbracket \Rightarrow$
 $u\text{-dom } (\text{units} ! i) \cap u\text{-dom } (\text{units} ! j) = \{\}$
and *bracket-lo-hi*: $u \in \text{set } \text{units} \Rightarrow u\text{-lo } u \sqsubseteq u\text{-hi } u$
and *bracket-hi-f*: $u \in \text{set } \text{units} \Rightarrow u\text{-hi } u \sqsubseteq f$
and *O2-honest*: $\llbracket u \in \text{set } \text{units}; k \in u\text{-dom } u \rrbracket \Rightarrow$
 $\exists c. u\text{-lo } u \sqsubseteq c \wedge c \sqsubseteq u\text{-hi } u \wedge$
 $u\text{-refresh } u \ k = \text{state-after } \sigma 0 \ (pfx \ c) \ k$
begin

lemma *dom-sub-scope*: $u \in \text{set } \text{units} \Rightarrow u\text{-dom } u \subseteq \text{scope}$
using *O1-cover* **by** *auto*

lemma *covering-unit-unique*:
assumes *u*: $u \in \text{set } \text{units} \ k \in u\text{-dom } u$
and *u'*: $u' \in \text{set } \text{units} \ k \in u\text{-dom } u'$
shows $u = u'$
proof (*rule ccontr*)
assume *ne*: $u \neq u'$
obtain *i* **where** *i*: $i < \text{length } \text{units} \ \text{units} ! i = u$
using *u(1)* **by** (*meson in-set-conv-nth*)
obtain *j* **where** *j*: $j < \text{length } \text{units} \ \text{units} ! j = u'$
using *u'(1)* **by** (*meson in-set-conv-nth*)
have $i \neq j$ **using** *ne i(2) j(2)* **by** *auto*
then have $u\text{-dom } (\text{units} ! i) \cap u\text{-dom } (\text{units} ! j) = \{\}$
using *O1-disjoint i(1) j(1)* **by** *blast*
with *i(2) j(2) u(2) u'(2)* **show** *False* **by** *auto*
qed

lemma *the-unit-props*:
assumes $k \in \text{scope}$
shows *the-unit* $k \in \text{set } \text{units}$ **and** $k \in u\text{-dom } (\text{the-unit } k)$
proof –
from *assms O1-cover* **obtain** *u* **where** $u: u \in \text{set } \text{units} \ k \in u\text{-dom } u$
by *auto*
have *the-unit* $k = u$
unfolding *the-unit-def*
proof (*rule the-equality*)
show $u \in \text{set } \text{units} \wedge k \in u\text{-dom } u$ **using** *u* **by** *simp*
next
fix *x* **assume** $x \in \text{set } \text{units} \wedge k \in u\text{-dom } x$
then show $x = u$ **using** *u covering-unit-unique* **by** *blast*

```

qed
then show the-unit  $k \in \text{set units } k \in u\text{-dom } (\text{the-unit } k)$ 
  using u by auto
qed

```

```

lemma the-unit-position-unique:
  assumes  $k: k \in \text{scope}$ 
    and  $i: i < \text{length units } \text{units} ! i = \text{the-unit } k$ 
    and  $j: j < \text{length units } \text{units} ! j = \text{the-unit } k$ 
  shows  $i = j$ 
proof (rule ccontr)
  assume  $i \neq j$ 
  then have  $u\text{-dom } (\text{units} ! i) \cap u\text{-dom } (\text{units} ! j) = \{\}$ 
    using O1-disjoint  $i(1) j(1)$  by blast
  then have  $u\text{-dom } (\text{the-unit } k) = \{\}$ 
    using  $i(2) j(2)$  by auto
  then show False
    using the-unit-props(2)[OF  $k$ ] by auto
qed

```

2.3 The contract-level cut theorem

The frontier-parametric workhorse: at ANY coordinate g that bounds every unit's high bracket edge, the canonical replay equals the source state. The proof is per-key local: Case A (some event for the key after its unit's low edge) — the last such event wins the fold and is also the last event for the key in the whole prefix; Case B (no such event) — O2 supplies an in-bracket coordinate where the read result is valid, and window invariance propagates its value to g . Nothing couples distinct units beyond O1 disjointness and the shared bound.

```

theorem contract-cut-at:
  assumes bound:  $\bigwedge u. u \in \text{set units} \implies u\text{-hi } u \sqsubseteq g$ 
    and  $k: k \in \text{scope}$ 
  shows sink-at  $g$   $k = \text{state-after } \sigma 0 \text{ (pfx } g) k$ 
proof -
  define u where  $u = \text{the-unit } k$ 
  have u-in:  $u \in \text{set units}$  and k-dom:  $k \in u\text{-dom } u$ 
    using the-unit-props[OF  $k$ ] u-def by auto
  have lo-hi:  $u\text{-lo } u \sqsubseteq u\text{-hi } u$ 
    by (rule bracket-lo-hi[OF u-in])
  have hi-g:  $u\text{-hi } u \sqsubseteq g$ 
    by (rule bound[OF u-in])
  have lo-g:  $u\text{-lo } u \sqsubseteq g$ 
    using lo-hi hi-g by (rule cle-trans)
  show ?thesis
proof (cases evs-for  $k$  (win ( $u\text{-lo } u$ )  $g$ ) = [])
  case False
  have split: evs-for  $k$  (pfx  $g$ ) =

```

```

      evs-for k (pfx (u-lo u)) @ evs-for k (win (u-lo u) g)
    by (rule evs-for-pfx-split[OF lo-g])
  have sink-at g k = ev-img (last (evs-for k (win (u-lo u) g)))
    using False by (simp add: sink-at-def u-def)
  moreover have evs-for k (pfx g) ≠ []
    using split False by auto
  moreover have last (evs-for k (pfx g)) = last (evs-for k (win (u-lo u) g))
    using split False by (simp add: last-append)
  ultimately show ?thesis
    by (simp add: state-after-last-event)
next
case True
from O2-honest[OF u-in k-dom] obtain c where
  c-lo: u-lo u ⊆ c and c-hi: c ⊆ u-hi u and
  honest: u-refresh u k = state-after σ 0 (pfx c) k
  by blast
have c-g: c ⊆ g
  using c-hi hi-g by (rule cle-trans)
have sink-at g k = u-refresh u k
  using True by (simp add: sink-at-def u-def)
also have ... = state-after σ 0 (pfx c) k
  by (rule honest)
also have ... = state-after σ 0 (pfx (u-lo u)) k
  by (rule win-invariant-state[OF c-lo c-g True])
also have ... = state-after σ 0 (pfx g) k
  by (rule win-invariant-state[OF lo-g cle-refl True, symmetric])
finally show ?thesis .
qed
qed

```

The per-unit form (paper Lemma 4.3, the closed-unit cut): the bound above is consumed only for the key's own unit, so at any coordinate g the canonical replay is exact on every key whose unit has closed by g , whatever the other units are doing.

theorem *contract-cut-unit*:

```

assumes k: k ∈ scope
  and hi: u-hi (the-unit k) ⊆ g
shows sink-at g k = state-after σ 0 (pfx g) k
proof -
  define u where u = the-unit k
  have u-in: u ∈ set units and k-dom: k ∈ u-dom u
    using the-unit-props[OF k] u-def by auto
  have lo-hi: u-lo u ⊆ u-hi u
    by (rule bracket-lo-hi[OF u-in])
  have hi-g: u-hi u ⊆ g
    using hi u-def by simp
  have lo-g: u-lo u ⊆ g
    using lo-hi hi-g by (rule cle-trans)
  show ?thesis

```

```

proof (cases evs-for k (win (u-lo u) g) = [])
  case False
  have split: evs-for k (pfx g) =
    evs-for k (pfx (u-lo u)) @ evs-for k (win (u-lo u) g)
    by (rule evs-for-pfx-split[OF lo-g])
  have sink-at g k = ev-img (last (evs-for k (win (u-lo u) g)))
    using False by (simp add: sink-at-def u-def)
  moreover have evs-for k (pfx g) ≠ []
    using split False by auto
  moreover have last (evs-for k (pfx g)) = last (evs-for k (win (u-lo u) g))
    using split False by (simp add: last-append)
  ultimately show ?thesis
    by (simp add: state-after-last-event)
next
  case True
  from O2-honest[OF u-in k-dom] obtain c where
    c-lo: u-lo u ⊆ c and c-hi: c ⊆ u-hi u and
    honest: u-refresh u k = state-after σ 0 (pfx c) k
    by blast
  have c-g: c ⊆ g
    using c-hi hi-g by (rule cle-trans)
  have sink-at g k = u-refresh u k
    using True by (simp add: sink-at-def u-def)
  also have ... = state-after σ 0 (pfx c) k
    by (rule honest)
  also have ... = state-after σ 0 (pfx (u-lo u)) k
    by (rule win-invariant-state[OF c-lo c-g True])
  also have ... = state-after σ 0 (pfx g) k
    by (rule win-invariant-state[OF lo-g cle-refl True, symmetric])
  finally show ?thesis .
qed
qed

```

The cut theorem: the replay of the emission is exactly the source state at the frontier, on the whole scope.

theorem *contract-cut*:

assumes $k \in \text{scope}$
shows $\text{sink-at } f \ k = \text{state-after } \sigma 0 \ (pfx \ f) \ k$
by (rule contract-cut-at[OF bracket-hi-f assms])

corollary *contract-cut-scope*:

$\forall k \in \text{scope}. \text{sink-at } f \ k = \text{state-after } \sigma 0 \ (pfx \ f) \ k$
using contract-cut **by** blast

2.4 The conservative trajectory after the latest high

The workhorse theorem already gives more than one correct state at f . A nonempty plan is a finite list, and coordinate denotations are totally ordered by prefix length. The recorded high edges therefore contain a maximal one

h . Every coordinate at or after h bounds every unit high, so *contract-cut-at* gives the source trajectory from h onward. This is the conservative trajectory available to every nonempty contract plan. A shared state witness can move the start back to a particular common read coordinate, but it is not needed for the existence of some trajectory start.

If the unit list is empty, O1 forces an empty scope. The scoped equality is then vacuous, but there is no recorded high edge to select.

lemma *latest-high-exists*:

```

assumes units-ne:  $\text{units} \neq []$ 
obtains  $h$  where  $h \in \text{u-hi } \text{'set units}$ 
    and  $\bigwedge u. u \in \text{set units} \implies \text{u-hi } u \sqsubseteq h$ 
proof –
  let  $?N = \text{den } \text{'u-hi } \text{'set units}$ 
  have fin: finite  $?N$  by simp
  have ne:  $?N \neq \{\}$  using units-ne by simp
  have max-in:  $\text{Max } ?N \in ?N$ 
    by (rule Max-in[OF fin ne])
  then obtain  $u0$  where u0-mem:  $u0 \in \text{set units}$ 
    and max-eq:  $\text{Max } ?N = \text{den } (\text{u-hi } u0)$ 
    by auto
  let  $?h = \text{u-hi } u0$ 
  have h-mem:  $?h \in \text{u-hi } \text{'set units}$ 
    using u0-mem by blast
  have h-den:  $\text{den } ?h = \text{Max } ?N$ 
    using max-eq by simp
  have bound:  $\bigwedge u. u \in \text{set units} \implies \text{u-hi } u \sqsubseteq ?h$ 
  proof –
    fix  $u$  assume u-mem:  $u \in \text{set units}$ 
    have  $\text{den } (\text{u-hi } u) \leq \text{Max } ?N$ 
      by (rule Max-ge[OF fin]) (use u-mem in auto)
    then show  $\text{u-hi } u \sqsubseteq ?h$ 
      by (simp add: cle-def h-den)
  qed
  show thesis by (rule that[OF h-mem bound])
qed

```

theorem *latest-high-trajectory*:

```

assumes units-ne:  $\text{units} \neq []$ 
obtains  $h$  where
   $h \in \text{u-hi } \text{'set units}$ 
   $\bigwedge u. u \in \text{set units} \implies \text{u-hi } u \sqsubseteq h$ 
   $h \sqsubseteq f$ 
   $\bigwedge g k. h \sqsubseteq g \implies g \sqsubseteq f \implies k \in \text{scope} \implies$ 
     $\text{sink-at } g k = \text{state-after } \sigma 0 \text{ (pf } x \text{ } g) k$ 
proof –
  from latest-high-exists[OF units-ne] obtain  $h$  where h-mem:  $h \in \text{u-hi } \text{'set units}$ 
    and h-max:  $\bigwedge u. u \in \text{set units} \implies \text{u-hi } u \sqsubseteq h$ 
    by blast

```

```

have  $h\text{-}f$ :  $h \sqsubseteq f$ 
proof -
  from  $h\text{-}mem$  obtain  $u$  where  $u \in \text{set units } h = u\text{-}hi \ u$  by auto
  then show  $?thesis$  using  $bracket\text{-}hi\text{-}f$  by simp
qed
have  $traj$ :  $\bigwedge g \ k. h \sqsubseteq g \implies g \sqsubseteq f \implies k \in \text{scope} \implies$ 
   $sink\text{-}at \ g \ k = \text{state}\text{-}after \ \sigma 0 \ (pfx \ g) \ k$ 
proof -
  fix  $g \ k$ 
  assume  $h\text{-}g$ :  $h \sqsubseteq g$  and  $g \sqsubseteq f$  and  $k$ :  $k \in \text{scope}$ 
  show  $sink\text{-}at \ g \ k = \text{state}\text{-}after \ \sigma 0 \ (pfx \ g) \ k$ 
  proof (rule  $contract\text{-}cut\text{-}at[OF \ - \ k]$ )
    fix  $u$  assume  $u \in \text{set units}$ 
    then have  $u\text{-}hi \ u \sqsubseteq h$  by (rule  $h\text{-}max$ )
    then show  $u\text{-}hi \ u \sqsubseteq g$  using  $h\text{-}g$  by (rule  $cle\text{-}trans$ )
  qed
qed
show  $thesis$  by (rule  $that[OF \ h\text{-}mem \ h\text{-}max \ h\text{-}f \ traj]$ )
qed

```

2.5 Corollary 1: exhibited shared-witness strengthening (T3)

The engine lemma, frontier-parametric: with one coordinate $cstar$ that is a shared state witness for the whole plan — in every bracket, and every read result valid AT it — the replay at every $g \sqsupseteq cstar$ equals the source state at g : the sink trajectory coincides with the source trajectory from that particular witness onward. This strengthens *latest-high-trajectory* by supplying a common read-state anchor and, when the witness precedes the latest high, an earlier trajectory interval. It does not create trajectory existence from nothing. Case A keys never consult the refresh; Case B keys are quiet from the witness to g , so the witness value is the g value.

lemma *shared-witness-trajectory-at*:

```

assumes  $wit\text{-}bracket$ :  $\bigwedge u. u \in \text{set units} \implies u\text{-}lo \ u \sqsubseteq cstar \wedge cstar \sqsubseteq u\text{-}hi \ u$ 
  and  $wit\text{-}honest$ :  $\bigwedge u \ k. u \in \text{set units} \implies k \in u\text{-}dom \ u \implies$ 
     $u\text{-}refresh \ u \ k = \text{state}\text{-}after \ \sigma 0 \ (pfx \ cstar) \ k$ 
  and  $cg$ :  $cstar \sqsubseteq g$ 
  and  $k$ :  $k \in \text{scope}$ 
shows  $sink\text{-}at \ g \ k = \text{state}\text{-}after \ \sigma 0 \ (pfx \ g) \ k$ 
proof -
  define  $u$  where  $u = \text{the}\text{-}unit \ k$ 
  have  $u\text{-}in$ :  $u \in \text{set units}$  and  $k\text{-}dom$ :  $k \in u\text{-}dom \ u$ 
  using  $\text{the}\text{-}unit\text{-}props[OF \ k] \ u\text{-}def$  by auto
  have  $lo\text{-}cs$ :  $u\text{-}lo \ u \sqsubseteq cstar$ 
  using  $wit\text{-}bracket[OF \ u\text{-}in]$  by auto
  have  $lo\text{-}g$ :  $u\text{-}lo \ u \sqsubseteq g$ 
  using  $lo\text{-}cs \ cg$  by (rule  $cle\text{-}trans$ )
  show  $?thesis$ 

```

```

proof (cases evs-for k (win (u-lo u) g) = [])
  case False
  have split: evs-for k (pfx g) =
    evs-for k (pfx (u-lo u)) @ evs-for k (win (u-lo u) g)
    by (rule evs-for-pfx-split[OF lo-g])
  have sink-at g k = ev-img (last (evs-for k (win (u-lo u) g)))
    using False by (simp add: sink-at-def u-def)
  moreover have evs-for k (pfx g) ≠ []
    using split False by auto
  moreover have last (evs-for k (pfx g)) = last (evs-for k (win (u-lo u) g))
    using split False by (simp add: last-append)
  ultimately show ?thesis
    by (simp add: state-after-last-event)
next
  case True
  have quiet-cs-g: evs-for k (win cstar g) = []
    using evs-for-win-split[OF lo-cs cg] True by auto
  have sink-at g k = u-refresh u k
    using True by (simp add: sink-at-def u-def)
  also have ... = state-after σ 0 (pfx cstar) k
    by (rule wit-honest[OF u-in k-dom])
  also have ... = state-after σ 0 (pfx g) k
    by (rule win-invariant-state[OF cg cle-refl quiet-cs-g, symmetric])
  finally show ?thesis .
qed
qed

```

Corollary 1: the trajectory claim runs from the exhibited shared-state anchor $cstar$ up to the observation frontier f ; beyond f it extends exactly as far as S-OBS is extended (Corollary 2's premise), never for free. The $g \sqsubseteq f$ premise scopes the claim to what the capture has observed; the fold-level engine above is what discharges it.

corollary *shared-witness-trajectory*:

```

assumes  $\bigwedge u. u \in \text{set units} \implies u\text{-lo } u \sqsubseteq cstar \wedge cstar \sqsubseteq u\text{-hi } u$ 
  and  $\bigwedge u k. u \in \text{set units} \implies k \in u\text{-dom } u \implies$ 
     $u\text{-refresh } u k = \text{state-after } \sigma 0 (pfx cstar) k$ 
  and  $cstar \sqsubseteq g$  and  $g \sqsubseteq f$  and  $k \in \text{scope}$ 
shows  $\text{sink-at } g k = \text{state-after } \sigma 0 (pfx g) k$ 
using assms shared-witness-trajectory-at by blast

```

2.6 Corollary 2: continuation under extended observation

Corollary 2: if S-OBS holds with f replaced by f' — the capture keeps consuming — then replay to f' equals the source state there: pure log replay preserves the cut. The extension premise $f \sqsubseteq f'$ is genuine (it is what makes every bracket bound f'); a capture that stops observing at f claims nothing beyond f .

theorem *continuation*:

```

    assumes  $\text{ff}' : f \sqsubseteq f'$ 
    and  $k : k \in \text{scope}$ 
    shows  $\text{sink-at } f' \ k = \text{state-after } \sigma 0 \ (\text{pfx } f') \ k$ 
  proof (rule contract-cut-at[ $OF - k$ ])
    fix  $u$  assume  $u \in \text{set units}$ 
    then have  $u\text{-hi } u \sqsubseteq f$  by (rule bracket-hi-f)
    then show  $u\text{-hi } u \sqsubseteq f'$  using  $\text{ff}'$  by (rule cle-trans)
  qed

end

```

2.7 Corollary 3: mixing (heterogeneous composition)

Corollary 3: the theorem quantifies over an arbitrary finite plan with per-unit brackets and arbitrary per-unit refresh provenance — heterogeneous bootstrap is the theorem’s normal case, not a special case. The composition content is mechanized as: two contract instances over the same source with disjoint scopes union to a contract instance (whose cut is then *contract-cut*), and a witness shared by ALL members lifts the composite to Corollary 1 (the premises decompose over the union). Members with distinct exhibited witnesses compose conservatively to T2. The union still has the trajectory supplied by *latest-high-trajectory*, beginning at its latest high, but composition alone supplies neither one common in-bracket state witness nor an earlier start. The tablesync fixture in theory *Contract-Witnesses* shows this automatic guarantee is sharp.

```

context coordinate-space
begin

```

theorem *mixing-union*:

```

    assumes  $A : \text{capture-contract } L \text{ den } \sigma 0 \ us1 \ sc1 \ f$ 
    and  $B : \text{capture-contract } L \text{ den } \sigma 0 \ us2 \ sc2 \ f$ 
    and  $\text{disj} : sc1 \cap sc2 = \{\}$ 
    shows  $\text{capture-contract } L \text{ den } \sigma 0 \ (us1 \ @ \ us2) \ (sc1 \cup sc2) \ f$ 
  proof -
    interpret  $a : \text{capture-contract } L \text{ den } \sigma 0 \ us1 \ sc1 \ f$  by (rule  $A$ )
    interpret  $b : \text{capture-contract } L \text{ den } \sigma 0 \ us2 \ sc2 \ f$  by (rule  $B$ )
    show ?thesis
  proof (unfold locales)
    show  $(\bigcup_{u \in \text{set } (us1 \ @ \ us2)} u\text{-dom } u) = sc1 \cup sc2$ 
    using  $a.O1\text{-cover } b.O1\text{-cover}$  by auto
  next
    fix  $i \ j$ 
    assume  $i : i < \text{length } (us1 \ @ \ us2)$  and  $j : j < \text{length } (us1 \ @ \ us2)$ 
    and  $ne : i \neq j$ 
    consider
      (AA)  $i < \text{length } us1 \ j < \text{length } us1$ 
    | (AB)  $i < \text{length } us1 \ \neg j < \text{length } us1$ 
    | (BA)  $\neg i < \text{length } us1 \ j < \text{length } us1$ 

```

```

| (BB)  $\neg i < \text{length } us1 \neg j < \text{length } us1$ 
by blast
then show  $u\text{-dom } ((us1 @ us2) ! i) \cap u\text{-dom } ((us1 @ us2) ! j) = \{\}$ 
proof cases
case AA
then show ?thesis
using a.O1-disjoint[OF AA(1) AA(2) ne] by (simp add: nth-append)
next
case AB
have m1:  $(us1 @ us2) ! i \in \text{set } us1$ 
using AB(1) by (simp add: nth-append)
have m2:  $(us1 @ us2) ! j \in \text{set } us2$ 
using AB(2) j by (auto simp: nth-append)
show ?thesis
using a.dom-sub-scope[OF m1] b.dom-sub-scope[OF m2] disj by blast
next
case BA
have m1:  $(us1 @ us2) ! i \in \text{set } us2$ 
using BA(1) i by (auto simp: nth-append)
have m2:  $(us1 @ us2) ! j \in \text{set } us1$ 
using BA(2) by (simp add: nth-append)
show ?thesis
using a.dom-sub-scope[OF m2] b.dom-sub-scope[OF m1] disj by blast
next
case BB
have b1:  $i - \text{length } us1 < \text{length } us2$ 
using i BB(1) by auto
have b2:  $j - \text{length } us1 < \text{length } us2$ 
using j BB(2) by auto
have b3:  $i - \text{length } us1 \neq j - \text{length } us1$ 
using ne BB by auto
have  $u\text{-dom } (us2 ! (i - \text{length } us1)) \cap u\text{-dom } (us2 ! (j - \text{length } us1)) = \{\}$ 
by (rule b.O1-disjoint[OF b1 b2 b3])
then show ?thesis
using BB by (simp add: nth-append)
qed
next
fix u assume u  $\in \text{set } (us1 @ us2)$ 
then show  $u\text{-lo } u \sqsubseteq u\text{-hi } u$ 
using a.bracket-lo-hi b.bracket-lo-hi by auto
next
fix u assume u  $\in \text{set } (us1 @ us2)$ 
then show  $u\text{-hi } u \sqsubseteq f$ 
using a.bracket-hi-f b.bracket-hi-f by auto
next
fix u k assume u  $\in \text{set } (us1 @ us2)$  and  $k \in u\text{-dom } u$ 
then show  $\exists c. u\text{-lo } u \sqsubseteq c \wedge c \sqsubseteq u\text{-hi } u \wedge$ 
 $u\text{-refresh } u \ k = \text{state-after } \sigma 0 \ (pfx \ c) \ k$ 
using a.O2-honest b.O2-honest by auto

```

qed
qed

The shared-witness lift: Corollary 1’s premises over the union decompose over the append, so one witness shared by all members of both plans makes the composite plan satisfy Corollary 1’s premises outright. With distinct witnesses this particular decomposition is unavailable. The union retains its T2 cut and conservative latest-high trajectory, while a common witness or any earlier start needs separate evidence.

lemma *mixing-shared-witness-lift*:

assumes $\bigwedge u. u \in \text{set } us1 \implies u\text{-lo } u \sqsubseteq cstar \wedge cstar \sqsubseteq u\text{-hi } u$
and $\bigwedge u. u \in \text{set } us2 \implies u\text{-lo } u \sqsubseteq cstar \wedge cstar \sqsubseteq u\text{-hi } u$
shows $\bigwedge u. u \in \text{set } (us1 @ us2) \implies u\text{-lo } u \sqsubseteq cstar \wedge cstar \sqsubseteq u\text{-hi } u$
using *assms* **by** *auto*

lemma *mixing-shared-honesty-lift*:

assumes $\bigwedge u k. u \in \text{set } us1 \implies k \in u\text{-dom } u \implies$
 $u\text{-refresh } u k = \text{state-after } \sigma 0 \text{ (pfx } cstar) k$
and $\bigwedge u k. u \in \text{set } us2 \implies k \in u\text{-dom } u \implies$
 $u\text{-refresh } u k = \text{state-after } \sigma 0 \text{ (pfx } cstar) k$
shows $\bigwedge u k. u \in \text{set } (us1 @ us2) \implies k \in u\text{-dom } u \implies$
 $u\text{-refresh } u k = \text{state-after } \sigma 0 \text{ (pfx } cstar) k$
using *assms* **by** (*metis Un-iff set-append*)

end

end

theory *Merge-Disciplines*

imports *Cut-Theorem*

begin

3 Emitted streams and the window-discard equivalence

Real merge disciplines must meet an equivalence obligation: a discipline’s emitted stream must replay to the same *sink* as the canonical rule. This theory builds the stream substrate and discharges the obligation for the classic *window-discard* discipline — the engine of the classic bridge, and the discipline the write-free instance reuses.

Stream model: an emitted stream is a list of events — each item writes a key’s value-or-absence — replayed by the same per-key fold as the source dynamics but starting from the EMPTY map. The empty start is the stream-replay convention: a key never emitted denotes absence, so absence-by-omission IS the absence result. Refresh entries for absent keys are there-

fore never emitted, while delete EVENTS are (they arrive from the log).

The emission is built occurrence-tagged: each item carries the source version it denotes — an event at log position p is tagged $Suc\ p$ (the state version after it commits); a unit's surviving refresh entries are emitted when the unit's window closes, tagged with the close position. Tags make the window-discard per-key monotonicity claim a sortedness statement (*window-discard-monotone*); dropping tags gives the plain stream. S-OBS enters exactly here: the emission is constructed from the consumed segment starting at $s0$, and the discipline premise $s0 \sqsubseteq u\text{-}lo\ u$ for every unit is S-OBS's consumption-start clause; the construction is single-pass, which is the discipline's retention requirement.

3.1 List auxiliaries

lemma *upt-split*:

$a \leq b \implies b \leq c \implies [a..<c] = [a..<b] @ [b..<c]$
by (*metis le-Suc-ex upt-add-eq-append*)

lemma *sorted-filter'*:

$sorted\ xs \implies sorted\ (filter\ P\ xs)$
by (*induction xs*) *auto*

lemma *concat-map-upt-single*:

assumes $zero: \bigwedge i. i < n \implies i \neq i0 \implies g\ i = []$
and $i0: i0 < n$
shows $concat\ (map\ g\ [0..<n]) = g\ i0$

proof —

have *split*: $[0..<n] = [0..<i0] @ [i0..<n]$

using $i0$ **by** (*intro upt-split*) *linarith+*

have *head*: $[i0..<n] = i0 \# [Suc\ i0..<n]$

using $i0$ **by** (*simp add: upt-rec*)

have *all-nil*: $\bigwedge xs. (\bigwedge i. i \in set\ xs \implies g\ i = []) \implies concat\ (map\ g\ xs) = []$

proof —

fix $xs :: nat\ list$

assume $\bigwedge i. i \in set\ xs \implies g\ i = []$

then show $concat\ (map\ g\ xs) = []$

by (*induction xs*) *auto*

qed

have *left*: $concat\ (map\ g\ [0..<i0]) = []$

by (*rule all-nil*) (*use zero i0 in auto*)

have *right*: $concat\ (map\ g\ [Suc\ i0..<n]) = []$

by (*rule all-nil*) (*use zero in auto*)

have $concat\ (map\ g\ [0..<n]) = concat\ (map\ g\ [0..<i0]) @ concat\ (map\ g\ [i0..<n])$

unfolding *split* **by** (*simp only: map-append concat-append*)

also have $\dots = concat\ (map\ g\ [0..<i0]) @ g\ i0 @ concat\ (map\ g\ [Suc\ i0..<n])$

unfolding *head* **by** (*simp only: list.map(2) concat.simps(2)*)

also have $\dots = g\ i0$

unfolding *left right* **by** *simp*

finally show *?thesis* .
qed

lemma *concat-if-map-filter*:
 $\text{concat } (\text{map } (\lambda p. \text{if } Q \text{ } p \text{ then } [h \text{ } p] \text{ else } [])) \text{ } ps = \text{map } h \text{ } (\text{filter } Q \text{ } ps)$
by (*induction ps*) *auto*

lemma *filter-concat-map*:
 $\text{filter } P \text{ } (\text{concat } (\text{map } g \text{ } xs)) = \text{concat } (\text{map } (\text{filter } P \circ g) \text{ } xs)$
by (*induction xs*) *auto*

lemma *distinct-filter-eq*:
 $\text{distinct } ks \implies \text{filter } (\lambda x. x = k) \text{ } ks = (\text{if } k \in \text{set } ks \text{ then } [k] \text{ else } [])$
by (*induction ks*) *auto*

lemma *sorted-map-Suc*: $\text{sorted } xs \implies \text{sorted } (\text{map } \text{Suc } xs)$
by (*induction xs*) *auto*

3.2 Stream replay and position normal forms

definition *stream-replay* :: (k, v) event list $\Rightarrow (k, v)$ state **where**
 $\text{stream-replay } S = \text{state-after } (\lambda -. \text{None}) \text{ } S$

The generic final-output lift separates two proof boundaries. The contract theorem establishes the canonical replay at the frontier. A merge discipline separately establishes that its finite emitted list has the same final replay, from the empty replay state, on every scoped key. Composing those equalities gives the source state at the frontier. This statement concerns final replay only. It does not identify physical prefixes of the emitted list with canonical replay prefixes, and it carries no transport, populated-sink, or exactly-once claim.

context *capture-contract*
begin

theorem *emitted-replay-cut*:
assumes *equiv*: $\bigwedge k. k \in \text{scope} \implies \text{stream-replay } E \text{ } k = \text{sink-at } f \text{ } k$
and *k*: $k \in \text{scope}$
shows $\text{stream-replay } E \text{ } k = \text{state-after } \sigma 0 \text{ } (\text{pfx } f) \text{ } k$
using *equiv*[*OF k*] *contract-cut*[*OF k*] **by** *simp*

corollary *emitted-replay-cut-scope*:
assumes *equiv*: $\bigwedge k. k \in \text{scope} \implies \text{stream-replay } E \text{ } k = \text{sink-at } f \text{ } k$
shows $\forall k \in \text{scope}. \text{stream-replay } E \text{ } k = \text{state-after } \sigma 0 \text{ } (\text{pfx } f) \text{ } k$
using *emitted-replay-cut*[*OF equiv*] **by** *blast*

end

context *coordinate-space*
begin

Positions of a key's events in a half-open position interval: the normal form every interleaving argument reduces to. *kposs* is sorted (a filtered enumeration), and windows and prefixes both project onto it.

definition *kposs* :: 'k $\Rightarrow$ nat $\Rightarrow$ nat $\Rightarrow$ nat list **where**
kposs k a b = filter (λp . ev-key (L ! p) = k) [a..**b**]

lemma *kposs-sorted*: sorted (*kposs* k a b)
unfolding *kposs-def* **by** (intro sorted-filter') simp

lemma *kposs-bounds*: $p \in \text{set } (kposs\ k\ a\ b) \implies a \leq p \wedge p < b$
by (auto simp: *kposs-def*)

lemma *kposs-split*:
 $a \leq b \implies b \leq c \implies kposs\ k\ a\ c = kposs\ k\ a\ b @ kposs\ k\ b\ c$
by (metis filter-append *kposs-def* upt-split)

lemma *win-map-nth*:
assumes *le*: $lo \sqsubseteq hi$
shows $win\ lo\ hi = \text{map } (!)\ L\ [\text{den } lo..**den } hi]**$
proof (rule *nth-equalityI*)
show $\text{length } (win\ lo\ hi) = \text{length } (\text{map } (!)\ L\ [\text{den } lo..**den } hi])**$
using *win-length[OF le]* **by** simp
fix *p* **assume** $p < \text{length } (win\ lo\ hi)$
then show $win\ lo\ hi\ !\ p = \text{map } (!)\ L\ [\text{den } lo..**den } hi]\ !\ p**$
using *win-nth[OF le]* *win-length[OF le]* **by** simp
qed

lemma *evs-for-win-kposs*:
 $lo \sqsubseteq hi \implies \text{evs-for } k\ (win\ lo\ hi) = \text{map } (!)\ L\ (kposs\ k\ (\text{den } lo)\ (\text{den } hi))$
by (simp add: *win-map-nth* *evs-for-def* *kposs-def* *filter-map* *comp-def*)

lemma *pfx-map-nth*: $pfx\ c = \text{map } (!)\ L\ [0..**den } c]**$
proof (rule *nth-equalityI*)
show $\text{length } (pfx\ c) = \text{length } (\text{map } (!)\ L\ [0..**den } c])**$
by simp
fix *p* **assume** $p < \text{length } (pfx\ c)$
then show $pfx\ c\ !\ p = \text{map } (!)\ L\ [0..**den } c]\ !\ p**$
by (simp add: *pfx-def* *den-le-len*)
qed

lemma *evs-for-pfx-kposs*:
 $\text{evs-for } k\ (pfx\ c) = \text{map } (!)\ L\ (kposs\ k\ 0\ (\text{den } c))$
by (simp add: *pfx-map-nth* *evs-for-def* *kposs-def* *filter-map* *comp-def*)

end

3.3 The window-discard emission

context *capture-plan*

begin

Survivors of a unit: keys of its domain that are present in the refresh AND untouched by the unit's own window — exactly the buffered rows the classic discipline still holds when the high watermark arrives. The emission takes a survivor ENUMERATION *svl* as a parameter (any duplicate-free listing of each unit's survivor set); real captures emit rows physically, so an enumeration always exists (*svl-exists* below), and no finiteness assumption on states is smuggled into the contract.

definition *survivors* :: ('k, 'v, 'c) cunit $\Rightarrow$ 'k set **where**
survivors u =
 $\{k \in u\text{-dom } u. u\text{-refresh } u \ k \neq \text{None} \wedge$
 $\text{evs-for } k \ (\text{win } (u\text{-lo } u) \ (u\text{-hi } u)) = []\}$

definition *svl-ok* :: (('k, 'v, 'c) cunit $\Rightarrow$ 'k list) $\Rightarrow$ bool **where**
svl-ok svl $\longleftrightarrow$
 $(\forall u \in \text{set units}. \text{distinct } (\text{svl } u) \wedge \text{set } (\text{svl } u) = \text{survivors } u)$

definition *close-block* ::
 $((('k, 'v, 'c) \text{ cunit} \Rightarrow 'k \text{ list}) \Rightarrow ('k, 'v, 'c) \text{ cunit})$
 $\Rightarrow (\text{nat} \times ('k, 'v) \text{ event}) \text{ list} \text{ **where**}$
close-block svl u =
 $\text{map } (\lambda k. (\text{den } (u\text{-hi } u), \text{Event } k \ (u\text{-refresh } u \ k))) \ (\text{svl } u)$

definition *closes-at* ::
 $((('k, 'v, 'c) \text{ cunit} \Rightarrow 'k \text{ list}) \Rightarrow \text{nat} \Rightarrow (\text{nat} \times ('k, 'v) \text{ event}) \text{ list} \text{ **where**}$
closes-at svl p =
 $\text{concat } (\text{map } (\lambda i. \text{if } \text{den } (u\text{-hi } (\text{units } ! \ i)) = p$
 $\text{then } \text{close-block } \text{svl } (\text{units } ! \ i) \text{ else } [])$
 $[0..<\text{length } \text{units}]])$

The tagged window-discard emission from consumption start *s0* to the frontier: a single pass over the consumed log segment; every event is passed through at its own position; when a position reaches a unit's high edge, that unit's survivors are emitted before the event at that position (their refresh entries are older than anything from the high edge onward). Units whose high edge sits at the frontier close after the last consumed event.

definition *emission-t* ::
 $((('k, 'v, 'c) \text{ cunit} \Rightarrow 'k \text{ list}) \Rightarrow 'c \Rightarrow (\text{nat} \times ('k, 'v) \text{ event}) \text{ list} \text{ **where**}$
emission-t svl s0 =
 $\text{concat } (\text{map } (\lambda p. \text{closes-at } \text{svl } p \ @ \ [(\text{Suc } p, L \ ! \ p)]) \ [\text{den } s0..<\text{den } f])$
 $\ @ \ \text{closes-at } \text{svl } (\text{den } f)$

definition *emission* ::
 $((('k, 'v, 'c) \text{ cunit} \Rightarrow 'k \text{ list}) \Rightarrow 'c \Rightarrow ('k, 'v) \text{ event list} \text{ **where**}$
emission svl s0 = $\text{map } \text{snd } (\text{emission-t } \text{svl } s0)$

definition *titems-for* ::

$'k \Rightarrow (\text{nat} \times ('k, 'v) \text{ event}) \text{ list} \Rightarrow (\text{nat} \times ('k, 'v) \text{ event}) \text{ list}$ **where**
 $\text{titems-for } k \ S = \text{filter } (\lambda x. \text{ev-key } (\text{snd } x) = k) \ S$

lemma *evs-for-map-snd*:

$\text{evs-for } k \ (\text{map } \text{snd } S) = \text{map } \text{snd } (\text{titems-for } k \ S)$

by (*simp add: evs-for-def titems-for-def filter-map comp-def*)

end

3.4 Per-key characterization of the emission

context *capture-contract*

begin

lemma *survivors-sub-dom*: $\text{survivors } u \subseteq u\text{-dom } u$

by (*auto simp: survivors-def*)

lemma *svl-exists*:

assumes *fin*: $\bigwedge u. u \in \text{set units} \implies \text{finite } (\text{survivors } u)$

shows $\exists \text{svl}. \text{svl-ok } \text{svl}$

proof –

define *svl* **where**

$\text{svl} = (\lambda u. \text{SOME } xs. \text{distinct } xs \wedge \text{set } xs = \text{survivors } u)$

have $\text{distinct } (\text{svl } u) \wedge \text{set } (\text{svl } u) = \text{survivors } u$

if *u*: $u \in \text{set units}$ **for** *u*

proof –

have $\exists xs. \text{distinct } xs \wedge \text{set } xs = \text{survivors } u$

using *finite-distinct-list*[*OF fin*[*OF u*]] **by** *metis*

then show *?thesis*

unfolding *svl-def* **by** (*rule someI-ex*)

qed

then have *svl-ok* *svl*

by (*simp add: svl-ok-def*)

then show *?thesis* **by** *blast*

qed

lemma *titems-close-block*:

assumes *svl*: *svl-ok* *svl* **and** *u-in*: $u \in \text{set units}$

shows $\text{titems-for } k \ (\text{close-block } \text{svl } u) =$

$(\text{if } k \in \text{set } (\text{svl } u)$

$\text{then } [(\text{den } (u\text{-hi } u), \text{Event } k \ (u\text{-refresh } u \ k))] \text{ else } [])$

proof –

have *dis*: $\text{distinct } (\text{svl } u)$

using *svl u-in* **by** (*simp add: svl-ok-def*)

have $\text{titems-for } k \ (\text{close-block } \text{svl } u) =$

$\text{map } (\lambda k'. (\text{den } (u\text{-hi } u), \text{Event } k' \ (u\text{-refresh } u \ k')))$

$(\text{filter } (\lambda k'. k' = k) (\text{svl } u))$

by (*simp add: titems-for-def close-block-def filter-map comp-def*)

then show *?thesis*

```

    using distinct-filter-eq[OF dis] by simp
qed

lemma titems-closes-at:
  assumes svl: svl-ok svl and k: k ∈ scope
  shows titems-for k (closes-at svl p) =
    (if den (u-hi (the-unit k)) = p ∧ k ∈ set (svl (the-unit k))
     then [(p, Event k (u-refresh (the-unit k) k))] else [])
proof -
  obtain i0 where i0: i0 < length units units ! i0 = the-unit k
    using the-unit-props(1)[OF k] by (meson in-set-conv-nth)
  have others: titems-for k (if den (u-hi (units ! i)) = p
    then close-block svl (units ! i) else []) = []
  if i: i < length units and ne: i ≠ i0 for i
proof -
  have ui-in: units ! i ∈ set units using i by simp
  have k ∉ set (svl (units ! i))
proof
  assume k ∈ set (svl (units ! i))
  then have k ∈ survivors (units ! i)
    using svl ui-in by (simp add: svl-ok-def)
  then have k ∈ u-dom (units ! i)
    using survivors-sub-dom by auto
  then have eq: units ! i = the-unit k
    using covering-unit-unique[OF ui-in - the-unit-props(1)[OF k]
      the-unit-props(2)[OF k]] by blast
  have i = i0
    by (rule the-unit-position-unique[OF k i eq i0(1) i0(2)])
  with ne show False ..
qed
then show ?thesis
  using titems-close-block[OF svl ui-in]
  by (simp add: titems-for-def)
qed
have titems-for k (closes-at svl p) =
  concat (map (λi. titems-for k (if den (u-hi (units ! i)) = p
    then close-block svl (units ! i) else []))
    [0..

```

The master characterization: a key's occurrences in the emission are

its consumed pre-close events, then (iff it survived) its unit's refresh entry at the close position, then its post-close events. Everything below — the equivalence and the monotonicity claim — reads off this shape.

lemma *titems-emission-master*:

assumes *svl*: *svl-ok svl*
and *s0lo*: $\bigwedge u. u \in \text{set units} \implies s0 \sqsubseteq u\text{-lo } u$
and *k*: $k \in \text{scope}$
shows *titems-for k (emission-t svl s0)* =
 $\text{map } (\lambda p. (\text{Suc } p, L ! p)) (k\text{poss } k (\text{den } s0) (\text{den } (u\text{-hi } (\text{the-unit } k))))$
 $\text{@ } (\text{if } k \in \text{set } (\text{svl } (\text{the-unit } k))$
 $\text{then } [(\text{den } (u\text{-hi } (\text{the-unit } k)),$
 $\text{Event } k (u\text{-refresh } (\text{the-unit } k) k))] \text{ else } [])$
 $\text{@ } \text{map } (\lambda p. (\text{Suc } p, L ! p)) (k\text{poss } k (\text{den } (u\text{-hi } (\text{the-unit } k))) (\text{den } f))$

proof –

define *u* **where** $u = \text{the-unit } k$
define *hp* **where** $hp = \text{den } (u\text{-hi } u)$
have *u-in*: $u \in \text{set units}$
using *the-unit-props*(1)[*OF k*] *u-def* **by** *simp*
have *s0-le-lo*: $\text{den } s0 \leq \text{den } (u\text{-lo } u)$
using *s0lo*[*OF u-in*] **by** (*simp add: cle-def*)
have *lo-le-hp*: $\text{den } (u\text{-lo } u) \leq hp$
using *bracket-lo-hi*[*OF u-in*] **by** (*simp add: cle-def hp-def*)
have *hp-le-f*: $hp \leq \text{den } f$
using *bracket-hi-f*[*OF u-in*] **by** (*simp add: cle-def hp-def*)
have *s0-le-hp*: $\text{den } s0 \leq hp$
using *s0-le-lo lo-le-hp* **by** *simp*

have *closes*: $\bigwedge p. \text{titems-for } k (\text{closes-at } svl p) =$
 $(\text{if } hp = p \wedge k \in \text{set } (\text{svl } u)$
 $\text{then } [(p, \text{Event } k (u\text{-refresh } u k))] \text{ else } [])$
using *titems-closes-at*[*OF svl k*] **by** (*simp add: u-def hp-def*)

have *dist*: *titems-for k (emission-t svl s0)* =
 $\text{concat } (\text{map } (\lambda p. \text{titems-for } k (\text{closes-at } svl p)$
 $\text{@ } (\text{if } \text{ev-key } (L ! p) = k \text{ then } [(\text{Suc } p, L ! p)] \text{ else } []))$
 $[\text{den } s0 .. < \text{den } f])$
 $\text{@ } \text{titems-for } k (\text{closes-at } svl (\text{den } f))$
unfolding *emission-t-def titems-for-def*
by (*simp add: filter-concat-map comp-def cong: map-cong if-cong*)

show *?thesis*

proof (*cases hp < den f*)
case *False*
then have *hp-f*: $hp = \text{den } f$
using *hp-le-f* **by** *simp*
have *no-close*: $\bigwedge p. p \in \text{set } [\text{den } s0 .. < \text{den } f] \implies$
 $\text{titems-for } k (\text{closes-at } svl p) = []$
using *closes hp-f* **by** *auto*
have *concat* ($\text{map } (\lambda p. \text{titems-for } k (\text{closes-at } svl p)$

```

      @ (if ev-key (L ! p) = k then [(Suc p, L ! p)] else [])
      [den s0..<den f])
    = concat (map (λp. if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
      [den s0..<den f])
    using no-close by (intro arg-cong[where f = concat] map-cong) auto
  also have ... = map (λp. (Suc p, L ! p)) (kposs k (den s0) (den f))
    by (simp add: concat-if-map-filter kposs-def)
  finally show ?thesis
    using dist closes hp-f
    by (simp add: u-def hp-def kposs-def)
next
case True
have split: [den s0..<den f] = [den s0..<hp] @ [hp..<den f]
  using s0-le-hp hp-le-f by (rule upt-split)
have head: [hp..<den f] = hp # [Suc hp..<den f]
  using True by (simp add: upt-rec)
have pre: concat (map (λp. titems-for k (closes-at svl p))
  @ (if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
  [den s0..<hp])
  = map (λp. (Suc p, L ! p)) (kposs k (den s0) hp)
proof -
  have ∧p. p ∈ set [den s0..<hp] ⇒ titems-for k (closes-at svl p) = []
    using closes by auto
  then have concat (map (λp. titems-for k (closes-at svl p))
    @ (if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
    [den s0..<hp])
    = concat (map (λp. if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
      [den s0..<hp])
    by (intro arg-cong[where f = concat] map-cong) auto
  then show ?thesis
    by (simp add: concat-if-map-filter kposs-def)
qed
have post: concat (map (λp. titems-for k (closes-at svl p))
  @ (if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
  [Suc hp..<den f])
  = map (λp. (Suc p, L ! p)) (kposs k (Suc hp) (den f))
proof -
  have ∧p. p ∈ set [Suc hp..<den f] ⇒ titems-for k (closes-at svl p) = []
    using closes by auto
  then have concat (map (λp. titems-for k (closes-at svl p))
    @ (if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
    [Suc hp..<den f])
    = concat (map (λp. if ev-key (L ! p) = k then [(Suc p, L ! p)] else []))
      [Suc hp..<den f])
    by (intro arg-cong[where f = concat] map-cong) auto
  then show ?thesis
    by (simp add: concat-if-map-filter kposs-def)
qed
have closes-hp: titems-for k (closes-at svl hp)

```

```

    = (if  $k \in \text{set } (svl \ u)$  then  $[(hp, \text{Event } k \ (u\text{-refresh } u \ k))]$  else  $[]$ )
  using closes[of hp] by simp
  have trail:  $\text{titems-for } k \ (\text{closes-at } s0 \ (\text{den } f)) = []$ 
  using closes True by auto
  have kposs-peel:  $kposs \ k \ hp \ (\text{den } f)$ 
    = (if  $\text{ev-key } (L \ ! \ hp) = k$  then  $hp \ \# \ kposs \ k \ (\text{Suc } hp) \ (\text{den } f)$ 
      else  $kposs \ k \ (\text{Suc } hp) \ (\text{den } f)$ )
  unfolding kposs-def using head by simp
  have peel:  $\text{titems-for } k \ (\text{emission-t } s0)$ 
    = concat (map ( $\lambda p. \text{titems-for } k \ (\text{closes-at } s0 \ p)$ 
      @ (if  $\text{ev-key } (L \ ! \ p) = k$  then  $[(\text{Suc } p, L \ ! \ p)]$  else  $[]$ )
      [den  $s0 \ .. < hp$ ])
      @  $\text{titems-for } k \ (\text{closes-at } s0 \ hp)$ 
      @ (if  $\text{ev-key } (L \ ! \ hp) = k$  then  $[(\text{Suc } hp, L \ ! \ hp)]$  else  $[]$ )
      @ concat (map ( $\lambda p. \text{titems-for } k \ (\text{closes-at } s0 \ p)$ 
      @ (if  $\text{ev-key } (L \ ! \ p) = k$  then  $[(\text{Suc } p, L \ ! \ p)]$  else  $[]$ )
      [Suc  $hp \ .. < \text{den } f$ ])
      @  $\text{titems-for } k \ (\text{closes-at } s0 \ (\text{den } f))$ )
  unfolding dist by (simp add: split head)
  show ?thesis
  unfolding peel pre post trail closes-hp
  by (simp add: kposs-peel flip: u-def hp-def)
qed
qed

```

3.5 The window-discard equivalence theorem

Window-discard equivalence: the emitted stream replays to the canonical *sink* on the scope. Together with *contract-cut* this is “the log wins” as a theorem: buffer the unit’s refresh, discard on any in-window event for a buffered key, emit survivors at the high edge — and the replay is exactly the source state at the frontier.

```

theorem window-discard-replay:
  assumes s0lo:  $\bigwedge u. u \in \text{set units} \implies s0 \sqsubseteq u\text{-lo } u$ 
  and k:  $k \in \text{scope}$ 
  shows stream-replay (emission s0) k = sink-at f k
proof -
  define u where  $u = \text{the-unit } k$ 
  define hp where  $hp = \text{den } (u\text{-hi } u)$ 
  have u-in:  $u \in \text{set units}$  and k-dom:  $k \in u\text{-dom } u$ 
  using the-unit-props[OF k] u-def by auto
  have lo-hi:  $u\text{-lo } u \sqsubseteq u\text{-hi } u$  using bracket-lo-hi[OF u-in] .
  have hi-f:  $u\text{-hi } u \sqsubseteq f$  using bracket-hi-f[OF u-in] .
  have lo-f:  $u\text{-lo } u \sqsubseteq f$  using lo-hi hi-f by (rule cle-trans)
  have s0-le-lo:  $\text{den } s0 \leq \text{den } (u\text{-lo } u)$ 
  using s0lo[OF u-in] by (simp add: cle-def)
  have lo-le-hp:  $\text{den } (u\text{-lo } u) \leq hp$ 

```

```

using lo-hi by (simp add: cle-def hp-def)
have hp-le-f:  $hp \leq den\ f$ 
using hi-f by (simp add: cle-def hp-def)
have s0-le-hp:  $den\ s0 \leq hp$  using s0-le-lo lo-le-hp by simp

have surv-iff:  $k \in set\ (svl\ u) \longleftrightarrow k \in survivors\ u$ 
using svl u-in by (simp add: svl-ok-def)

have master:  $evs\text{-}for\ k\ (emission\ svl\ s0) =$ 
   $map\ (!)\ L\ (kposs\ k\ (den\ s0)\ hp)$ 
   $@\ (if\ k \in set\ (svl\ u)\ then\ [Event\ k\ (u\text{-}refresh\ u\ k)]\ else\ [])$ 
   $@\ map\ (!)\ L\ (kposs\ k\ hp\ (den\ f))$ 
unfolding emission-def  $evs\text{-}for\text{-}map\text{-}snd$ 
using titems-emission-master[OF svl s0lo k]
by (simp add: u-def hp-def comp-def if-distrib map-append
  cong: if-cong)

have win-lo-hi:  $evs\text{-}for\ k\ (win\ (u\text{-}lo\ u)\ (u\text{-}hi\ u))$ 
   $= map\ (!)\ L\ (kposs\ k\ (den\ (u\text{-}lo\ u))\ hp)$ 
using  $evs\text{-}for\text{-}win\text{-}kposs$ [OF lo-hi] by (simp add: hp-def)
have win-lo-f:  $evs\text{-}for\ k\ (win\ (u\text{-}lo\ u)\ f)$ 
   $= map\ (!)\ L\ (kposs\ k\ (den\ (u\text{-}lo\ u))\ (den\ f))$ 
using  $evs\text{-}for\text{-}win\text{-}kposs$ [OF lo-f] by simp
have kposs-lo-f-split:
   $kposs\ k\ (den\ (u\text{-}lo\ u))\ (den\ f)$ 
   $= kposs\ k\ (den\ (u\text{-}lo\ u))\ hp\ @\ kposs\ k\ hp\ (den\ f)$ 
using lo-le-hp hp-le-f by (rule kposs-split)
have kposs-s0-hp-split:
   $kposs\ k\ (den\ s0)\ hp$ 
   $= kposs\ k\ (den\ s0)\ (den\ (u\text{-}lo\ u))\ @\ kposs\ k\ (den\ (u\text{-}lo\ u))\ hp$ 
using s0-le-lo lo-le-hp by (rule kposs-split)

show ?thesis
proof (cases  $kposs\ k\ (den\ (u\text{-}lo\ u))\ (den\ f) = []$ )
case False
then have sink:  $sink\text{-}at\ f\ k$ 
   $= ev\text{-}img\ (last\ (map\ (!)\ L\ (kposs\ k\ (den\ (u\text{-}lo\ u))\ (den\ f))))$ 
by (simp add: sink-at-def win-lo-f flip: u-def)
show ?thesis
proof (cases  $kposs\ k\ hp\ (den\ f) = []$ )
case post-ne: False
have last-sink:  $last\ (kposs\ k\ (den\ (u\text{-}lo\ u))\ (den\ f)) = last\ (kposs\ k\ hp\ (den$ 
f))
using kposs-lo-f-split post-ne by (simp add: last-append)
have stream-ne:  $evs\text{-}for\ k\ (emission\ svl\ s0) \neq []$ 
using master post-ne by auto
have last-stream:  $last\ (evs\text{-}for\ k\ (emission\ svl\ s0))$ 
   $= (!)\ L\ (last\ (kposs\ k\ hp\ (den\ f)))$ 
using master post-ne by (simp add: last-append last-map)

```

```

have stream-replay (emission s0) k
  = ev-img (L ! last (kposs k hp (den f)))
  unfolding stream-replay-def
  using state-after-last-event[OF stream-ne] last-stream by simp
moreover have sink-at f k = ev-img (L ! last (kposs k hp (den f)))
  using sink last-sink False by (simp add: last-map)
ultimately show ?thesis by simp
next
case post-e: True
have win-ne: kposs k (den (u-lo u)) hp ≠ []
  using False kposs-lo-f-split post-e by auto
have not-surv: k ∉ set (svl u)
  using surv-iff win-lo-hi win-ne by (auto simp: survivors-def)
have master': evs-for k (emission s0)
  = map (!) L (kposs k (den s0) hp)
  using master not-surv post-e by simp
have s0-ne: kposs k (den s0) hp ≠ []
  using kposs-s0-hp-split win-ne by auto
have last-pre: last (kposs k (den s0) hp) = last (kposs k (den (u-lo u)) hp)
  using kposs-s0-hp-split win-ne by (simp add: last-append)
have last-sink: last (kposs k (den (u-lo u)) (den f))
  = last (kposs k (den (u-lo u)) hp)
  using kposs-lo-f-split post-e by simp
have stream-replay (emission s0) k
  = ev-img (L ! last (kposs k (den (u-lo u)) hp))
  unfolding stream-replay-def
  by (subst state-after-key)
  (simp add: master' s0-ne last-pre last-map)
moreover have sink-at f k = ev-img (L ! last (kposs k (den (u-lo u)) hp))
  using sink last-sink False by (simp add: last-map)
ultimately show ?thesis by simp
qed
next
case True
have win-e: kposs k (den (u-lo u)) hp = []
  and post-e: kposs k hp (den f) = []
  using True kposs-lo-f-split by auto
have no-win-evs: evs-for k (win (u-lo u) (u-hi u)) = []
  using win-lo-hi win-e by simp
have sink: sink-at f k = u-refresh u k
  using True by (simp add: sink-at-def win-lo-f flip: u-def)
have pre-only: kposs k (den s0) hp = kposs k (den s0) (den (u-lo u))
  using kposs-s0-hp-split win-e by simp
show ?thesis
proof (cases u-refresh u k)
case (Some v)
then have k ∈ survivors u
  using k-dom no-win-evs by (simp add: survivors-def)
then have surv: k ∈ set (svl u)

```

```

    using surv-iff by simp
  have evs-for k (emission svl s0)
    = map (!) L (kposs k (den s0) hp) @ [Event k (u-refresh u k)]
    using master surv post-e by simp
  then have stream-replay (emission svl s0) k = u-refresh u k
    unfolding stream-replay-def
    by (simp add: state-after-key last-append)
  then show ?thesis using sink by simp
next
case None
then have not-surv: k ∉ set (svl u)
  using surv-iff by (auto simp: survivors-def)
have master': evs-for k (emission svl s0)
  = map (!) L (kposs k (den s0) (den (u-lo u)))
  using master not-surv post-e pre-only by simp
from O2-honest[OF u-in k-dom] obtain c where
  c-lo: u-lo u ⊆ c and c-hi: c ⊆ u-hi u and
  honest: u-refresh u k = state-after σ0 (pfx c) k
  by blast
show ?thesis
proof (cases kposs k (den s0) (den (u-lo u)) = [])
case True
  then have evs-for k (emission svl s0) = []
    using master' by simp
  then have stream-replay (emission svl s0) k = None
    unfolding stream-replay-def by (simp add: state-after-no-events)
  then show ?thesis using sink None by simp
next
case pre-ne: False
define q where q = last (kposs k (den s0) (den (u-lo u)))
have lo-le-c: den (u-lo u) ≤ den c using c-lo by (simp add: cle-def)
have c-le-hp: den c ≤ hp using c-hi by (simp add: cle-def hp-def)
have win-lo-c-e: kposs k (den (u-lo u)) (den c) = []
  using win-e kposs-split[OF lo-le-c c-le-hp] by auto
have c-split: kposs k 0 (den c)
  = kposs k 0 (den s0) @ kposs k (den s0) (den (u-lo u))
  using kposs-split[OF le0 s0-le-lo, of k]
    kposs-split[OF - lo-le-c, of 0 k] win-lo-c-e
  by (metis append-Nil2 le0 s0-le-lo order-trans)
have pfx-ne: evs-for k (pfx c) ≠ []
  using evs-for-pfx-kposs c-split pre-ne by simp
have last-pfx: last (kposs k 0 (den c)) = q
  using c-split pre-ne q-def by (simp add: last-append)
have state-after σ0 (pfx c) k = ev-img (L ! q)
  using state-after-last-event[OF pfx-ne] evs-for-pfx-kposs last-pfx pre-ne
  by (simp add: last-map last-append c-split)
with honest None have img-q: ev-img (L ! q) = None by simp
have stream-replay (emission svl s0) k = ev-img (L ! q)
  unfolding stream-replay-def

```

```

    by (subst state-after-key)
      (simp add: master' pre-ne q-def last-map)
  then show ?thesis using sink None img-q by simp
qed
qed
qed
qed

```

3.6 Window-discard per-key monotonicity

Under window-discard the emission is per-key monotone outright: the occurrence tags of a key's items never decrease along the stream — a key emits its events in commit order, plus its refresh at the unit's high edge if it survived the window, and a survivor's later post-window events follow the refresh. Never an older state over a newer one.

theorem *window-discard-monotone*:

```

  assumes svl: svl-ok svl
    and s0lo:  $\bigwedge u. u \in \text{set units} \implies s0 \sqsubseteq u\text{-lo } u$ 
    and k:  $k \in \text{scope}$ 
  shows sorted (map fst (titems-for k (emission-t svl s0)))
proof -
  define hp where hp = den (u-hi (the-unit k))
  have m: map fst (titems-for k (emission-t svl s0)) =
    map Suc (kposs k (den s0) hp)
    @ (if  $k \in \text{set (svl (the-unit k))}$  then [hp] else [])
    @ map Suc (kposs k hp (den f))
  using titems-emission-master[OF svl s0lo k]
  by (cases  $k \in \text{set (svl (the-unit k))}$ ) (simp-all add: hp-def comp-def)
  have s-pre: sorted (map Suc (kposs k (den s0) hp))
  by (simp add: sorted-map-Suc kposs-sorted)
  have s-post: sorted (map Suc (kposs k hp (den f)))
  by (simp add: sorted-map-Suc kposs-sorted)
  have pre-le:  $\bigwedge a. a \in \text{set (map Suc (kposs k (den s0) hp))} \implies a \leq hp$ 
  using kposs-bounds by (auto intro: Suc-leI)
  have post-ge:  $\bigwedge b. b \in \text{set (map Suc (kposs k hp (den f)))} \implies hp \leq b$ 
  using kposs-bounds by (auto intro: le-SucI)
  have cross:  $\bigwedge x y. x \in \text{set (kposs k (den s0) hp)} \implies$ 
     $y \in \text{set (kposs k hp (den f))} \implies x \leq y$ 
  using kposs-bounds by (metis less-le-trans less-imp-le)
  show ?thesis
proof (cases  $k \in \text{set (svl (the-unit k))}$ )
  case True
  have sorted (map Suc (kposs k (den s0) hp)
    @ hp # map Suc (kposs k hp (den f)))
  using s-pre s-post pre-le post-ge cross
  by (auto simp: sorted-append intro: order.trans)
  then show ?thesis using m True by simp
next

```

```

case False
have sorted (map Suc (kposs k (den s0) hp)
              @ map Suc (kposs k hp (den f)))
  using s-pre s-post pre-le post-ge cross
  by (auto simp: sorted-append intro: order.trans)
then show ?thesis using m False by simp
qed
qed

end

end

```

```

theory Contract-Witnesses
  imports Merge-Disciplines
begin

```

4 Constructed witnesses and boundary fixtures

House discipline: every main result is exercised at concrete witnesses, so none is vacuously true. *wit*-prefixed facts are positive witnesses; *fix*-prefixed facts are negative or boundary fixtures. Four fixtures over small concrete logs:

- the *torn read*: a unit whose refresh is valid per key at DIFFERENT in-bracket coordinates, with provably NO single in-bracket coordinate serving all keys — an instance the single-coordinate obligation would reject, yet the cut theorem still delivers the frontier state;
- the *tablesnc* shape (Corollary 3’s sharpness boundary): two point-bracket units, each T3 on its own scope at its own witness, whose composite has no common in-bracket witness and provably breaks the source-trajectory property at an intermediate coordinate. The automatic composition result is T2 with its conservative trajectory beginning at the latest high;
- the *amortized shared bracket*: one bracket shared by two units with one shared state witness — Corollary 1 lifts the composite to T3, so amortization preserved the tier;
- a *surviving refresh* under window-discard: a unit whose refresh entry survives its window and must override a pre-low consumed event in the emitted stream — the equivalence theorem’s subtlest path, evaluated concretely.

4.1 The witness log and coordinate space

definition $L-w :: (nat, nat)$ event list **where**
 $L-w = [Event\ 1\ (Some\ 1),\ Event\ 2\ (Some\ 2)]$

definition $den-w :: nat \Rightarrow nat$ **where**
 $den-w\ c = \min\ c\ 2$

definition $s0-w :: (nat, nat)$ state **where**
 $s0-w = (\lambda-. None)$

interpretation wc : coordinate-space $L-w\ den-w$
by $unfold-locales\ (simp\ add:\ den-w-def\ L-w-def)$

lemma $den-w-cases$: $den-w\ c = 0 \vee den-w\ c = 1 \vee den-w\ c = 2$
by $(simp\ add:\ den-w-def)\ linarith$

4.2 Fixture 1: the torn read

definition $torn-unit :: (nat, nat, nat)$ cunit **where**
 $torn-unit = (\lambda u-dom = \{1, 2\}, u-lo = 0, u-hi = 2,$
 $u-refresh = (\lambda k. if\ k = 2\ then\ Some\ 2\ else\ None))$

interpretation $torn$: capture-contract $L-w\ den-w\ s0-w\ [torn-unit]\ \{1, 2\}\ 2$

proof $(unfold-locales)$

show $(\bigcup u \in set\ [torn-unit].\ u-dom\ u) = \{1, 2\}$
by $(simp\ add:\ torn-unit-def)$

next

fix $i\ j$
assume $i < length\ [torn-unit]\ j < length\ [torn-unit]\ i \neq j$
then show $u-dom\ ([torn-unit]!\ i) \cap u-dom\ ([torn-unit]!\ j) = \{\}$
by $simp$

next

fix u **assume** $u \in set\ [torn-unit]$
then show $wc.cle\ (u-lo\ u)\ (u-hi\ u)$
by $(simp\ add:\ torn-unit-def\ wc.cle-def\ den-w-def)$

next

fix u **assume** $u \in set\ [torn-unit]$
then show $wc.cle\ (u-hi\ u)\ 2$
by $(simp\ add:\ torn-unit-def\ wc.cle-def\ den-w-def)$

next

fix $u\ k$ **assume** $u: u \in set\ [torn-unit]$ **and** $k: k \in u-dom\ u$
then have $u-eq: u = torn-unit$ **and** $k-cases: k = 1 \vee k = 2$
by $(auto\ simp:\ torn-unit-def)$

show $\exists c. wc.cle\ (u-lo\ u)\ c \wedge wc.cle\ c\ (u-hi\ u) \wedge$
 $u-refresh\ u\ k = state-after\ s0-w\ (wc.pfx\ c)\ k$

proof $(cases\ k = 1)$

case $True$

then show $?thesis$

unfolding $wc.cle-def\ wc.pfx-def$

```

    by (intro exI[of - 0])
      (simp add: u-eq torn-unit-def den-w-def L-w-def
        s0-w-def state-after-def apply-ev-def)
  next
    case False
    with k-cases have k = 2 by simp
    then show ?thesis
      unfolding wc.cle-def wc.pfx-def
      by (intro exI[of - 2])
        (simp add: u-eq torn-unit-def den-w-def L-w-def
          s0-w-def state-after-def apply-ev-def)
qed
qed

```

lemma *wit-torn-contract*:

```

capture-contract L-w den-w s0-w [torn-unit] {1, 2} 2
by (rule torn.capture-contract-axioms)

```

NO single in-bracket coordinate is valid for both keys of the unit — key 1's absence result is valid only before the first event, key 2's presence result only after the second. Single-coordinate validity would reject this instance; the contract admits it, and the cut still holds (*wit-torn-cut* below). Together with theorem *capture-contract.contract-cut*, this shows that per-key validity suffices even when no shared read coordinate exists.

lemma *fix-torn-no-single-witness*:

```

¬ (∃ c. wc.cle (u-lo torn-unit) c ∧ wc.cle c (u-hi torn-unit) ∧
  (∀ k ∈ u-dom torn-unit.
    u-refresh torn-unit k = state-after s0-w (wc.pfx c) k))

```

proof

```

assume ∃ c. wc.cle (u-lo torn-unit) c ∧ wc.cle c (u-hi torn-unit) ∧
  (∀ k ∈ u-dom torn-unit.
    u-refresh torn-unit k = state-after s0-w (wc.pfx c) k)

```

then obtain *c* **where**

```

honest: ∀ k ∈ u-dom torn-unit.
  u-refresh torn-unit k = state-after s0-w (wc.pfx c) k

```

by *blast*

have *h1*: *None* = *state-after s0-w (take (den-w c) L-w) 1*

and *h2*: *Some 2* = *state-after s0-w (take (den-w c) L-w) 2*

using *honest* **unfolding** *wc.pfx-def* **by** (*auto simp: torn-unit-def*)

from *den-w-cases*[*of c*] **show** *False*

proof (*elim disjE*)

assume *d*: *den-w c = 0*

then show *False*

using *h2* **by** (*simp add: d L-w-def s0-w-def state-after-def apply-ev-def*)

next

assume *d*: *den-w c = 1*

then show *False*

using *h2* **by** (*simp add: d L-w-def s0-w-def state-after-def apply-ev-def*)

next

```

    assume d: den-w c = 2
    then show False
      using h1 by (simp add: d L-w-def s0-w-def state-after-def apply-ev-def)
  qed
qed

```

```

lemma wit-torn-the-unit [simp]:
  assumes k ∈ {1 :: nat, 2}
  shows torn.the-unit k = torn-unit
  using torn.the-unit-props[OF assms] by simp

```

```

lemma wit-torn-cut:
  torn.sink-at 2 1 = Some 1 torn.sink-at 2 2 = Some 2
proof -
  have c1: torn.sink-at 2 1 = state-after s0-w (wc.pfx 2) 1
    by (rule torn.contract-cut) simp
  show torn.sink-at 2 1 = Some 1
    unfolding c1 wc.pfx-def
    by (simp add: den-w-def L-w-def s0-w-def state-after-def apply-ev-def)
  have c2: torn.sink-at 2 2 = state-after s0-w (wc.pfx 2) 2
    by (rule torn.contract-cut) simp
  show torn.sink-at 2 2 = Some 2
    unfolding c2 wc.pfx-def
    by (simp add: den-w-def L-w-def s0-w-def state-after-def apply-ev-def)
qed

```

4.3 Fixture 2: the tablesync shape (composition is sharp at T2)

```

definition syncA :: (nat, nat, nat) cunit where
  syncA = (u-dom = {1}, u-lo = 0, u-hi = 0, u-refresh = (λ-. None))

```

```

definition syncB :: (nat, nat, nat) cunit where
  syncB = (u-dom = {2}, u-lo = 2, u-hi = 2,
    u-refresh = (λk. if k = 2 then Some 2 else None))

```

```

interpretation sync: capture-contract L-w den-w s0-w [syncA, syncB] {1, 2} 2

```

```

proof (unfold-locales)
  show (⋃ u ∈ set [syncA, syncB]. u-dom u) = {1, 2}
    by (auto simp: syncA-def syncB-def)
next
  fix i j
  assume i: i < length [syncA, syncB] and j: j < length [syncA, syncB]
    and ne: i ≠ j
  then have (i = 0 ∧ j = 1) ∨ (i = 1 ∧ j = 0)
    by auto
  then show u-dom ([syncA, syncB] ! i) ∩ u-dom ([syncA, syncB] ! j) = {}
    by (auto simp: syncA-def syncB-def)
next

```

```

fix  $u$  assume  $u \in \text{set } [\text{syncA}, \text{syncB}]$ 
then show  $\text{wc.cle } (u\text{-lo } u) (u\text{-hi } u)$ 
  by (auto simp: syncA-def syncB-def wc.cle-def den-w-def)
next
fix  $u$  assume  $u \in \text{set } [\text{syncA}, \text{syncB}]$ 
then show  $\text{wc.cle } (u\text{-hi } u) 2$ 
  by (auto simp: syncA-def syncB-def wc.cle-def den-w-def)
next
fix  $u$   $k$  assume  $u: u \in \text{set } [\text{syncA}, \text{syncB}]$  and  $k: k \in u\text{-dom } u$ 
show  $\exists c. \text{wc.cle } (u\text{-lo } u) c \wedge \text{wc.cle } c (u\text{-hi } u) \wedge$ 
   $u\text{-refresh } u k = \text{state-after } s0\text{-w } (\text{wc.pfx } c) k$ 
proof (cases  $u = \text{syncA}$ )
  case True
  then have  $k = 1$  using  $k$  by (simp add: syncA-def)
  then show ?thesis
    unfolding  $\text{wc.cle-def wc.pfx-def}$ 
    by (intro exI[of - 0])
    (simp add: True syncA-def den-w-def L-w-def
       $s0\text{-w-def state-after-def apply-ev-def$ )
  next
  case False
  then have  $uB: u = \text{syncB}$  using  $u$  by simp
  then have  $k = 2$  using  $k$  by (simp add: syncB-def)
  then show ?thesis
    unfolding  $\text{wc.cle-def wc.pfx-def}$ 
    by (intro exI[of - 2])
    (simp add: uB syncB-def den-w-def L-w-def
       $s0\text{-w-def state-after-def apply-ev-def$ )
qed
qed

```

lemma *wit-sync-contract*:
capture-contract $L\text{-w den-w } s0\text{-w } [\text{syncA}, \text{syncB}] \{1, 2\} 2$
by (*rule sync.capture-contract-axioms*)

lemma *wit-sync-the-unit-2* [*simp*]: $\text{sync.the-unit } 2 = \text{syncB}$
using *sync.the-unit-props*[*of* 2] **by** (*auto simp: syncA-def*)

The composite breaks the source-trajectory property at the intermediate coordinate 1: unit B's refresh (valid at ITS witness, the log end) already shows key 2 present, while the source at coordinate 1 does not. Hence no trajectory onset strictly before the frontier exists for this composite plan. Its latest high is the frontier 2, so *latest-high-trajectory* still supplies the degenerate conservative interval from 2 to 2. The two point brackets also have no common in-bracket coordinate. Each member alone nevertheless has its own exhibited shared witness, as shown by *wit-sync-memberA-T3* and *wit-sync-memberB-T3*.

lemma *fix-sync-trajectory-breaks*:

$sync.sink-at\ 1\ 2 \neq state-after\ s0-w\ (wc.pfx\ 1)\ 2$
proof –
 have $win-e: wc.win\ (u-lo\ syncB)\ 1 = []$
 unfolding $wc.win-def$ by (simp add: $syncB-def\ den-w-def\ L-w-def$)
 have $sync.sink-at\ 1\ 2 = Some\ 2$
 unfolding $sync.sink-at-def\ wit-sync-the-unit-2\ win-e$
 by (simp add: $syncB-def$)
 moreover have $state-after\ s0-w\ (wc.pfx\ 1)\ 2 = None$
 unfolding $wc.pfx-def$
 by (simp add: $den-w-def\ L-w-def\ s0-w-def\ state-after-def\ apply-ev-def$)
 ultimately show ?thesis by simp
qed

lemma *fix-sync-no-early-onset*:
 $\neg (\exists\ cstar. wc.cle\ cstar\ 1 \wedge$
 $(\forall\ g. wc.cle\ cstar\ g \longrightarrow wc.cle\ g\ 2 \longrightarrow$
 $(\forall\ k \in \{1, 2\}. sync.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k)))$
proof
 assume $\exists\ cstar. wc.cle\ cstar\ 1 \wedge$
 $(\forall\ g. wc.cle\ cstar\ g \longrightarrow wc.cle\ g\ 2 \longrightarrow$
 $(\forall\ k \in \{1, 2\}. sync.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k))$
 then obtain $cstar$ where $cs: wc.cle\ cstar\ 1$
 and $traj: \bigwedge g. wc.cle\ cstar\ g \implies wc.cle\ g\ 2 \implies$
 $\forall k \in \{1, 2\}. sync.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k$
 by blast
 have $wc.cle\ 1\ 2$
 by (simp add: $wc.cle-def\ den-w-def$)
 then have $sync.sink-at\ 1\ 2 = state-after\ s0-w\ (wc.pfx\ 1)\ 2$
 using $traj[OF\ cs]$ by simp
 then show *False*
 using *fix-sync-trajectory-breaks* by simp
qed

The paper states Proposition 2 with an onset strictly below the frontier. The preceding lemma is an equivalent fixture-specific form: in this fixture, strictly below frontier 2 is exactly at or below coordinate 1. The following statement gives the paper’s quantifier exactly. Strictness is expressed by $wc.cle\ 2 \wedge \neg wc.cle\ 2\ cstar$.

lemma *fix-sync-no-strictly-below-frontier-onset*:
 $\neg (\exists\ cstar. wc.cle\ cstar\ 2 \wedge \neg wc.cle\ 2\ cstar \wedge$
 $(\forall\ g. wc.cle\ cstar\ g \longrightarrow wc.cle\ g\ 2 \longrightarrow$
 $(\forall\ k \in \{1, 2\}. sync.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k)))$
proof
 assume $\exists\ cstar. wc.cle\ cstar\ 2 \wedge \neg wc.cle\ 2\ cstar \wedge$
 $(\forall\ g. wc.cle\ cstar\ g \longrightarrow wc.cle\ g\ 2 \longrightarrow$
 $(\forall\ k \in \{1, 2\}. sync.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k))$
 then obtain $cstar$ where $below: wc.cle\ cstar\ 2 \neg wc.cle\ 2\ cstar$

and *traj*: $\bigwedge g. wc.cle\ cstar\ g \implies wc.cle\ g\ 2 \implies$
 $\forall k \in \{1, 2\}.$
 $sync.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k$
by *blast*
have *den-le-one*: $den-w\ cstar \leq 1$
using *below unfolding wc.cle-def*
by (*simp add: den-w-def*)
have *cs1*: $wc.cle\ cstar\ 1$
using *den-le-one* **by** (*simp add: wc.cle-def den-w-def*)
have *one-two*: $wc.cle\ 1\ 2$
by (*simp add: wc.cle-def den-w-def*)
have $sync.sink-at\ 1\ 2 = state-after\ s0-w\ (wc.pfx\ 1)\ 2$
using *traj[OF cs1 one-two]* **by** *simp*
then show *False*
using *fix-sync-trajectory-breaks* **by** *simp*
qed

lemma *fix-sync-no-common-bracket-witness*:
 $\neg (\exists cstar. \forall u \in set\ [syncA, syncB].$
 $wc.cle\ (u-lo\ u)\ cstar \wedge wc.cle\ cstar\ (u-hi\ u))$
proof
assume $\exists cstar. \forall u \in set\ [syncA, syncB].$
 $wc.cle\ (u-lo\ u)\ cstar \wedge wc.cle\ cstar\ (u-hi\ u)$
then obtain *cstar* **where** *all*:
 $\forall u \in set\ [syncA, syncB].$
 $wc.cle\ (u-lo\ u)\ cstar \wedge wc.cle\ cstar\ (u-hi\ u)$ **by** *blast*
have *upper0*: $wc.cle\ cstar\ 0$
using *all* **by** (*simp add: syncA-def*)
have *lower2*: $wc.cle\ 2\ cstar$
using *all* **by** (*simp add: syncB-def*)
have $den-w\ cstar \leq 0$ **and** $2 \leq den-w\ cstar$
using *upper0 lower2* **by** (*simp-all add: wc.cle-def den-w-def*)
then show *False* **by** *linarith*
qed

interpretation *syncAo*: *capture-contract L-w den-w s0-w [syncA] {1} 2*
proof (*unfold-locales*)
show $(\bigcup u \in set\ [syncA].\ u-dom\ u) = \{1\}$
by (*simp add: syncA-def*)
next
fix *i j*
assume $i < length\ [syncA]\ j < length\ [syncA]\ i \neq j$
then show $u-dom\ ([syncA]!\ i) \cap u-dom\ ([syncA]!\ j) = \{\}$ **by** *simp*
next
fix *u* **assume** $u \in set\ [syncA]$
then show $wc.cle\ (u-lo\ u)\ (u-hi\ u)$
by (*simp add: syncA-def wc.cle-def den-w-def*)
next
fix *u* **assume** $u \in set\ [syncA]$

then show $wc.cle (u-hi\ u)\ 2$
 by (simp add: syncA-def wc.cle-def den-w-def)
 next
 fix $u\ k$ assume $u \in set\ [syncA]$ and $k \in u-dom\ u$
 then show $\exists c. wc.cle (u-lo\ u)\ c \wedge wc.cle\ c\ (u-hi\ u) \wedge$
 $u-refresh\ u\ k = state-after\ s0-w\ (wc.pfx\ c)\ k$
 unfolding wc.cle-def wc.pfx-def
 by (intro exI[of - 0])
 (simp add: syncA-def den-w-def L-w-def
 s0-w-def state-after-def apply-ev-def)
 qed

lemma wit-sync-memberA-shared-bracket:
 $\bigwedge u. u \in set\ [syncA] \implies$
 $wc.cle (u-lo\ u)\ 0 \wedge wc.cle\ 0\ (u-hi\ u)$
 by (simp add: syncA-def wc.cle-def den-w-def)

lemma wit-sync-memberA-shared-honesty:
 $\bigwedge u\ k. u \in set\ [syncA] \implies k \in u-dom\ u \implies$
 $u-refresh\ u\ k = state-after\ s0-w\ (wc.pfx\ 0)\ k$
 unfolding wc.pfx-def
 by (simp add: syncA-def den-w-def L-w-def
 s0-w-def state-after-def apply-ev-def)

lemma wit-sync-memberA-T3:
 assumes $wc.cle\ 0\ g$ and $wc.cle\ g\ 2$ and $k \in \{1 :: nat\}$
 shows $syncAo.sink-at\ g\ k = state-after\ s0-w\ (wc.pfx\ g)\ k$
 by (rule syncAo.shared-witness-trajectory[OF
 wit-sync-memberA-shared-bracket wit-sync-memberA-shared-honesty assms])

interpretation syncBo: capture-contract L-w den-w s0-w [syncB] {2} 2
 proof (unfold-locales)
 show $(\bigcup u \in set\ [syncB].\ u-dom\ u) = \{2\}$
 by (simp add: syncB-def)
 next
 fix $i\ j$
 assume $i < length\ [syncB]\ j < length\ [syncB]\ i \neq j$
 then show $u-dom\ ([syncB]!\ i) \cap u-dom\ ([syncB]!\ j) = \{\}$ by simp
 next
 fix u assume $u \in set\ [syncB]$
 then show $wc.cle (u-lo\ u)\ (u-hi\ u)$
 by (simp add: syncB-def wc.cle-def den-w-def)
 next
 fix u assume $u \in set\ [syncB]$
 then show $wc.cle (u-hi\ u)\ 2$
 by (simp add: syncB-def wc.cle-def den-w-def)
 next
 fix $u\ k$ assume $u \in set\ [syncB]$ and $k \in u-dom\ u$
 then show $\exists c. wc.cle (u-lo\ u)\ c \wedge wc.cle\ c\ (u-hi\ u) \wedge$

$u\text{-refresh } u \ k = \text{state-after } s0\text{-w } (wc.pfx \ c) \ k$
unfolding $wc.cle\text{-def } wc.pfx\text{-def}$
by ($\text{intro } exI[of \ 2]$)
 $(simp \ add: \ syncB\text{-def } den\text{-w}\text{-def } L\text{-w}\text{-def}$
 $s0\text{-w}\text{-def } state\text{-after}\text{-def } apply\text{-ev}\text{-def})$
qed

lemma $wit\text{-sync}\text{-member}B\text{-shared}\text{-bracket}$:
 $\bigwedge u. u \in \text{set } [syncB] \implies$
 $wc.cle \ (u\text{-lo } u) \ 2 \wedge wc.cle \ 2 \ (u\text{-hi } u)$
by ($simp \ add: \ syncB\text{-def } wc.cle\text{-def } den\text{-w}\text{-def}$)

lemma $wit\text{-sync}\text{-member}B\text{-shared}\text{-honesty}$:
 $\bigwedge u \ k. u \in \text{set } [syncB] \implies k \in u\text{-dom } u \implies$
 $u\text{-refresh } u \ k = \text{state-after } s0\text{-w } (wc.pfx \ 2) \ k$
unfolding $wc.pfx\text{-def}$
by ($auto \ simp: \ syncB\text{-def } den\text{-w}\text{-def } L\text{-w}\text{-def } s0\text{-w}\text{-def}$
 $state\text{-after}\text{-def } apply\text{-ev}\text{-def}$)

lemma $wit\text{-sync}\text{-member}B\text{-T3}$:
assumes $wc.cle \ 2 \ g$ **and** $wc.cle \ g \ 2$ **and** $k \in \{2 :: nat\}$
shows $syncBo.sink\text{-at } g \ k = \text{state-after } s0\text{-w } (wc.pfx \ g) \ k$
proof ($rule \ syncBo.shared\text{-witness}\text{-trajectory}[of \ 2 \ g \ k]$)
show $\bigwedge u. u \in \text{set } [syncB] \implies wc.cle \ (u\text{-lo } u) \ 2 \wedge wc.cle \ 2 \ (u\text{-hi } u)$
by ($rule \ wit\text{-sync}\text{-member}B\text{-shared}\text{-bracket}$)
show $\bigwedge u \ k. u \in \text{set } [syncB] \implies k \in u\text{-dom } u \implies$
 $u\text{-refresh } u \ k = \text{state-after } s0\text{-w } (wc.pfx \ 2) \ k$
by ($rule \ wit\text{-sync}\text{-member}B\text{-shared}\text{-honesty}$)
qed ($use \ assms \ in \ auto$)

theorem $wit\text{-sync}\text{-composition}\text{-sharpness}$:
shows $capture\text{-contract } L\text{-w } den\text{-w } s0\text{-w } [syncA] \ \{1\} \ 2$
and $capture\text{-contract } L\text{-w } den\text{-w } s0\text{-w } [syncB] \ \{2\} \ 2$
and $\forall u \in \text{set } [syncA].$
 $wc.cle \ (u\text{-lo } u) \ 0 \wedge wc.cle \ 0 \ (u\text{-hi } u)$
and $\forall u \in \text{set } [syncA]. \forall k \in u\text{-dom } u.$
 $u\text{-refresh } u \ k = \text{state-after } s0\text{-w } (wc.pfx \ 0) \ k$
and $\forall u \in \text{set } [syncB].$
 $wc.cle \ (u\text{-lo } u) \ 2 \wedge wc.cle \ 2 \ (u\text{-hi } u)$
and $\forall u \in \text{set } [syncB]. \forall k \in u\text{-dom } u.$
 $u\text{-refresh } u \ k = \text{state-after } s0\text{-w } (wc.pfx \ 2) \ k$
and $capture\text{-contract } L\text{-w } den\text{-w } s0\text{-w } [syncA, syncB] \ \{1, 2\} \ 2$
and $\neg (\exists cstar. \forall u \in \text{set } [syncA, syncB].$
 $wc.cle \ (u\text{-lo } u) \ cstar \wedge wc.cle \ cstar \ (u\text{-hi } u))$
and $\neg (\exists cstar. wc.cle \ cstar \ 1 \wedge$
 $(\forall g. wc.cle \ cstar \ g \longrightarrow wc.cle \ g \ 2 \longrightarrow$
 $(\forall k \in \{1, 2\}.$
 $sync.sink\text{-at } g \ k = \text{state-after } s0\text{-w } (wc.pfx \ g) \ k)))$
and $\neg (\exists cstar. wc.cle \ cstar \ 2 \wedge \neg wc.cle \ 2 \ cstar \wedge$

$(\forall g. wc.cle \text{ cstar } g \longrightarrow wc.cle \text{ } g \text{ } 2 \longrightarrow$
 $(\forall k \in \{1, 2\}. \text{ sync.sink-at } g \text{ } k = \text{ state-after } s0\text{-w } (wc.pfx \text{ } g) \text{ } k)))$
by (rule syncAo.capture-contract-axioms,
rule syncBo.capture-contract-axioms,
use wit-sync-memberA-shared-bracket **in** blast,
use wit-sync-memberA-shared-honesty **in** blast,
use wit-sync-memberB-shared-bracket **in** blast,
use wit-sync-memberB-shared-honesty **in** blast,
rule sync.capture-contract-axioms,
rule fix-sync-no-common-bracket-witness,
rule fix-sync-no-early-onset,
rule fix-sync-no-strictly-below-frontier-onset)

4.4 Fixture 3: amortized shared bracket with a shared witness (T3)

definition $shrC :: (nat, nat, nat) \text{ cunit where}$
 $shrC = \langle u\text{-dom} = \{1\}, u\text{-lo} = 0, u\text{-hi} = 2,$
 $u\text{-refresh} = (\lambda k. \text{ if } k = 1 \text{ then Some } 1 \text{ else None}) \rangle$

definition $shrD :: (nat, nat, nat) \text{ cunit where}$
 $shrD = \langle u\text{-dom} = \{2\}, u\text{-lo} = 0, u\text{-hi} = 2, u\text{-refresh} = (\lambda _. \text{ None}) \rangle$

interpretation shr : capture-contract $L\text{-w den-w } s0\text{-w } [shrC, shrD] \{1, 2\} \text{ } 2$

proof (unfold-locales)

show $(\bigcup u \in \text{set } [shrC, shrD]. u\text{-dom } u) = \{1, 2\}$
by (auto simp: shrC-def shrD-def)

next

fix $i \text{ } j$
assume $i: i < \text{length } [shrC, shrD] \text{ and } j: j < \text{length } [shrC, shrD]$
and $ne: i \neq j$
then have $(i = 0 \wedge j = 1) \vee (i = 1 \wedge j = 0)$
by auto
then show $u\text{-dom } ([shrC, shrD] ! i) \cap u\text{-dom } ([shrC, shrD] ! j) = \{\}$
by (auto simp: shrC-def shrD-def)

next

fix $u \text{ assume } u \in \text{set } [shrC, shrD]$
then show $wc.cle \text{ } (u\text{-lo } u) \text{ } (u\text{-hi } u)$
by (auto simp: shrC-def shrD-def wc.cle-def den-w-def)

next

fix $u \text{ assume } u \in \text{set } [shrC, shrD]$
then show $wc.cle \text{ } (u\text{-hi } u) \text{ } 2$
by (auto simp: shrC-def shrD-def wc.cle-def den-w-def)

next

fix $u \text{ } k \text{ assume } u: u \in \text{set } [shrC, shrD] \text{ and } k: k \in u\text{-dom } u$
show $\exists c. wc.cle \text{ } (u\text{-lo } u) \text{ } c \wedge wc.cle \text{ } c \text{ } (u\text{-hi } u) \wedge$
 $u\text{-refresh } u \text{ } k = \text{ state-after } s0\text{-w } (wc.pfx \text{ } c) \text{ } k$

proof (cases $u = shrC$)

```

case True
then have  $k = 1$  using  $k$  by (simp add: shrC-def)
then show ?thesis
  unfolding wc.cle-def wc.pfx-def
  by (intro exI[of - 1])
    (simp add: True shrC-def den-w-def L-w-def
      s0-w-def state-after-def apply-ev-def)
next
case False
then have  $uD: u = shrD$  using  $u$  by simp
then have  $k = 2$  using  $k$  by (simp add: shrD-def)
then show ?thesis
  unfolding wc.cle-def wc.pfx-def
  by (intro exI[of - 1])
    (simp add: uD shrD-def den-w-def L-w-def
      s0-w-def state-after-def apply-ev-def)
qed
qed

```

One bracket, amortized over both units, with ONE shared state witness (coordinate 1): Corollary 1 lifts the composite to T3 — the sink trajectory tracks the source from the witness onward. Amortization preserved the tier.

lemma *wit-shared-bracket-T3*:

```

assumes wc.cle 1 g and wc.cle g 2 and  $k \in \{1 :: nat, 2\}$ 
shows shr.sink-at g k = state-after s0-w (wc.pfx g) k
proof (rule shr.shared-witness-trajectory[of 1 g k])
  show  $\bigwedge u. u \in \text{set } [shrC, shrD] \implies wc.cle (u-lo\ u) 1 \wedge wc.cle 1 (u-hi\ u)$ 
    by (auto simp: shrC-def shrD-def wc.cle-def den-w-def)
  show  $\bigwedge u\ k. u \in \text{set } [shrC, shrD] \implies k \in u-dom\ u \implies$ 
     $u-refresh\ u\ k = state-after\ s0-w\ (wc.pfx\ 1)\ k$ 
    unfolding wc.pfx-def
    by (auto simp: shrC-def shrD-def den-w-def L-w-def s0-w-def
      state-after-def apply-ev-def)
qed (use assms in auto)

```

4.5 Fixture 4: a surviving refresh under window-discard

definition $L-s :: (nat, nat)\ event\ list$ **where**
 $L-s = [Event\ 3\ (Some\ 7), Event\ 1\ (Some\ 1)]$

interpretation *sc*: *coordinate-space L-s den-w*
by *unfold-locales (simp add: den-w-def L-s-def)*

definition *surv-unit* :: $(nat, nat, nat)\ cunit$ **where**
 $surv-unit = \langle u-dom = \{3\}, u-lo = 1, u-hi = 2,$
 $u-refresh = (\lambda k. \text{if } k = 3 \text{ then } Some\ 7 \text{ else } None) \rangle$

interpretation *surv*: *capture-contract L-s den-w s0-w [surv-unit] {3} 2*
proof (*unfold-locales*)

```

  show  $(\bigcup_{u \in \text{set } [\text{surv-unit}]} u\text{-dom } u) = \{3\}$ 
    by (simp add: surv-unit-def)
next
  fix i j
  assume  $i < \text{length } [\text{surv-unit}] \ j < \text{length } [\text{surv-unit}] \ i \neq j$ 
  then show  $u\text{-dom } ([\text{surv-unit}] ! i) \cap u\text{-dom } ([\text{surv-unit}] ! j) = \{\}$  by simp
next
  fix u assume  $u \in \text{set } [\text{surv-unit}]$ 
  then show  $sc.cle (u\text{-lo } u) (u\text{-hi } u)$ 
    by (simp add: surv-unit-def sc.cle-def den-w-def)
next
  fix u assume  $u \in \text{set } [\text{surv-unit}]$ 
  then show  $sc.cle (u\text{-hi } u) \ 2$ 
    by (simp add: surv-unit-def sc.cle-def den-w-def)
next
  fix u k assume  $u \in \text{set } [\text{surv-unit}]$  and  $k \in u\text{-dom } u$ 
  then show  $\exists c. sc.cle (u\text{-lo } u) \ c \wedge sc.cle \ c \ (u\text{-hi } u) \wedge$ 
     $u\text{-refresh } u \ k = \text{state-after } s0\text{-w } (sc.pfx \ c) \ k$ 
    unfolding sc.cle-def sc.pfx-def
    by (intro exI[of - 1])
      (simp add: surv-unit-def den-w-def
        L-s-def s0-w-def state-after-def apply-ev-def)
qed

```

```

lemma wit-surv-svl-ok:  $\text{surv.svl-ok } (\lambda\cdot. [3])$ 
proof -
  have  $\text{evs-for } 3 \ (sc.win (u\text{-lo } \text{surv-unit}) (u\text{-hi } \text{surv-unit})) = []$ 
    unfolding sc.win-def
    by (simp add: surv-unit-def den-w-def L-s-def evs-for-def)
  then have  $\text{surv.survivors } \text{surv-unit} = \{3\}$ 
    by (auto simp: surv.survivors-def surv-unit-def)
  then show ?thesis
    by (simp add: surv.svl-ok-def)
qed

```

Key 3's pre-window event (position 0, value 7) is consumed and emitted; the unit's window contains no event for key 3, so its refresh entry survives and is emitted at the window close — AFTER the pre-window event. The equivalence theorem says the stream replays to the canonical sink; the concrete evaluation confirms both sides at the value *Some* 7.

```

lemma wit-surv-replay-theorem:
   $\text{stream-replay } (\text{surv.emission } (\lambda\cdot. [3]) \ 0) \ 3 = \text{surv.sink-at } 2 \ 3$ 
proof (rule surv.window-discard-replay[OF wit-surv-svl-ok])
  show  $\bigwedge u. u \in \text{set } [\text{surv-unit}] \implies sc.cle \ 0 \ (u\text{-lo } u)$ 
    by (simp add: surv-unit-def sc.cle-def den-w-def)
qed simp

```

```

lemma wit-surv-the-unit [simp]:  $\text{surv.the-unit } 3 = \text{surv-unit}$ 
  using surv.the-unit-props[of 3] by simp

```

```

lemma wit-surv-replay-value:
  stream-replay (surv.emission ( $\lambda\cdot. [3]$ ) 0) 3 = Some 7
proof –
  have win-e: evs-for 3 (sc.win (u-lo surv-unit) 2) = []
    unfolding sc.win-def
    by (simp add: surv-unit-def den-w-def L-s-def evs-for-def)
  have surv.sink-at 2 3 = Some 7
    unfolding surv.sink-at-def wit-surv-the-unit win-e
    by (simp add: surv-unit-def)
  then show ?thesis
    using wit-surv-replay-theorem by simp
qed

end

```

```

theory Classic-Instance
  imports Cut-Theorem
           DBLog-Virtual-Cuts.DBLog-Run
           DBLog-Virtual-Cuts.Continuation
begin

```

5 Classic DBLog satisfies the contract

The classic watermarked DBLog formalization is imported as the separate session *DBLog_Virtual_Cuts*. Its shared-session refactoring preserves the published v2.1 mathematical content, as documented in *ARTIFACT_PROVENANCE.md* at the archive root. This theory proves that every wellformed classic DBLog run yields a capture plan satisfying the contract, so the contract-level cut theorem covers the classic instance. The classic model’s chunk evidence is SINGLE-COORDINATE (each chunk carries one read coordinate, and its wellformedness clause WF6 makes every in-domain read result equal the source state AT that one coordinate), which INSTANTIATES the contract’s per-key existential O2 — the classic instance imports upward into the strictly weaker obligation. The bridge direction (contract $\Rightarrow$ classic wellformedness) is not claimed.

The mapping, clause by clause (classic model name $\rightarrow$ contract name):

- source history H (coordinate-tagged source events, WF-H1 non-decreasing) $\rightarrow$ the log *log-of* H : Insert and Update collapse to a *Some* image, Delete to *None* (*img-of*) — 2020 §3.1’s all-columns post-image assumption is S-IMG at the import;
- a coordinate $c \rightarrow$ the prefix of all events with coordinate at-or-before c (*den-of*): under WF-H1 sortedness the at-or-before filter IS a list prefix. The declared endpoint normalization for this instance:

a coordinate denotes the prefix INCLUDING every event at that coordinate — the same at-or-before convention the classic model's own *latest-src-event* uses, and same-coordinate runs land wholly inside a denotation, matching the classic model's index tie-breaking (its *source-pos-order* resolves equal coordinates by position, and its latest-lookup takes the LAST qualifying position);

- $\text{chunk} \rightarrow \text{unit}$: $\text{domain} \rightarrow u\text{-dom}$; lower watermark $\rightarrow u\text{-lo}$; upper watermark TRUNCATED AT THE FRONTIER $\rightarrow u\text{-hi}$; the chunk's read map $\rightarrow u\text{-refresh}$. The truncation is needed because the classic model's wellformedness bounds the READ coordinate by the frontier (WF5) but deliberately never bounds the upper watermark — its own headline theorems never consume the watermark bracketing — while the contract's plans live at-or-before the frontier by definition. WF4 (bracketing) plus WF5 keep the read coordinate inside the truncated bracket, so nothing is lost;
- WF1(b)/(c) partition clauses $\rightarrow$ O1; WF4 totality + WF6 read validity at the read coordinate $\rightarrow$ O2 (the per-key existential witnessed uniformly by the chunk's one coordinate); O3 is the totality half of WF4 made representational; O5 is discharged by construction (one event vocabulary on both paths — the import maps the same history that generated the CDC stream).

The classic model's observed-CDC clauses (WF2, WF7) and its replay/certificate layers are NOT consumed here: the contract's log is the source history itself, and observation lives at the contract's discipline layer (S-OBS). The consistency corollary at the end ties the two developments' conclusions together: the contract's canonical sink at the frontier equals the classic model's *Src* there — the contract-level theorem generalizes the classic model's result and never contradicts it.

5.1 History-to-log mapping

definition $\text{img-of} :: ('k, 'v) \text{ source-event} \Rightarrow 'v \text{ option}$ **where**
 $\text{img-of } e = (\text{case } e \text{ of Insert - } v \Rightarrow \text{Some } v$
 $\quad \quad \quad | \text{Update - } v \Rightarrow \text{Some } v$
 $\quad \quad \quad | \text{Delete - } \Rightarrow \text{None})$

definition $\text{ev-of} :: \text{src-coord} \times ('k, 'v) \text{ source-event} \Rightarrow ('k, 'v) \text{ event}$ **where**
 $\text{ev-of } p = \text{Event } (\text{key-of } (\text{hist-event } p)) (\text{img-of } (\text{hist-event } p))$

definition $\text{log-of} :: ('k, 'v) \text{ src-history} \Rightarrow ('k, 'v) \text{ event list}$ **where**
 $\text{log-of } H = \text{map ev-of } H$

definition $\text{den-of} :: ('k, 'v) \text{ src-history} \Rightarrow \text{src-coord} \Rightarrow \text{nat}$ **where**

```

den-of H c = length (filter (λp. hist-coord p ≤ c) H)

lemma length-log-of [simp]: length (log-of H) = length H
  by (simp add: log-of-def)

lemma den-of-le-len: den-of H c ≤ length (log-of H)
  by (simp add: den-of-def)

lemma den-of-mono:
  assumes c ≤ c'
  shows den-of H c ≤ den-of H c'
  unfolding den-of-def
proof (induction H)
  case Nil
  then show ?case by simp
next
  case (Cons p H)
  then show ?case
    using assms by (auto intro: order.trans)
qed

```

5.2 Sorted histories: the at-or-before filter is a prefix

```

lemma sorted-filter-le-takeWhile:
  assumes sorted (map f xs)
  shows filter (λx. f x ≤ c) xs = takeWhile (λx. f x ≤ c) xs
  using assms
proof (induction xs)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  show ?case
  proof (cases f x ≤ c)
    case True
    then show ?thesis using Cons by simp
  next
    case False
    have ∧y. y ∈ set xs ⇒ ¬ f y ≤ c
    proof -
      fix y assume y ∈ set xs
      then have f x ≤ f y using Cons.prem by simp
      then show ¬ f y ≤ c using False by (auto intro: order.trans)
    qed
    then have filter (λx. f x ≤ c) xs = []
      by (simp add: filter-empty-conv)
    then show ?thesis using False by simp
  qed
qed

```

lemma *den-of-take*:
 assumes *sorted* (*map hist-coord H*)
 shows *take* (*den-of H c*) *H* = *filter* ($\lambda p. \text{hist-coord } p \leq c$) *H*
proof –
 have *filter* ($\lambda p. \text{hist-coord } p \leq c$) *H* = *takeWhile* ($\lambda p. \text{hist-coord } p \leq c$) *H*
 by (*rule sorted-filter-le-takeWhile[OF assms]*)
 then show ?thesis
 unfolding *den-of-def*
 by (*metis takeWhile-eq-take*)
qed

5.3 Generic filter-by-position normal form

lemma *filter-map-nth-upt*:
 $\text{filter } Q \text{ } xs = \text{map } (!) \text{ } xs \text{ } (\text{filter } (\lambda i. Q \text{ } (xs ! i)) [0..<\text{length } xs])$
proof –
 have *filter* *Q xs* = *filter* *Q* (*map* (*!*) *xs*) [*0..<length xs*])
 by (*simp add: map-nth*)
 also have ... = *map* (*!*) *xs* (*filter* (*Q* $\circ$ *!*) *xs*) [*0..<length xs*])
 by (*rule filter-map*)
 finally show ?thesis
 by (*simp add: comp-def*)
qed

5.4 State correspondence: the contract fold meets *Src*

lemma *evs-for-log-of*:
 $\text{evs-for } k \text{ } (\text{log-of } (\text{filter } (\lambda p. \text{hist-coord } p \leq c) \text{ } H))$
 $= \text{map ev-of } (\text{filter } (\lambda p. \text{hist-coord } p \leq c \wedge \text{key-of } (\text{hist-event } p) = k) \text{ } H)$
proof –
 have *evs-for* *k* (*log-of* (*filter* ($\lambda p. \text{hist-coord } p \leq c$) *H*))
 $= \text{map ev-of } (\text{filter } (\lambda p. \text{key-of } (\text{hist-event } p) = k)$
 $\quad (\text{filter } (\lambda p. \text{hist-coord } p \leq c) \text{ } H))$
 by (*simp add: log-of-def evs-for-def filter-map comp-def ev-of-def*)
 then show ?thesis
 by (*simp add: conj-commute*)
qed

lemma *state-correspondence*:
 fixes *b0* :: (*'k*, *'v*) *state* and *H* :: (*'k*, *'v*) *src-history*
 assumes *srt*: *sorted* (*map hist-coord H*)
 shows *state-after* *b0* (*take* (*den-of H c*) (*log-of H*)) *k* = *Src b0 H c k*
proof –
 have *take-log*: *take* (*den-of H c*) (*log-of H*)
 $= \text{log-of } (\text{filter } (\lambda p. \text{hist-coord } p \leq c) \text{ } H)$
 by (*simp add: log-of-def take-map den-of-take[OF srt]*)
 define *P* where
 $P = (\lambda p :: \text{src-coord} \times (\text{'k}, \text{'v}) \text{ source-event.}$
 $\quad \text{hist-coord } p \leq c \wedge \text{key-of } (\text{hist-event } p) = k)$

```

define cand where cand = filter ( $\lambda i. P (H ! i)$ ) [ $0..<\text{length } H$ ]
have elems: filter P H = map ( $(!) H$ ) cand
  unfolding cand-def by (rule filter-map-nth-upt)
have evs: evs-for k (take (den-of H c) (log-of H)) = map ev-of (map ( $(!) H$ )
cand)
  by (simp add: take-log evs-for-log-of elems flip: P-def)
have latest: latest-src-event H c k
  = (if cand = [] then None else Some (last cand))
  unfolding latest-src-event-def cand-def P-def
  by (simp add: src-le-eq-less-eq Let-def conj-commute)
show ?thesis
proof (cases cand = [])
  case True
  then have evs-for k (take (den-of H c) (log-of H)) = []
    by (simp add: evs)
  then have lhs: state-after b0 (take (den-of H c) (log-of H)) k = b0 k
    by (simp add: state-after-no-events)
  have Src b0 H c k = b0 k
    using latest True by (simp add: Src-def)
  then show ?thesis using lhs by simp
next
  case False
  then have ne: evs-for k (take (den-of H c) (log-of H))  $\neq$  []
    by (simp add: evs)
  have lhs: state-after b0 (take (den-of H c) (log-of H)) k
    = ev-img (ev-of (H ! last cand))
    using state-after-last-event[OF ne] evs False
    by (simp add: last-map)
  have Src b0 H c k = img-of (hist-event (H ! last cand))
    using latest False
    by (simp add: Src-def img-of-def split: source-event.split)
  then show ?thesis
    using lhs by (simp add: ev-of-def)
qed
qed

```

5.5 Derived units and the import theorem

definition *unit-of-chunk* ::
 ($'k :: \text{linorder}, 'v$) $\text{run} \Rightarrow \text{nat} \Rightarrow ('k, 'v, \text{src-coord}) \text{cunit}$ **where**
unit-of-chunk *R ch* =
 ($\llbracket u\text{-dom} = \text{chunk-domain } R \text{ ch},$
 $u\text{-lo} = \text{chunk-lower-watermark } R \text{ ch},$
 $u\text{-hi} = \min (\text{chunk-upper-watermark } R \text{ ch}) (\text{frontier-of } R),$
 $u\text{-refresh} = (\lambda k. \text{case } \text{chunk-read-result } R \text{ ch } k \text{ of}$
 $\quad \text{Some } m \Rightarrow m \mid \text{None} \Rightarrow \text{None}) \rrbracket$)

definition *units-of* ::
 ($'k :: \text{linorder}, 'v$) $\text{run} \Rightarrow ('k, 'v, \text{src-coord}) \text{cunit list}$ **where**

$units-of\ R = map\ (unit-of-chunk\ R)\ (chunks-list\ R)$

Projections of the wellformedness clauses this import consumes. The wellformedness body is one large conjunction; searchful methods are unreliable at that hypothesis size, so each projection first splits the body into its verbatim conjuncts by deterministic conjunction elimination plus assumption matching, then instantiates the one small fact it needs.

lemma *wf-src-binding*:

assumes *wellformed-dblog-run* $b0\ R\ H$

shows *src-history-of* $R = H$

using *assms* **unfolding** *wellformed-dblog-run-def* **by** (*rule conjunct1*)

lemma *wf-src-wellformed*:

assumes *wf*: *wellformed-dblog-run* $b0\ R\ H$

shows *wellformed-src-history* H

proof –

note *wf-body* = *wf*[*unfolded wellformed-dblog-run-def*]

from *wf-body* **show** *?thesis* **by** (*elim conjE*)

qed

lemma *wf-domains-disjoint*:

assumes *wf*: *wellformed-dblog-run* $b0\ R\ H$

and *ch*: $ch1 \in chunks\ R\ ch2 \in chunks\ R\ ch1 \neq ch2$

shows *chunk-domain* $R\ ch1 \cap chunk-domain\ R\ ch2 = \{\}$

proof –

have *all*: $\forall ch1 \in chunks\ R. \forall ch2 \in chunks\ R.$

$ch1 \neq ch2 \longrightarrow chunk-domain\ R\ ch1 \cap chunk-domain\ R\ ch2 = \{\}$

using *wf*[*unfolded wellformed-dblog-run-def*] **by** (*elim conjE*)

from *bspec*[*OF* *bspec*[*OF* *all ch(1)*] *ch(2)*] *ch(3)* **show** *?thesis* **by** *blast*

qed

lemma *wf-domains-cover*:

assumes *wf*: *wellformed-dblog-run* $b0\ R\ H$

shows *canonical-chunk-ownership-domain* $R = scope-of\ R$

proof –

note *wf-body* = *wf*[*unfolded wellformed-dblog-run-def*]

from *wf-body* **show** *?thesis* **by** (*elim conjE*)

qed

lemma *wf-read-evidence*:

assumes *wellformed-dblog-run* $b0\ R\ H$

shows $\forall ch \in chunks\ R.$

$src-le\ (chunk-lower-watermark\ R\ ch)\ (chunk-read-coordinate\ R\ ch)$

$\wedge\ src-le\ (chunk-read-coordinate\ R\ ch)\ (chunk-upper-watermark\ R\ ch)$

$\wedge\ (\forall k \in chunk-domain\ R\ ch.$

$\exists m. chunk-read-result\ R\ ch\ k = Some\ m)$

$\wedge\ (\forall k \in chunk-domain\ R\ ch. \forall m.$

$chunk-read-result\ R\ ch\ k = Some\ m$

$\longleftrightarrow Refresh\ k\ m\ (chunk-read-coordinate\ R\ ch)$

$\in \text{set } (\text{clean-prefix-of } R)$

using *assms*[*unfolded wellformed-dblog-run-def*] **by** (*elim conjE*)

lemma *wf-watermark-bracket*:

assumes *wf*: *wellformed-dblog-run* *b0* *R* *H* **and** *ch*: *ch* $\in$ *chunks* *R*

shows *chunk-lower-watermark* *R* *ch* $\leq$ *chunk-read-coordinate* *R* *ch*

and *chunk-read-coordinate* *R* *ch* $\leq$ *chunk-upper-watermark* *R* *ch*

using *bspec*[*OF wf-read-evidence*[*OF wf*] *ch*]

by (*simp-all add: src-le-eq-less-eq*)

lemma *wf-read-total*:

assumes *wf*: *wellformed-dblog-run* *b0* *R* *H*

and *ch*: *ch* $\in$ *chunks* *R* **and** *k*: *k* $\in$ *chunk-domain* *R* *ch*

shows $\exists m. \text{chunk-read-result } R \text{ } ch \text{ } k = \text{Some } m$

using *bspec*[*OF wf-read-evidence*[*OF wf*] *ch*] *k* **by** *blast*

lemma *wf-read-before-frontier*:

assumes *wf*: *wellformed-dblog-run* *b0* *R* *H* **and** *ch*: *ch* $\in$ *chunks* *R*

shows *chunk-read-coordinate* *R* *ch* $\leq$ *frontier-of* *R*

proof –

have *all*: $\forall ch \in \text{chunks } R. \text{src-le } (\text{chunk-read-coordinate } R \text{ } ch) (\text{frontier-of } R)$

using *wf*[*unfolded wellformed-dblog-run-def*] **by** (*elim conjE*)

from *bspec*[*OF all ch*] **show** *?thesis* **by** (*simp add: src-le-eq-less-eq*)

qed

lemma *wf-read-honest*:

assumes *wf*: *wellformed-dblog-run* *b0* *R* *H*

and *ch*: *ch* $\in$ *chunks* *R* **and** *k*: *k* $\in$ *chunk-domain* *R* *ch*

and *m*: *chunk-read-result* *R* *ch* *k* = *Some* *m*

shows *m* = *Src* *b0* *H* (*chunk-read-coordinate* *R* *ch*) *k*

proof –

have *all*: $\forall ch \in \text{chunks } R. \forall k \in \text{chunk-domain } R \text{ } ch. \forall m. \text{chunk-read-result } R \text{ } ch \text{ } k = \text{Some } m \longrightarrow m = \text{Src } b0 \text{ } H \text{ } (\text{chunk-read-coordinate } R \text{ } ch) \text{ } k$

using *wf*[*unfolded wellformed-dblog-run-def*] **by** (*elim conjE*)

from *bspec*[*OF bspec*[*OF all ch*] *k*] *m* **show** *?thesis* **by** *blast*

qed

theorem *classic-imports-upward*:

assumes *wf*: *wellformed-dblog-run* *b0* *R* *H*

shows *capture-contract* (*log-of* *H*) (*den-of* *H*) *b0* (*units-of* *R*)

(*scope-of* *R*) (*frontier-of* *R*)

proof –

have *wf-h*: *wellformed-src-history* *H*

by (*rule wf-src-wellformed*[*OF wf*])

have *srt*: *sorted* (*map hist-coord* *H*)

using *wf-h* **by** (*rule wellformed-src-history-sorted*)

interpret *pc*: *coordinate-space* *log-of* *H* *den-of* *H*

by *unfold-locales* (*rule den-of-le-len*)

```

show ?thesis
proof (unfold-locale)
  show ( $\bigcup u \in \text{set } (\text{units-of } R). \text{u-dom } u = \text{scope-of } R$ )
    using wf-domains-cover[OF wf]
    by (simp add: units-of-def unit-of-chunk-def
      canonical-chunk-ownership-domain-def chunks-list-set)
next
  fix i j
  assume i:  $i < \text{length } (\text{units-of } R)$  and j:  $j < \text{length } (\text{units-of } R)$ 
    and ne:  $i \neq j$ 
  have li:  $i < \text{length } (\text{chunks-list } R)$  and lj:  $j < \text{length } (\text{chunks-list } R)$ 
    using i j by (simp-all add: units-of-def)
  have mem-i:  $\text{chunks-list } R ! i \in \text{chunks } R$ 
    and mem-j:  $\text{chunks-list } R ! j \in \text{chunks } R$ 
    using li lj chunks-list-set by (metis nth-mem)+
  have chunks-list  $R ! i \neq \text{chunks-list } R ! j$ 
    using chunks-list-distinct li lj ne nth-eq-iff-index-eq by blast
  then have chunk-domain  $R (\text{chunks-list } R ! i)$ 
     $\cap \text{chunk-domain } R (\text{chunks-list } R ! j) = \{\}$ 
    by (rule wf-domains-disjoint[OF wf mem-i mem-j])
  then show  $\text{u-dom } (\text{units-of } R ! i) \cap \text{u-dom } (\text{units-of } R ! j) = \{\}$ 
    using li lj by (simp add: units-of-def unit-of-chunk-def)
next
  fix u assume  $u \in \text{set } (\text{units-of } R)$ 
  then obtain ch where ch:  $ch \in \text{chunks } R$  and u:  $u = \text{unit-of-chunk } R \text{ ch}$ 
    by (auto simp: units-of-def chunks-list-set)
  have rc-min:  $\text{chunk-read-coordinate } R \text{ ch}$ 
     $\leq \min (\text{chunk-upper-watermark } R \text{ ch}) (\text{frontier-of } R)$ 
    using wf-watermark-bracket(2)[OF wf ch] wf-read-before-frontier[OF wf ch]
    by (simp add: min.bounded-iff)
  have chunk-lower-watermark  $R \text{ ch}$ 
     $\leq \min (\text{chunk-upper-watermark } R \text{ ch}) (\text{frontier-of } R)$ 
    using wf-watermark-bracket(1)[OF wf ch] rc-min
    by (rule order.trans)
  then show  $\text{pc.cle } (u\text{-lo } u) (u\text{-hi } u)$ 
    by (simp add: u unit-of-chunk-def pc.cle-def den-of-mono)
next
  fix u assume  $u \in \text{set } (\text{units-of } R)$ 
  then obtain ch where ch:  $ch \in \text{chunks } R$  and u:  $u = \text{unit-of-chunk } R \text{ ch}$ 
    by (auto simp: units-of-def chunks-list-set)
  show  $\text{pc.cle } (u\text{-hi } u) (\text{frontier-of } R)$ 
    by (simp add: u unit-of-chunk-def pc.cle-def den-of-mono)
next
  fix u k assume  $u \in \text{set } (\text{units-of } R)$  and k:  $k \in \text{u-dom } u$ 
  then obtain ch where ch:  $ch \in \text{chunks } R$  and u:  $u = \text{unit-of-chunk } R \text{ ch}$ 
    by (auto simp: units-of-def chunks-list-set)
  have k-dom:  $k \in \text{chunk-domain } R \text{ ch}$ 
    using k by (simp add: u unit-of-chunk-def)
  obtain m where m:  $\text{chunk-read-result } R \text{ ch } k = \text{Some } m$ 

```

```

    using wf-read-total[OF wf ch k-dom] by blast
  have refresh: u-refresh u k = m
    by (simp add: u unit-of-chunk-def m)
  have honest: m = Src b0 H (chunk-read-coordinate R ch) k
    by (rule wf-read-honest[OF wf ch k-dom m])
  show  $\exists c. pc.cle (u-lo u) c \wedge pc.cle c (u-hi u) \wedge$ 
     $u-refresh u k = state-after b0 (pc.pfx c) k$ 
  proof (intro exI[of - chunk-read-coordinate R ch] conjI)
    show pc.cle (u-lo u) (chunk-read-coordinate R ch)
      using wf-watermark-bracket(1)[OF wf ch]
      by (simp add: u unit-of-chunk-def pc.cle-def den-of-mono)
    have chunk-read-coordinate R ch
       $\leq \min (chunk-upper-watermark R ch) (frontier-of R)$ 
      using wf-watermark-bracket(2)[OF wf ch] wf-read-before-frontier[OF wf ch]
      by (simp add: min.bounded-iff)
    then show pc.cle (chunk-read-coordinate R ch) (u-hi u)
      by (simp add: u unit-of-chunk-def pc.cle-def den-of-mono)
    show u-refresh u k = state-after b0 (pc.pfx (chunk-read-coordinate R ch)) k
      using refresh honest state-correspondence[OF srt]
      by (simp add: pc.pfx-def)
  qed
qed
qed
qed

```

5.6 Consistency with the classic model's conclusion

```

corollary classic-sink-is-Src:
  assumes wf: wellformed-dblog-run b0 R H
    and k: k  $\in$  scope-of R
  shows capture-plan.sink-at (log-of H) (den-of H) (units-of R)
    (frontier-of R) k
    = Src b0 H (frontier-of R) k
  proof -
    interpret cc: capture-contract log-of H den-of H b0 units-of R
      scope-of R frontier-of R
    by (rule classic-imports-upward[OF wf])
    have srt: sorted (map hist-coord H)
      using wf-src-wellformed[OF wf] by (rule wellformed-src-history-sorted)
    have cc.sink-at (frontier-of R) k
      = state-after b0 (cc.pfx (frontier-of R)) k
      by (rule cc.contract-cut[OF k])
    also have ... = Src b0 H (frontier-of R) k
      using state-correspondence[OF srt] by (simp add: cc.pfx-def)
    finally show ?thesis .
  qed
end

```

```

theory ReadOnly-Instance
  imports Merge-Disciplines
begin

```

6 A write-free instance: read-only incremental snapshots over executed-GTID sets

This theory models read-only incremental snapshots in the style of Debezium. The read-only incremental snapshot takes the executed global-transaction-ID set as low and high watermarks by `READING` it before and after each chunk select ("uses the executed global transaction IDs (GTID) set as high and low watermarks"); an event is in the window iff its GTID lies in the high set minus the low set; dedup is an in-memory window-discard.

The formalization:

- *gtid-history*: one commit-ordered log whose positions carry GTIDs, with the single assumption that equal GTIDs occupy `CONTIGUOUS` position blocks (a transaction commits as one contiguous run of row events). *L* contains completed transaction blocks only: no event from an active or aborted transaction inhabits the history. This is the model-level content of the single-linear-history requirements — named here the **A-linear** assumption class: *gtid-mode* enforced, and, when the connector reads a replica, the commit-order-preserving apply that makes the replica's history `THE` history. A configuration violating these is outside the instance's premises, exactly as the contract's `T0` boundary prescribes.
- Executed-set snapshots at transaction boundaries are the set-valued coordinates (no total order on '*g set* is available or needed — subset inclusion is partial; the `DENOTATIONS` are nested). *gden* recovers the denoted prefix length; *gset-inj-on-boundaries* makes it well-defined.
- *oracle-agreement*: the instance's own operational membership decision — "GTID in high-set minus low-set" — agrees exactly with the denotational window. This discharges the operational-fidelity requirement for this instance; it is the lemma this instance rests on.
- *readonly-incremental*: the plan locale; its *A-read* assumption is `O2` verbatim at instance level — per-key in-bracket validity enters as a `NAMED` assumption rather than being derived from an isolation level, and each read witness sits at a transaction boundary (reads see committed states). The sublocale then instantiates the full contract: the cut theorem and the window-discard equivalence follow.

No watermark writes: the locale constrains only `READS` — the bracket coordinates are observed executed-set snapshots, and no assumption places

any marker/watermark event in L . Contrast the classic instance, whose brackets ARE log positions of watermark UPDATE events. Heartbeat-driven window-closure detection is a liveness device for DECIDING when the high snapshot may be taken; it has no safety content and is deliberately not a locale assumption.

6.1 GTID-labeled histories and executed-set coordinates

locale *gtid-history* =
fixes $L :: ('k, 'v) \text{ event list}$
and $gtid :: nat \Rightarrow 'g$
assumes *contiguous*:
 $\bigwedge p \ q \ r. p \leq q \implies q \leq r \implies r < \text{length } L \implies$
 $gtid \ p = gtid \ r \implies gtid \ q = gtid \ p$
begin

definition $gset :: nat \Rightarrow 'g \text{ set}$ **where**
 $gset \ n = gtid \ \{p. p < n\}$

A position n is a (transaction) boundary when no transaction straddles it: no earlier position shares a GTID with any position at or after n . Executed-set snapshots are taken between transactions, so observed coordinates live exactly on boundaries.

definition $boundary :: nat \Rightarrow bool$ **where**
 $boundary \ n \longleftrightarrow$
 $n \leq \text{length } L \wedge$
 $(\forall p \ q. p < n \longrightarrow n \leq q \longrightarrow q < \text{length } L \longrightarrow gtid \ p \neq gtid \ q)$

lemma *boundary-zero*: $boundary \ 0$
by (*simp add: boundary-def*)

lemma *boundary-len*: $boundary \ (\text{length } L)$
by (*simp add: boundary-def*)

lemma *boundary-le-len*: $boundary \ n \implies n \leq \text{length } L$
by (*simp add: boundary-def*)

lemma *gset-mono*: $m \leq n \implies gset \ m \subseteq gset \ n$
by (*auto simp: gset-def*)

lemma *gset-strict-new*:
assumes bm : $boundary \ m$ **and** $m\text{-len}$: $m < \text{length } L$
shows $gtid \ m \notin gset \ m$
proof
assume $gtid \ m \in gset \ m$
then obtain p **where** $p < m$ **and** $gtid \ p = gtid \ m$
by (*auto simp: gset-def*)
then show *False*

```

    using bm m-len by (auto simp: boundary-def)
qed

lemma gset-inj-on-boundaries:
  assumes bm: boundary m and bn: boundary n and eq: gset m = gset n
  shows m = n
proof (rule ccontr)
  assume ne: m ≠ n
  then consider (mn) m < n | (nm) n < m
    by linarith
  then show False
  proof cases
    case mn
    have m-len: m < length L
    using mn boundary-le-len[OF bn] by simp
    have gtid m ∈ gset n
    using mn by (auto simp: gset-def)
    then show False
    using gset-strict-new[OF bm m-len] eq by simp
  next
    case nm
    have n-len: n < length L
    using nm boundary-le-len[OF bm] by simp
    have gtid n ∈ gset m
    using nm by (auto simp: gset-def)
    then show False
    using gset-strict-new[OF bn n-len] eq by simp
  qed
qed

definition gden :: 'g set ⇒ nat where
  gden S = (if ∃ n. boundary n ∧ gset n = S
    then THE n. boundary n ∧ gset n = S else 0)

lemma gden-gset [simp]:
  assumes b: boundary n
  shows gden (gset n) = n
proof -
  have (THE m. boundary m ∧ gset m = gset n) = n
  proof (rule the-equality)
    show boundary n ∧ gset n = gset n
    using b by simp
  next
    fix m assume boundary m ∧ gset m = gset n
    then show m = n
    using b gset-inj-on-boundaries by blast
  qed
  then show ?thesis
  using b by (auto simp: gden-def)

```

qed

lemma *gden-le-len*: $gden\ S \leq length\ L$
proof (*cases* $\exists n. boundary\ n \wedge gset\ n = S$)
 case True
 then obtain n **where** n : $boundary\ n\ gset\ n = S$
 by *blast*
 then have $gden\ S = n$
 using *gden-gset* **by** *blast*
 then show *?thesis*
 using *boundary-le-len*[*OF* $n(1)$] **by** *simp*
next
 case False
 then have $gden\ S = 0$
 unfolding *gden-def* **by** (*rule if-not-P*)
 then show *?thesis* **by** *simp*
qed
end

sublocale *gtid-history* $\subseteq$ *gcs*: *coordinate-space* $L\ gden$
 by *unfold-locales* (*rule gden-le-len*)

context *gtid-history*
begin

The membership-oracle agreement: for coordinates observed at boundaries, the instance’s operational decision “the event’s GTID is in the high set and not in the low set” holds exactly when the event’s position lies in the denotational half-open window. This is the operational-fidelity requirement, discharged for the executed-GTID-set coordinate space.

theorem *oracle-agreement*:
 assumes blo : $boundary\ nlo$ **and** bhi : $boundary\ nhi$
 and p : $p < length\ L$
 shows $gtid\ p \in gset\ nhi - gset\ nlo \longleftrightarrow nlo \leq p \wedge p < nhi$
proof
 assume mem : $gtid\ p \in gset\ nhi - gset\ nlo$
 have *not-lt*: $\neg p < nlo$
 proof
 assume $p < nlo$
 then have $gtid\ p \in gset\ nlo$
 by (*auto simp: gset-def*)
 then show *False* **using** mem **by** *simp*
 qed
 have $p < nhi$
proof (*rule ccontr*)
 assume $\neg p < nhi$
 then have $hi-le-p$: $nhi \leq p$ **by** *simp*
 from mem **obtain** q **where** $q < nhi\ gtid\ q = gtid\ p$

```

    by (auto simp: gset-def)
  then show False
    using bhi hi-le-p p by (auto simp: boundary-def)
qed
with not-lt show  $nlo \leq p \wedge p < nhi$  by simp
next
  assume  $nlo \leq p \wedge p < nhi$ 
  then have lo-le:  $nlo \leq p$  and lt-hi:  $p < nhi$  by auto
  have gtid  $p \in gset\ nhi$ 
    using lt-hi by (auto simp: gset-def)
  moreover have gtid  $p \notin gset\ nlo$ 
  proof
    assume gtid  $p \in gset\ nlo$ 
    then obtain q where  $q < nlo$  and gtid  $q = gtid\ p$ 
      by (auto simp: gset-def)
    then show False
      using blo lo-le p by (auto simp: boundary-def)
  qed
  ultimately show gtid  $p \in gset\ nhi - gset\ nlo$  by simp
qed

corollary oracle-agreement-den:
  assumes blo: boundary nlo and bhi: boundary nhi
    and p:  $p < length\ L$ 
  shows gtid  $p \in gset\ nhi - gset\ nlo \longleftrightarrow$ 
    gden ( $gset\ nlo$ )  $\leq p \wedge p < gden\ (gset\ nhi)$ 
  using oracle-agreement[OF blo bhi p] by (simp add: blo bhi)

end

```

6.2 The read-only incremental plan

```

locale readonly-incremental = gtid-history L gtid
  for L :: ('k, 'v) event list and gtid :: nat  $\Rightarrow$  'g +
  fixes  $\sigma 0$  :: ('k, 'v) state
    and units :: ('k, 'v, 'g set) cunit list
    and scope :: 'k set
    and nf :: nat
    and ulo uhi :: ('k, 'v, 'g set) cunit  $\Rightarrow$  nat
  assumes f-boundary: boundary nf
    and lo-boundary:  $\bigwedge u. u \in set\ units \implies$ 
      boundary (ulo u)  $\wedge u-lo\ u = gset\ (ulo\ u)$ 
    and hi-boundary:  $\bigwedge u. u \in set\ units \implies$ 
      boundary (uhi u)  $\wedge u-hi\ u = gset\ (uhi\ u)$ 
    and bracket-ord:  $\bigwedge u. u \in set\ units \implies ulo\ u \leq uhi\ u$ 
    and bracket-f:  $\bigwedge u. u \in set\ units \implies uhi\ u \leq nf$ 
    and RO1-cover:  $(\bigcup u \in set\ units. u-dom\ u) = scope$ 
    and RO1-disjoint:  $\bigwedge i\ j. i < length\ units \implies j < length\ units \implies$ 
       $i \neq j \implies u-dom\ (units\ !\ i) \cap u-dom\ (units\ !\ j) = \{\}$ 

```

and *A-read*: $\bigwedge u k. u \in \text{set units} \implies k \in u\text{-dom } u \implies$
 $\exists n. \text{boundary } n \wedge \text{ulo } u \leq n \wedge n \leq \text{uhi } u \wedge$
 $u\text{-refresh } u k = \text{state-after } \sigma 0 \text{ (take } n L) k$

The named assumptions: the locale's history shape is A-linear (single linear history, contiguous transactions); *A-read* is the chunk-select read assumption (O2 at instance level, witnesses at transaction boundaries); bracket coordinates are OBSERVED executed-set snapshots (*lo-boundary/hi-boundary*, with no watermark writes). Everything else is the plan shape (partition, bracket order, frontier bound).

sublocale *readonly-incremental* $\subseteq$ *rc*: *capture-contract* *L* *gden* $\sigma 0$ *units* *scope* *gset* *nf*

proof (*unfold-locales*)

show $(\bigcup u \in \text{set units}. u\text{-dom } u) = \text{scope}$
by (*rule RO1-cover*)

next

fix *i j*

assume $i < \text{length units } j < \text{length units } i \neq j$

then show $u\text{-dom } (\text{units } ! i) \cap u\text{-dom } (\text{units } ! j) = \{\}$

by (*rule RO1-disjoint*)

next

fix *u* **assume** $u: u \in \text{set units}$

show *gcs.cle* (*u-lo* *u*) (*u-hi* *u*)

using *lo-boundary*[*OF* *u*] *hi-boundary*[*OF* *u*] *bracket-ord*[*OF* *u*]

by (*simp add: gcs.cle-def*)

next

fix *u* **assume** $u: u \in \text{set units}$

show *gcs.cle* (*u-hi* *u*) (*gset* *nf*)

using *hi-boundary*[*OF* *u*] *bracket-f*[*OF* *u*] *f-boundary*

by (*simp add: gcs.cle-def*)

next

fix *u k* **assume** $u: u \in \text{set units}$ **and** $k: k \in u\text{-dom } u$

from *A-read*[*OF* *u k*] **obtain** *n* **where**

n: *boundary* *n* *ulo* *u* $\leq n$ $n \leq \text{uhi } u$

u-refresh *u k* = *state-after* $\sigma 0$ (*take* *n L*) *k*

by *blast*

show $\exists c. \text{gcs.cle } (u\text{-lo } u) c \wedge \text{gcs.cle } c (u\text{-hi } u) \wedge$

u-refresh *u k* = *state-after* $\sigma 0$ (*gcs.pfx* *c*) *k*

proof (*intro exI*[*of* - *gset* *n*] *conjI*)

show *gcs.cle* (*u-lo* *u*) (*gset* *n*)

using *lo-boundary*[*OF* *u*] *n*(1,2) **by** (*simp add: gcs.cle-def*)

show *gcs.cle* (*gset* *n*) (*u-hi* *u*)

using *hi-boundary*[*OF* *u*] *n*(1,3) **by** (*simp add: gcs.cle-def*)

show *u-refresh* *u k* = *state-after* $\sigma 0$ (*gcs.pfx* (*gset* *n*)) *k*

using *n*(1,4) **by** (*simp add: gcs.pfx-def*)

qed

qed

context *readonly-incremental*

begin

The write-free plan is a contract instance, so THE theorem applies — the canonical replay at the frontier snapshot is exactly the source state there (tier T2), and the in-memory window-discard discipline the connector actually runs is covered by the §6 equivalence theorem, reused verbatim.

corollary *readonly-cut*:

assumes $k \in \text{scope}$

shows $\text{rc.sink-at } (gset \text{ nf}) \ k = \text{state-after } \sigma 0 \ (take \text{ nf } L) \ k$

proof –

have $\text{rc.sink-at } (gset \text{ nf}) \ k = \text{state-after } \sigma 0 \ (\text{gcs.pfx } (gset \text{ nf})) \ k$

by $(rule \ \text{rc.contract-cut}[OF \ \text{assms}])$

then show *?thesis*

by $(simp \ add: \ \text{gcs.pfx-def } f\text{-boundary})$

qed

corollary *readonly-window-discard*:

assumes $\text{rc.svl-ok } \text{svl}$

and $\bigwedge u. u \in \text{set units} \implies \text{gcs.cle } s0g \ (u\text{-lo } u)$

and $k \in \text{scope}$

shows $\text{stream-replay } (\text{rc.emission } \text{svl } s0g) \ k = \text{rc.sink-at } (gset \text{ nf}) \ k$

by $(rule \ \text{rc.window-discard-replay}[OF \ \text{assms}])$

The two proof boundaries composed explicitly: an exact survivor enumeration and consumption beginning at or before every low edge make the formal window-discard emission replay to the canonical sink, and the contract cut then identifies that final replay with the source state at the observed executed-set frontier. The equality is from the empty stream-replay state and is not a physical-prefix or delivery guarantee.

corollary *readonly-emitted-cut*:

assumes $\text{svl}: \text{rc.svl-ok } \text{svl}$

and $s0lo: \bigwedge u. u \in \text{set units} \implies \text{gcs.cle } s0g \ (u\text{-lo } u)$

and $k: k \in \text{scope}$

shows $\text{stream-replay } (\text{rc.emission } \text{svl } s0g) \ k = \text{state-after } \sigma 0 \ (take \text{ nf } L) \ k$

using $\text{readonly-window-discard}[OF \ \text{svl } s0lo \ k] \ \text{readonly-cut}[OF \ k]$

by *simp*

end

6.3 Constructed witness (non-vacuity)

definition $L\text{-r} :: (\text{nat}, \text{nat}) \ \text{event list} \ \text{where}$

$L\text{-r} = [\text{Event } 1 \ (\text{Some } 5), \ \text{Event } 2 \ (\text{Some } 6)]$

definition $\text{gtid-r} :: \text{nat} \Rightarrow \text{nat} \ \text{where}$

$\text{gtid-r } p = p$

interpretation $gh: \text{gtid-history } L\text{-r } \text{gtid-r}$

```

by unfold-locales (auto simp: gtid-r-def)

lemma gh-boundary-all:  $n \leq 2 \implies gh.boundary\ n$ 
  unfolding gh.boundary-def
  by (auto simp: gtid-r-def L-r-def)

definition unit-r :: (nat, nat, nat set) cunit where
  unit-r = ( $\llbracket u-dom = \{1, 2\}, u-lo = gh.gset\ 0, u-hi = gh.gset\ 2,$ 
     $u-refresh = (\lambda k. \text{if } k = 1 \text{ then Some } 5$ 
       $\text{else if } k = 2 \text{ then Some } 6 \text{ else None}) \rrbracket$ )

interpretation rw: readonly-incremental L-r gtid-r  $\lambda\cdot$ . None [unit-r]
  {1, 2} 2  $\lambda\cdot$ . 0  $\lambda\cdot$ . 2
proof (unfold-locales)
  show gh.boundary 2
    by (simp add: gh-boundary-all)
next
  fix u assume u  $\in$  set [unit-r]
  then show gh.boundary 0  $\wedge$  u-lo u = gh.gset 0
    by (simp add: unit-r-def gh-boundary-all)
next
  fix u assume u  $\in$  set [unit-r]
  then show gh.boundary 2  $\wedge$  u-hi u = gh.gset 2
    by (simp add: unit-r-def gh-boundary-all)
next
  fix u assume u  $\in$  set [unit-r]
  then show (0 :: nat)  $\leq$  2 by simp
next
  fix u assume u  $\in$  set [unit-r]
  then show (2 :: nat)  $\leq$  2 by simp
next
  show ( $\bigcup u \in \text{set } [unit-r]. u-dom\ u = \{1, 2\}$ )
    by (simp add: unit-r-def)
next
  fix i j
  assume i < length [unit-r] j < length [unit-r] i  $\neq$  j
  then show u-dom ([unit-r] ! i)  $\cap$  u-dom ([unit-r] ! j) = {}
    by simp
next
  fix u k assume u  $\in$  set [unit-r] and k  $\in$  u-dom u
  then show  $\exists n. gh.boundary\ n \wedge 0 \leq n \wedge n \leq 2 \wedge$ 
     $u-refresh\ u\ k = state-after\ (\lambda\cdot$ . None) (take n L-r) k
    by (intro exI[of - 2] conjI gh-boundary-all)
      (auto simp: unit-r-def L-r-def state-after-def apply-ev-def)
qed

lemma wit-readonly-cut:
  k  $\in$  {1, 2}  $\implies$ 
    rw.rc.sink-at (gh.gset 2) k = state-after ( $\lambda\cdot$ . None) (take 2 L-r) k

```

```

by (rule rw.readonly-cut)

lemma wit-readonly-oracle:
  gtid-r 1 ∈ gh.gset 2 − gh.gset 0 ∧ gtid-r 0 ∉ gh.gset 0
by (auto simp: gh.gset-def gtid-r-def L-r-def)

end

```

```

theory Range-Merge
  imports Merge-Disciplines
begin

```

7 The buffered-range-merge discipline

The second real merge discipline of the equivalence catalog, and the engine of the parallel-chunks instance: read the unit’s window events and *upsert* them onto the refresh buffer, then emit the merged unit ONCE when the window closes. The Flink CDC shape: “Upsert the read binlog records into the buffered chunk records, and emit all records in the buffer as final output” — all as INSERT records, denoting row state as of the chunk’s high watermark.

The equivalence obligation is discharged under two clauses:

- **(a) withholding:** the unit’s keys are withheld from stream emission over $(s_0, hi_i]$ — the merged unit is their sole pre- hi_i emission. Formalized by the gate *rm-gate*: a scoped event passes to the stream only at-or-after its covering unit’s high edge. Without this clause a pre- lo_i event for a later-deleted key replays to the wrong value; the fixture at the end of this theory exhibits exactly that failure, so the clause is load-bearing, not decorative.
- **(b) value-or-absence at the high edge:** the merged unit denotes value-or-absence AT hi_i per key — the key’s LAST in-window event decides (*merged*), so a delete-then-reinsert key is present again and emits normally. Keys absent at hi_i are emitted ZERO times; under the empty-map stream-replay convention (*stream-replay*) zero-emission IS the absence result. Theorem *merged-at-hi* makes clause (b) exact: the merged buffer equals the source state at the unit’s high edge, key by key.

The discipline’s observability premise differs from window-discard’s in a way worth naming: window-discard consumes a single pass from $s_0 \sqsubseteq lo_i$, while range-merge reads each window by a RANGED backfill (“read the binlog records that belong to the snapshot chunk from LOW offset to HIGH offset” — coordinate-addressable access, the S-OBS retention requirement),

so the STREAM only needs $s_0 \sqsubseteq hi_i$: everything below a unit's high edge that the stream would contribute is withheld by clause (a) anyway, and the window content enters through the buffer. The theorems below therefore take the weaker premise $s_0 \sqsubseteq u-hi\ u$ — the formal content of the handoff condition the parallel-chunks instance names as an assumption.

Same *sink* under (a)+(b); DIFFERENT emitted stream: the window-discard emission interleaves a unit's in-window events at their own positions, the range-merge emission absorbs them into one merged block. Both replay to the canonical *sink-at* at the frontier, which is the point of the equivalence requirement.

7.1 List auxiliary

lemma *concat-map-nil*:

$(\bigwedge x. x \in \text{set } xs \implies g\ x = []) \implies \text{concat } (\text{map } g\ xs) = []$
by (*induction xs*) *auto*

7.2 The merged buffer and the gated emission

context *capture-plan*

begin

The buffer after the backfill upsert: the key's last in-window event decides; a window-quiet key keeps its read result. Present keys of a unit (*rm-present*) are the rows the discipline emits at the close; like the window-discard emission, the close takes a duplicate-free ENUMERATION *rml* of each unit's present set (real captures emit rows physically, so one always exists — *rml-exists* below).

definition *merged* :: $('k, 'v, 'c)\ \text{cunit} \Rightarrow 'k \Rightarrow 'v\ \text{option}$ **where**
 $\text{merged } u\ k =$
 (if *evs-for* $k\ (\text{win } (u\text{-lo } u)\ (u\text{-hi } u)) = []$
 then *u-refresh* $u\ k$
 else *ev-img* (*last* (*evs-for* $k\ (\text{win } (u\text{-lo } u)\ (u\text{-hi } u))))$)

definition *rm-present* :: $('k, 'v, 'c)\ \text{cunit} \Rightarrow 'k\ \text{set}$ **where**
 $\text{rm-present } u = \{k \in u\text{-dom } u. \text{merged } u\ k \neq \text{None}\}$

definition *rml-ok* :: $(('k, 'v, 'c)\ \text{cunit} \Rightarrow 'k\ \text{list}) \Rightarrow \text{bool}$ **where**
 $\text{rml-ok } rml \iff$
 $(\forall u \in \text{set } \text{units}. \text{distinct } (rml\ u) \wedge \text{set } (rml\ u) = \text{rm-present } u)$

definition *rm-close-block* ::
 $(('k, 'v, 'c)\ \text{cunit} \Rightarrow 'k\ \text{list}) \Rightarrow ('k, 'v, 'c)\ \text{cunit}$
 $\Rightarrow (\text{nat} \times ('k, 'v)\ \text{event})\ \text{list}$ **where**
 $\text{rm-close-block } rml\ u =$
 $\text{map } (\lambda k. (\text{den } (u\text{-hi } u), \text{Event } k\ (\text{merged } u\ k)))\ (rml\ u)$

definition *rm-closes-at* ::

((('k, 'v, 'c) cunit $\Rightarrow$ 'k list) $\Rightarrow$ nat $\Rightarrow$ (nat $\times$ ('k, 'v) event) list) **where**
rm-closes-at rml p =
 concat (map (λi . if den (u-hi (units ! i)) = p
 then rm-close-block rml (units ! i) else [])
 [0.. length units])

Clause (a) as a per-position gate: an event whose key is in scope passes to the stream only at-or-after its covering unit's high edge; out-of-scope events pass unconditionally (they never bear on the scoped claim).

definition *rm-gate* :: nat $\Rightarrow$ bool **where**

rm-gate p $\longleftrightarrow$
 (ev-key (L ! p) $\in$ scope $\longrightarrow$ den (u-hi (the-unit (ev-key (L ! p)))) $\leq$ p)

definition *rm-emission-t* ::

((('k, 'v, 'c) cunit $\Rightarrow$ 'k list) $\Rightarrow$ 'c $\Rightarrow$ (nat $\times$ ('k, 'v) event) list) **where**
rm-emission-t rml s0 =
 concat (map (λp . *rm-closes-at* rml p
 @ (if *rm-gate* p then [(Suc p, L ! p)] else []))
 [den s0.. den f])
 @ *rm-closes-at* rml (den f)

definition *rm-emission* ::

((('k, 'v, 'c) cunit $\Rightarrow$ 'k list) $\Rightarrow$ 'c $\Rightarrow$ ('k, 'v) event list) **where**
rm-emission rml s0 = map snd (*rm-emission-t* rml s0)

end

7.3 Clause (b): the merged buffer is the state at the high edge

context *capture-contract*

begin

lemma *rm-present-sub-dom*: *rm-present* u $\subseteq$ u-dom u

by (auto simp: *rm-present-def*)

lemma *rml-exists*:

assumes *fin*: $\bigwedge u. u \in \text{set units} \implies \text{finite } (\text{rm-present } u)$

shows $\exists \text{rml}. \text{rml-ok } \text{rml}$

proof –

define *rml* **where**

rml = (λu . SOME xs. distinct xs $\wedge$ set xs = *rm-present* u)

have distinct (*rml* u) $\wedge$ set (*rml* u) = *rm-present* u

if u: u $\in$ set units **for** u

proof –

have $\exists \text{xs}. \text{distinct } \text{xs} \wedge \text{set } \text{xs} = \text{rm-present } u$

using *finite-distinct-list[OF fin[OF u]]* **by** metis

then show ?thesis

```

    unfolding rml-def by (rule someI-ex)
  qed
  then have rml-ok rml
    by (simp add: rml-ok-def)
  then show ?thesis by blast
qed

theorem merged-at-hi:
  assumes u-in:  $u \in \text{set units}$  and k-dom:  $k \in u\text{-dom } u$ 
  shows merged  $u\ k = \text{state-after } \sigma 0\ (\text{pfx } (u\text{-hi } u))\ k$ 
proof (cases evs-for  $k\ (\text{win } (u\text{-lo } u)\ (u\text{-hi } u)) = []$ )
  case True
  from O2-honest[OF u-in k-dom] obtain  $c$  where
    c-lo:  $u\text{-lo } u \sqsubseteq c$  and c-hi:  $c \sqsubseteq u\text{-hi } u$  and
    honest:  $u\text{-refresh } u\ k = \text{state-after } \sigma 0\ (\text{pfx } c)\ k$ 
  by blast
  have merged  $u\ k = u\text{-refresh } u\ k$ 
    using True by (simp add: merged-def)
  also have  $\dots = \text{state-after } \sigma 0\ (\text{pfx } c)\ k$ 
    by (rule honest)
  also have  $\dots = \text{state-after } \sigma 0\ (\text{pfx } (u\text{-lo } u))\ k$ 
    by (rule win-invariant-state[OF c-lo c-hi True])
  also have  $\dots = \text{state-after } \sigma 0\ (\text{pfx } (u\text{-hi } u))\ k$ 
    by (rule win-invariant-state[OF bracket-lo-hi[OF u-in] cle-refl True,
      symmetric])
  finally show ?thesis .
next
  case False
  have split:  $\text{evs-for } k\ (\text{pfx } (u\text{-hi } u)) =$ 
     $\text{evs-for } k\ (\text{pfx } (u\text{-lo } u))\ @\ \text{evs-for } k\ (\text{win } (u\text{-lo } u)\ (u\text{-hi } u))$ 
    by (rule evs-for-pfx-split[OF bracket-lo-hi[OF u-in]])
  have ne:  $\text{evs-for } k\ (\text{pfx } (u\text{-hi } u)) \neq []$ 
    using split False by auto
  have merged  $u\ k = \text{ev-img } (\text{last } (\text{evs-for } k\ (\text{win } (u\text{-lo } u)\ (u\text{-hi } u))))$ 
    using False by (simp add: merged-def)
  also have  $\dots = \text{ev-img } (\text{last } (\text{evs-for } k\ (\text{pfx } (u\text{-hi } u))))$ 
    using split False by (simp add: last-append)
  also have  $\dots = \text{state-after } \sigma 0\ (\text{pfx } (u\text{-hi } u))\ k$ 
    by (rule state-after-last-event[OF ne, symmetric])
  finally show ?thesis .
qed

```

7.4 Per-key characterization of the gated emission

lemma *titems-rm-close-block*:

```

  assumes rml: rml-ok rml and u-in:  $u \in \text{set units}$ 
  shows titems-for  $k\ (\text{rm-close-block } rml\ u) =$ 
    (if  $k \in \text{set } (rml\ u)$ 
     then  $[(\text{den } (u\text{-hi } u), \text{Event } k\ (\text{merged } u\ k))]$  else  $[]$ )

```

```

proof –
  have dis: distinct (rml u)
    using rml u-in by (simp add: rml-ok-def)
  have titems-for k (rm-close-block rml u) =
    map ( $\lambda k'. (\text{den } (u\text{-hi } u), \text{Event } k' (\text{merged } u \ k'))$ )
      (filter ( $\lambda k'. k' = k$ ) (rml u))
    by (simp add: titems-for-def rm-close-block-def filter-map comp-def)
  then show ?thesis
    using distinct-filter-eq[OF dis] by simp
qed

lemma titems-rm-closes-at:
  assumes rml: rml-ok rml and k:  $k \in \text{scope}$ 
  shows titems-for k (rm-closes-at rml p) =
    (if den (u-hi (the-unit k)) = p  $\wedge k \in \text{set } (rml \ (the\text{-unit } k))$ 
      then [(p, Event k (merged (the-unit k) k))] else [])
proof –
  obtain i0 where i0:  $i0 < \text{length } \text{units } \text{units} \ ! \ i0 = \text{the-unit } k$ 
    using the-unit-props(1)[OF k] by (meson in-set-conv-nth)
  have others: titems-for k (if den (u-hi (units ! i)) = p
    then rm-close-block rml (units ! i) else []) = []
    if i:  $i < \text{length } \text{units}$  and ne:  $i \neq i0$  for i
proof –
  have ui-in: units ! i  $\in \text{set } \text{units}$  using i by simp
  have  $k \notin \text{set } (rml \ (\text{units} \ ! \ i))$ 
proof
  assume  $k \in \text{set } (rml \ (\text{units} \ ! \ i))$ 
  then have  $k \in \text{rm-present } (\text{units} \ ! \ i)$ 
    using rml ui-in by (simp add: rml-ok-def)
  then have  $k \in u\text{-dom } (\text{units} \ ! \ i)$ 
    using rm-present-sub-dom by auto
  then have eq: units ! i = the-unit k
    using covering-unit-unique[OF ui-in - the-unit-props(1)[OF k]
      the-unit-props(2)[OF k]] by blast
  have i = i0
    by (rule the-unit-position-unique[OF k i eq i0(1) i0(2)])
  with ne show False ..
qed
then show ?thesis
  using titems-rm-close-block[OF rml ui-in]
  by (simp add: titems-for-def)
qed
have titems-for k (rm-closes-at rml p) =
  concat (map ( $\lambda i. \text{titems-for } k \ (\text{if } \text{den } (u\text{-hi } (\text{units} \ ! \ i)) = p$ 
    then rm-close-block rml (units ! i)
    else []))
    [0..length units])
  unfolding rm-closes-at-def titems-for-def
  by (simp add: filter-concat-map comp-def if-distrib)

```



```

fix p
show titems-for k (rm-closes-at rml p
  @ (if rm-gate p then [(Suc p, L ! p)] else [])
  = titems-for k (rm-closes-at rml p)
  @ (if rm-gate p ∧ ev-key (L ! p) = k
    then [(Suc p, L ! p)] else [])
  by (cases rm-gate p) (simp-all add: titems-for-def)
qed

have expand: titems-for k (rm-emission-t rml s0) =
  concat (map (λp. titems-for k
    (rm-closes-at rml p
      @ (if rm-gate p then [(Suc p, L ! p)] else []))
    [den s0..den f])
  @ titems-for k (rm-closes-at rml (den f))
unfolding rm-emission-t-def titems-for-def
by (simp add: filter-concat-map comp-def)

have maps: map (λp. titems-for k
  (rm-closes-at rml p
    @ (if rm-gate p then [(Suc p, L ! p)] else []))
  [den s0..den f])
  = map (λp. titems-for k (rm-closes-at rml p)
    @ (if rm-gate p ∧ ev-key (L ! p) = k
      then [(Suc p, L ! p)] else []))
  [den s0..den f])
by (rule map-cong[OF refl]) (rule item)

have dist: titems-for k (rm-emission-t rml s0) =
  concat (map (λp. titems-for k (rm-closes-at rml p)
    @ (if rm-gate p ∧ ev-key (L ! p) = k
      then [(Suc p, L ! p)] else []))
    [den s0..den f])
  @ titems-for k (rm-closes-at rml (den f))
by (simp only: expand maps)

show ?thesis
proof (cases hp < den f)
  case False
  then have hp-f: hp = den f
  using hp-le-f by simp
  have kposs-e: kposs k hp (den f) = []
  using hp-f by (simp add: kposs-def)
  have main-nil: concat (map (λp. titems-for k (rm-closes-at rml p)
    @ (if rm-gate p ∧ ev-key (L ! p) = k
      then [(Suc p, L ! p)] else []))
    [den s0..den f]) = []
  proof (rule concat-map-nil)
  fix p assume p ∈ set [den s0..den f]

```

```

then have p-lt:  $p < \text{den } f$  by simp
have titems-for  $k$  (rm-closes-at rml  $p$ ) = []
  using closes hp-f p-lt by auto
moreover have  $\neg (\text{rm-gate } p \wedge \text{ev-key } (L ! p) = k)$ 
  using gate-k hp-f p-lt by auto
ultimately show titems-for  $k$  (rm-closes-at rml  $p$ )
  @ (if rm-gate  $p \wedge \text{ev-key } (L ! p) = k$ 
    then [(Suc  $p$ ,  $L ! p$ )] else []) = []
  by simp
qed
have tail-eq: titems-for  $k$  (rm-closes-at rml ( $\text{den } f$ ))
  = (if  $k \in \text{set } (\text{rml } u)$  then [( $\text{den } f$ , Event  $k$  (merged  $u$   $k$ ))] else [])
  using closes[of  $\text{den } f$ ] hp-f by simp
show ?thesis
  unfolding dist main-nil tail-eq
  using hp-f kposs-e by (simp add: u-def hp-def)
next
case True
have split: [ $\text{den } s0..<\text{den } f$ ] = [ $\text{den } s0..<hp$ ] @ [ $hp..<\text{den } f$ ]
  using s0-le-hp hp-le-f by (rule upt-split)
have head: [ $hp..<\text{den } f$ ] =  $hp \#$  [Suc  $hp..<\text{den } f$ ]
  using True by (simp add: upt-rec)
have pre: concat (map ( $\lambda p$ . titems-for  $k$  (rm-closes-at rml  $p$ )
  @ (if rm-gate  $p \wedge \text{ev-key } (L ! p) = k$ 
    then [(Suc  $p$ ,  $L ! p$ )] else [])))
  [ $\text{den } s0..<hp$ ]) = []
proof (rule concat-map-nil)
  fix  $p$  assume  $p \in \text{set } [\text{den } s0..<hp]$ 
  then have p-lt:  $p < hp$  by simp
  have titems-for  $k$  (rm-closes-at rml  $p$ ) = []
    using closes p-lt by auto
  moreover have  $\neg (\text{rm-gate } p \wedge \text{ev-key } (L ! p) = k)$ 
    using gate-k p-lt by auto
  ultimately show titems-for  $k$  (rm-closes-at rml  $p$ )
    @ (if rm-gate  $p \wedge \text{ev-key } (L ! p) = k$ 
      then [(Suc  $p$ ,  $L ! p$ )] else []) = []
    by simp
qed
have post: concat (map ( $\lambda p$ . titems-for  $k$  (rm-closes-at rml  $p$ )
  @ (if rm-gate  $p \wedge \text{ev-key } (L ! p) = k$ 
    then [(Suc  $p$ ,  $L ! p$ )] else [])))
  [Suc  $hp..<\text{den } f$ ]
  = map ( $\lambda p$ . (Suc  $p$ ,  $L ! p$ )) (kposs  $k$  (Suc  $hp$ ) ( $\text{den } f$ ))
proof -
  have concat (map ( $\lambda p$ . titems-for  $k$  (rm-closes-at rml  $p$ )
    @ (if rm-gate  $p \wedge \text{ev-key } (L ! p) = k$ 
      then [(Suc  $p$ ,  $L ! p$ )] else [])))
    [Suc  $hp..<\text{den } f$ ]
    = concat (map ( $\lambda p$ . if ev-key  $(L ! p) = k$ 

```

```

      then [(Suc p, L ! p)] else []
    [(Suc hp..

```

7.5 The range-merge equivalence theorem

When a key sees no post-close event, the canonical replay value IS the merged result — the bridge between the discipline's buffer and the fold.

lemma *sink-merged-post-quiet*:

assumes $k: k \in \text{scope}$

and $\text{post}: kposs\ k\ (\text{den}\ (u\text{-hi}\ (\text{the-unit}\ k)))\ (\text{den}\ f) = []$

shows $\text{sink-at}\ f\ k = \text{merged}\ (\text{the-unit}\ k)\ k$

proof –

define u **where** $u = \text{the-unit}\ k$

```

define hp where hp = den (u-hi u)
have u-in: u ∈ set units
  using the-unit-props(1)[OF k] u-def by simp
have lo-hi: u-lo u ⊆ u-hi u using bracket-lo-hi[OF u-in] .
have hi-f: u-hi u ⊆ f using bracket-hi-f[OF u-in] .
have lo-f: u-lo u ⊆ f using lo-hi hi-f by (rule cle-trans)
have lo-le-hp: den (u-lo u) ≤ hp
  using lo-hi by (simp add: cle-def hp-def)
have hp-le-f: hp ≤ den f
  using hi-f by (simp add: cle-def hp-def)
have post': kposs k hp (den f) = []
  using post by (simp add: u-def hp-def)
have win-lo-f: evs-for k (win (u-lo u) f)
  = map (!) L (kposs k (den (u-lo u)) (den f))
  using evs-for-win-kposs[OF lo-f] by simp
have win-lo-hi: evs-for k (win (u-lo u) (u-hi u))
  = map (!) L (kposs k (den (u-lo u)) hp)
  using evs-for-win-kposs[OF lo-hi] by (simp add: hp-def)
have kposs-collapse: kposs k (den (u-lo u)) (den f)
  = kposs k (den (u-lo u)) hp
  using kposs-split[OF lo-le-hp hp-le-f] post' by simp
show ?thesis
proof (cases kposs k (den (u-lo u)) hp = [])
  case True
    have sink-at f k = u-refresh u k
      using True kposs-collapse
      by (simp add: sink-at-def win-lo-f flip: u-def)
    moreover have merged u k = u-refresh u k
      using True win-lo-hi by (simp add: merged-def)
    ultimately show ?thesis by (simp add: u-def)
  next
    case False
      have sink-at f k = ev-img (last (evs-for k (win (u-lo u) f)))
        using False kposs-collapse
        by (simp add: sink-at-def win-lo-f flip: u-def)
      also have ... = ev-img (L ! last (kposs k (den (u-lo u)) hp))
        using win-lo-f kposs-collapse False by (simp add: last-map)
      also have ... = ev-img (last (evs-for k (win (u-lo u) (u-hi u))))
        using win-lo-hi False by (simp add: last-map)
      also have ... = merged u k
        using False win-lo-hi by (simp add: merged-def)
      finally show ?thesis by (simp add: u-def)
qed
qed

```

Buffered-range-merge equivalence: under clauses (a) and (b) the gated emission replays to the canonical *sink* on the scope. Together with *contract-cut* this covers the Flink-shaped capture: upsert the window onto the buffer, emit the merged unit once, stream only above the high edge — and

the replay is exactly the source state at the frontier.

theorem *range-merge-replay*:

assumes *rml*: *rml-ok rml*
and *s0hi*: $\bigwedge u. u \in \text{set units} \implies s0 \sqsubseteq u\text{-hi } u$
and *k*: $k \in \text{scope}$
shows *stream-replay* (*rm-emission rml s0*) *k* = *sink-at f k*

proof –

define *u* **where** *u* = *the-unit k*
define *hp* **where** *hp* = *den (u-hi u)*
have *u-in*: $u \in \text{set units}$ **and** *k-dom*: $k \in u\text{-dom } u$
using *the-unit-props*[*OF k*] *u-def* **by** *auto*
have *lo-hi*: $u\text{-lo } u \sqsubseteq u\text{-hi } u$ **using** *bracket-lo-hi*[*OF u-in*] .
have *hi-f*: $u\text{-hi } u \sqsubseteq f$ **using** *bracket-hi-f*[*OF u-in*] .
have *lo-f*: $u\text{-lo } u \sqsubseteq f$ **using** *lo-hi hi-f* **by** (*rule cle-trans*)
have *lo-le-hp*: $\text{den } (u\text{-lo } u) \leq hp$
using *lo-hi* **by** (*simp add: cle-def hp-def*)
have *hp-le-f*: $hp \leq \text{den } f$
using *hi-f* **by** (*simp add: cle-def hp-def*)

have *present-iff*: $k \in \text{set } (rml \ u) \longleftrightarrow \text{merged } u \ k \neq \text{None}$
using *rml u-in k-dom* **by** (*auto simp: rml-ok-def rm-present-def*)

have *master*: *evs-for k (rm-emission rml s0)* =
 $(\text{if } k \in \text{set } (rml \ u) \text{ then } [\text{Event } k \ (\text{merged } u \ k)] \text{ else } [])$
 $@ \text{map } (!) \ L \ (kposs \ k \ hp \ (\text{den } f))$
unfolding *rm-emission-def evs-for-map-snd*
using *titems-rm-emission-master*[*OF rml s0hi k*]
by (*simp add: u-def hp-def comp-def if-distrib map-append*
cong: if-cong)

show *?thesis*

proof (*cases kposs k hp (den f)* = [])
case *post-ne*: *False*
have *win-lo-f*: *evs-for k (win (u-lo u) f)*
 $= \text{map } (!) \ L \ (kposs \ k \ (\text{den } (u\text{-lo } u)) \ (\text{den } f))$
using *evs-for-win-kposs*[*OF lo-f*] **by** *simp*
have *kposs-lo-f-split*:
 $kposs \ k \ (\text{den } (u\text{-lo } u)) \ (\text{den } f)$
 $= kposs \ k \ (\text{den } (u\text{-lo } u)) \ hp \ @ \ kposs \ k \ hp \ (\text{den } f)$
using *lo-le-hp hp-le-f* **by** (*rule kposs-split*)
have *lo-f-ne*: $kposs \ k \ (\text{den } (u\text{-lo } u)) \ (\text{den } f) \neq []$
using *kposs-lo-f-split post-ne* **by** *auto*
have *last-lo-f*: *last (kposs k (den (u-lo u)) (den f))*
 $= \text{last } (kposs \ k \ hp \ (\text{den } f))$
using *kposs-lo-f-split post-ne* **by** (*simp add: last-append*)
have *sink*: *sink-at f k* = *ev-img (L ! last (kposs k hp (den f)))*
using *lo-f-ne last-lo-f*
by (*simp add: sink-at-def win-lo-f last-map flip: u-def*)
have *stream-ne*: *evs-for k (rm-emission rml s0)* $\neq []$

```

    using master post-ne by auto
  have last-stream: last (evs-for k (rm-emission rml s0))
    = (!) L (last (kposs k hp (den f)))
    using master post-ne by (simp add: last-append last-map)
  have stream-replay (rm-emission rml s0) k
    = ev-img (L ! last (kposs k hp (den f)))
    unfolding stream-replay-def
    using state-after-last-event[OF stream-ne] last-stream by simp
  then show ?thesis using sink by simp
next
case post-e: True
have sink: sink-at f k = merged u k
  using sink-merged-post-quiet[OF k] post-e
  by (simp add: u-def hp-def)
show ?thesis
proof (cases k ∈ set (rml u))
case True
  have evs-for k (rm-emission rml s0) = [Event k (merged u k)]
    using master True post-e by simp
  then have stream-replay (rm-emission rml s0) k = merged u k
    unfolding stream-replay-def by (simp add: state-after-key)
  then show ?thesis using sink by simp
next
case False
  have evs-for k (rm-emission rml s0) = []
    using master False post-e by simp
  then have stream-replay (rm-emission rml s0) k = None
    unfolding stream-replay-def by (simp add: state-after-no-events)
  moreover have merged u k = None
    using present-iff False by simp
  ultimately show ?thesis using sink by simp
qed
qed
qed

```

7.6 Range-merge per-key monotonicity

The gated emission is per-key monotone by construction: a key emits its merged entry at the close, then its post-close events in commit order — never an older state over a newer one. This is the “all as INSERT records” emission shape with the during-replay monotonicity proved separately for each discipline.

theorem *range-merge-monotone*:

```

  assumes rml: rml-ok rml
    and s0hi:  $\bigwedge u. u \in \text{set units} \implies s0 \sqsubseteq u\text{-hi } u$ 
    and k:  $k \in \text{scope}$ 
  shows sorted (map fst (titems-for k (rm-emission-t rml s0)))
proof —

```

```

define hp where hp = den (u-hi (the-unit k))
have m: map fst (titems-for k (rm-emission-t rml s0)) =
  (if k ∈ set (rml (the-unit k)) then [hp] else [])
  @ map Suc (kposs k hp (den f))
using titems-rm-emission-master[OF rml s0hi k]
by (cases k ∈ set (rml (the-unit k))) (simp-all add: hp-def comp-def)
have s-post: sorted (map Suc (kposs k hp (den f)))
by (simp add: sorted-map-Suc kposs-sorted)
have post-ge:  $\bigwedge b. b \in \text{set } (\text{map } \text{Suc } (\text{kposs } k \text{ hp } (\text{den } f))) \implies hp \leq b$ 
using kposs-bounds by (auto intro: le-SucI)
show ?thesis
proof (cases k ∈ set (rml (the-unit k)))
  case True
    have sorted (hp # map Suc (kposs k hp (den f)))
      using s-post post-ge by auto
    then show ?thesis using m True by simp
  next
    case False
      then show ?thesis using m s-post by simp
qed
qed
end

```

7.7 Scalar offset coordinates (shared by the offset instances)

The parallel-chunks and dump-splice instances both run over scalar log offsets (binlog positions, LSN-class coordinates). The shared denotation: an offset denotes the prefix of that length, capped at the committed history — on every offset a plan actually uses (all at-or-below the frontier, hence in range) the cap is the identity, and comparisons agree with the denotational $\sqsubseteq$ (*offden-le-iff*); the operational-fidelity requirement is discharged per instance from exactly these two facts.

definition *offden* :: (*'k*, *'v*) *event list* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
offden *L* *c* = *min* *c* (*length* *L*)

lemma *offden-le-len*: *offden* *L* *c* $\leq$ *length* *L*
by (*simp* *add*: *offden-def*)

lemma *offden-id*: *c* $\leq$ *length* *L* $\implies$ *offden* *L* *c* = *c*
by (*simp* *add*: *offden-def*)

lemma *offden-mono*: *a* $\leq$ *b* $\implies$ *offden* *L* *a* $\leq$ *offden* *L* *b*
by (*simp* *add*: *offden-def*)

lemma *offden-le-iff*:
a $\leq$ *length* *L* $\implies$ *b* $\leq$ *length* *L* $\implies$ (*offden* *L* *a* $\leq$ *offden* *L* *b*) = (*a* $\leq$ *b*)
by (*simp* *add*: *offden-def*)

7.8 Necessity of the withholding clause (a) — fixture

The withholding counterexample: a pre-*lo* event for a later-deleted key. The log inserts key 1 and then deletes it inside the unit's window; the refresh (read before the delete) is valid in-bracket; the merged buffer is empty at the high edge (clause (b): the last in-window event is the delete), so the lawful gated emission is EMPTY and replays to the correct absence result. A variant WITHOUT clause (a) passes the pre-*lo* insert to the stream while the in-window delete is still absorbed into the buffer — its stream is $[Event\ 1\ (Some\ 5)]$, replaying key 1 to a value the source no longer has. The withholding clause is what stands between the discipline and that failure.

definition $L-a :: (nat, nat) \text{ event list where}$
 $L-a = [Event\ 1\ (Some\ 5), Event\ 1\ None]$

definition $den-a :: nat \Rightarrow nat \text{ where}$
 $den-a\ c = \min\ c\ 2$

definition $unit-a :: (nat, nat, nat) \text{ cunit where}$
 $unit-a = (\!|u\text{-dom} = \{1\}, u\text{-lo} = 1, u\text{-hi} = 2,$
 $u\text{-refresh} = (\lambda k. \text{if } k = 1 \text{ then } Some\ 5 \text{ else } None)\!|)$

interpretation acs : coordinate-space $L-a\ den-a$
 by $unfold\text{-}locales\ (simp\ add: den\text{-}a\text{-}def\ L\text{-}a\text{-}def)$

interpretation rma : capture-contract $L-a\ den-a\ \lambda\cdot. None\ [unit\text{-}a]\ \{1\}\ 2$

proof ($unfold\text{-}locales$)

show $(\bigcup u \in set\ [unit\text{-}a].\ u\text{-dom}\ u) = \{1\}$
 by ($simp\ add: unit\text{-}a\text{-}def$)

next

fix $i\ j$

assume $i < length\ [unit\text{-}a]\ j < length\ [unit\text{-}a]\ i \neq j$

then show $u\text{-dom}\ ([unit\text{-}a]\ !\ i) \cap u\text{-dom}\ ([unit\text{-}a]\ !\ j) = \{\}$ by $simp$

next

fix u assume $u \in set\ [unit\text{-}a]$

then show $acs.cle\ (u\text{-lo}\ u)\ (u\text{-hi}\ u)$

by ($simp\ add: unit\text{-}a\text{-}def\ acs.cle\text{-}def\ den\text{-}a\text{-}def$)

next

fix u assume $u \in set\ [unit\text{-}a]$

then show $acs.cle\ (u\text{-hi}\ u)\ 2$

by ($simp\ add: unit\text{-}a\text{-}def\ acs.cle\text{-}def\ den\text{-}a\text{-}def$)

next

fix $u\ k$ assume $u \in set\ [unit\text{-}a]$ and $k \in u\text{-dom}\ u$

then show $\exists c. acs.cle\ (u\text{-lo}\ u)\ c \wedge acs.cle\ c\ (u\text{-hi}\ u) \wedge$

$u\text{-refresh}\ u\ k = state\text{-}after\ (\lambda\cdot. None)\ (acs.pfx\ c)\ k$

unfolding $acs.cle\text{-}def\ acs.pfx\text{-}def$

by ($intro\ exI[of\ -\ 1]$)

($simp\ add: unit\text{-}a\text{-}def\ den\text{-}a\text{-}def\ L\text{-}a\text{-}def\ state\text{-}after\text{-}def\ apply\text{-}ev\text{-}def$)

qed

```

lemma wit-rma-the-unit [simp]: rma.the-unit 1 = unit-a
  using rma.the-unit-props[of 1] by simp

lemma wit-rma-merged-absent: rma.merged unit-a 1 = None
proof -
  have evs-for 1 (acs.win (u-lo unit-a) (u-hi unit-a)) = [Event 1 None]
    unfolding acs.win-def
    by (simp add: unit-a-def den-a-def L-a-def evs-for-def)
  then show ?thesis
    by (simp add: rma.merged-def)
qed

lemma wit-rma-present-empty: rma.rm-present unit-a = {}
proof -
  have  $\bigwedge x. x \in u\text{-dom } unit\text{-a} \implies rma.merged \text{ unit-a } x = None$ 
    using wit-rma-merged-absent by (simp add: unit-a-def)
  then show ?thesis
    by (auto simp: rma.rm-present-def)
qed

lemma wit-rma-rml-ok: rma.rml-ok ( $\lambda\cdot$ . [])
  by (simp add: rma.rml-ok-def wit-rma-present-empty)

lemma wit-rma-lawful-replay:
  stream-replay (rma.rm-emission ( $\lambda\cdot$ . []) 0) 1 = rma.sink-at 2 1
proof (rule rma.range-merge-replay[OF wit-rma-rml-ok])
  show  $\bigwedge u. u \in set [unit-a] \implies acs.cle \ 0 \ (u\text{-hi } u)$ 
    by (simp add: unit-a-def acs.cle-def den-a-def)
qed simp

lemma wit-rma-lawful-absent: rma.sink-at 2 1 = None
proof -
  have win-e: evs-for 1 (acs.win (u-lo unit-a) 2) = [Event 1 None]
    unfolding acs.win-def
    by (simp add: unit-a-def den-a-def L-a-def evs-for-def)
  show ?thesis
    unfolding rma.sink-at-def wit-rma-the-unit win-e by simp
qed

lemma fix-no-withholding-wrong:
  stream-replay [Event 1 (Some 5)] (1 :: nat) = Some (5 :: nat)
  state-after ( $\lambda\cdot$ . None) (take 2 L-a) 1 = None
  by (simp-all add: stream-replay-def L-a-def state-after-def apply-ev-def)

end

```

theory Parallel-Chunks-Instance

```

imports Range-Merge
begin

```

8 A parallel-chunks instance: Flink-style incremental snapshots

This theory models the parallel chunk algorithm described for Flink CDC 3.6.0’s MySQL connector. The algorithm records the current binlog position as LOW before the chunk select and as HIGH after it (write-free: both are READS of a server-side offset, no watermark write exists anywhere in the algorithm), backfills “the binlog records that belong to the snapshot chunk from LOW offset to HIGH offset”, upserts them onto the buffered chunk records, and emits the merged buffer once, all as INSERT records; chunks run in parallel across readers. The documented lineage line sits directly under the algorithm: “The algorithm is inspired by [DBLog Paper]”.

The formalization:

- Scalar offset coordinates via *offden* (*Range-Merge*); the declared endpoint-to-prefix normalization for this instance: the protocol description records the consumed span as offset-inclusive “[LOW, HIGH]” shorthand, raw-offset edge inclusivity undocumented, so the instance DECLARES the normalization $[LOW, HIGH] \mapsto \llbracket HIGH \rrbracket \setminus \llbracket LOW \rrbracket$ — realized below by the brackets denoting offset prefixes and the window machinery; *offset-window* exhibits the declared window as exactly the $[LOW, HIGH)$ position segment of the log, and *offset-fidelity* discharges the operational-fidelity clause for offset comparisons.
- *A-read* is O2 verbatim at instance level — per-key in-bracket validity enters as a NAMED assumption rather than being derived from an isolation level, mirroring the read-only instance’s assumption of the same name.
- Two further instantiation requirements enter as NAMED locale assumptions, never silently. They live at different proof boundaries: (i) *A-handoff* — the streaming-phase start offset relative to the chunks’ HIGH watermarks plus the cross-phase dedup rule. The development DERIVES the sufficient condition the equivalence needs: consumption starts at-or-before every chunk’s HIGH edge, with the §6 clause-(a) withholding as the cross-phase dedup rule — exactly the weaker-than-window-discard observability premise of *range-merge-replay*. This premise belongs only to the actual-emission locale below, not to the plan-level contract or its cut theorem. (ii) *A-identity* — final logical ownership (the paper’s A-identity).

The docs state exactly-once holds only if the chunk key column is never updated; a moving chunk key relocates a key between units mid-plan, so the FINAL plan would not partition the scope unless operational normalization restores stable logical ownership. Formally the assumption below states only that final unit domains are disjoint. It does not itself assert physical immutability of a particular split column.

- The parallel structure needs NO new coordination argument — every locale assumption below is per-chunk except disjointness itself, the contract instantiation is proved unit-by-unit, and composition with ANY other conforming plan over a disjoint scope is Corollary 3’s mixing (*parallel-mix-any*). Nothing couples two chunks anywhere in the development.

backfill.skip: with backfill skipped, changes during the snapshot phase “will be consumed later in change log reading phase instead of being merged into the snapshot”, and the docs price it: “Skipping backfill might lead to data inconsistency because some change log events happened within the snapshot phase might be replayed (only at-least-once semantic is promised).” Contract-level placement: T1 at most (§5 — the emission as shipped discharges no merge-discipline equivalence), and only under a separately named external convergence mechanism; its exact placement is not established here and is deliberately NOT forced here.

Guarantee level: plan-level T2 — reached below (*parallel-cut*) without a handoff premise. The actual modeled emitted stream reaches the same final source state under the additional handoff premise (*parallel-emitted-cut*) and is per-key monotone (*parallel-monotone*).

8.1 The parallel-chunks plan

```

locale parallel-chunks-plan =
  fixes  $L :: ('k, 'v)$  event list
    and  $\sigma 0 :: ('k, 'v)$  state
    and  $n :: \text{nat}$ 
    and  $c\text{dom} :: \text{nat} \Rightarrow 'k$  set
    and  $c\text{low } c\text{high} :: \text{nat} \Rightarrow \text{nat}$ 
    and  $c\text{read} :: \text{nat} \Rightarrow ('k, 'v)$  state
    and  $\text{scope} :: 'k$  set
    and  $n\text{f} :: \text{nat}$ 
  assumes bracket-ord:  $\bigwedge i. i < n \implies c\text{low } i \leq c\text{high } i$ 
    and bracket-f:  $\bigwedge i. i < n \implies c\text{high } i \leq n\text{f}$ 
    and f-len:  $n\text{f} \leq \text{length } L$ 
    and POI-cover:  $(\bigcup i \in \{0..<n\}. c\text{dom } i) = \text{scope}$ 
    and A-identity:  $\bigwedge i\ j. i < n \implies j < n \implies i \neq j \implies$ 
       $c\text{dom } i \cap c\text{dom } j = \{\}$ 

```

and $A\text{-read}$: $\bigwedge i k. i < n \implies k \in \text{cdom } i \implies$
 $\exists c. \text{clow } i \leq c \wedge c \leq \text{chigh } i \wedge$
 $\text{cread } i k = \text{state-after } \sigma 0 \text{ (take } c L) k$
begin

definition $\text{punit} :: \text{nat} \Rightarrow ('k, 'v, \text{nat}) \text{ cunit}$ **where**
 $\text{punit } i = (\text{u-dom} = \text{cdom } i, \text{u-lo} = \text{clow } i, \text{u-hi} = \text{chigh } i,$
 $\text{u-refresh} = \text{cread } i)$

definition $\text{punits} :: ('k, 'v, \text{nat}) \text{ cunit list}$ **where**
 $\text{punits} = \text{map } \text{punit } [0..<n]$

lemma length-punits $[\text{simp}]$: $\text{length } \text{punits} = n$
by $(\text{simp add: } \text{punits-def})$

lemma punits-nth $[\text{simp}]$: $i < n \implies \text{punits} ! i = \text{punit } i$
by $(\text{simp add: } \text{punits-def})$

lemma punits-mem :
assumes $u \in \text{set } \text{punits}$
obtains i **where** $i < n$ **and** $u = \text{punit } i$
using assms **by** $(\text{auto simp: } \text{punits-def})$

end

sublocale $\text{parallel-chunks-plan} \subseteq \text{ocs}$: $\text{coordinate-space } L \text{ offden } L$
by $\text{unfold-locales (rule offden-le-len)}$

sublocale $\text{parallel-chunks-plan} \subseteq \text{pc}$: $\text{capture-contract } L \text{ offden } L \sigma 0 \text{ punits scope}$
 nf

proof (unfold-locales)
have $(\bigcup u \in \text{set } \text{punits}. \text{u-dom } u) = (\bigcup i \in \{0..<n\}. \text{cdom } i)$
by $(\text{auto simp: } \text{punits-def } \text{punit-def})$
then show $(\bigcup u \in \text{set } \text{punits}. \text{u-dom } u) = \text{scope}$
by $(\text{simp add: } \text{PO1-cover})$

next
fix $i j$
assume $i < \text{length } \text{punits } j < \text{length } \text{punits } i \neq j$
then show $\text{u-dom } (\text{punits} ! i) \cap \text{u-dom } (\text{punits} ! j) = \{\}$
using $A\text{-identity}$ **by** $(\text{simp add: } \text{punit-def})$

next
fix u **assume** $u \in \text{set } \text{punits}$
then obtain i **where** $i < n$ $u = \text{punit } i$
by $(\text{rule } \text{punits-mem})$
show $\text{ocs.cle } (\text{u-lo } u) (\text{u-hi } u)$
using $\text{bracket-ord}[OF i(1)]$
by $(\text{simp add: } i(2) \text{ punit-def } \text{ocs.cle-def } \text{offden-mono})$

next
fix u **assume** $u \in \text{set } \text{punits}$

```

then obtain  $i$  where  $i: i < n \ u = \text{punit } i$ 
  by (rule punits-mem)
show  $\text{ocs.cle } (u\text{-hi } u) \text{ nf}$ 
  using  $\text{bracket-f}[OF \ i(1)]$ 
  by (simp add: i(2) punit-def ocs.cle-def offden-mono)
next
  fix  $u \ k$  assume  $u \in \text{set punits}$  and  $k: k \in u\text{-dom } u$ 
  then obtain  $i$  where  $i: i < n \ u = \text{punit } i$ 
    by (auto elim: punits-mem)
  have  $k\text{-dom}: k \in \text{cdom } i$ 
    using  $k$  by (simp add: i(2) punit-def)
  from  $A\text{-read}[OF \ i(1) \ k\text{-dom}]$  obtain  $c$  where
     $c: \text{clow } i \leq c \ c \leq \text{chigh } i \ \text{cread } i \ k = \text{state-after } \sigma 0 \ (\text{take } c \ L) \ k$ 
    by blast
  have  $c\text{-len}: c \leq \text{length } L$ 
    using  $c(2) \ \text{bracket-f}[OF \ i(1)] \ f\text{-len}$  by simp
  show  $\exists c. \text{ocs.cle } (u\text{-lo } u) \ c \wedge \text{ocs.cle } c \ (u\text{-hi } u) \wedge$ 
     $u\text{-refresh } u \ k = \text{state-after } \sigma 0 \ (\text{ocs.pfx } c) \ k$ 
  proof (intro exI[of - c] conjI)
    show  $\text{ocs.cle } (u\text{-lo } u) \ c$ 
      using  $c(1)$  by (simp add: i(2) punit-def ocs.cle-def offden-mono)
    show  $\text{ocs.cle } c \ (u\text{-hi } u)$ 
      using  $c(2)$  by (simp add: i(2) punit-def ocs.cle-def offden-mono)
    show  $u\text{-refresh } u \ k = \text{state-after } \sigma 0 \ (\text{ocs.pfx } c) \ k$ 
      using  $c(3)$ 
      by (simp add: i(2) punit-def ocs.pfx-def offden-id[OF c-len])
  qed
qed

```

The emission locale adds only the streaming handoff needed to identify the modeled gated range-merge stream with the plan's canonical replay. All plan construction, contract membership, the cut, and heterogeneous mixing remain in *parallel-chunks-plan* and therefore do not depend syntactically on this premise.

```

locale parallel-chunks =
  parallel-chunks-plan  $L \ \sigma 0 \ n \ \text{cdom} \ \text{clow} \ \text{chigh} \ \text{cread} \ \text{scope} \ \text{nf}$ 
  for  $L :: ('k, 'v) \text{ event list}$ 
    and  $\sigma 0 :: ('k, 'v) \text{ state}$ 
    and  $n :: \text{nat}$ 
    and  $\text{cdom} :: \text{nat} \Rightarrow 'k \text{ set}$ 
    and  $\text{clow } \text{chigh} :: \text{nat} \Rightarrow \text{nat}$ 
    and  $\text{cread} :: \text{nat} \Rightarrow ('k, 'v) \text{ state}$ 
    and  $\text{scope} :: 'k \text{ set}$ 
    and  $\text{nf} :: \text{nat} +$ 
  fixes  $s0\text{off} :: \text{nat}$ 
  assumes  $A\text{-handoff}: \bigwedge i. i < n \implies s0\text{off} \leq \text{chigh } i$ 

```

8.2 Fidelity, cut, equivalence, and mixing at the instance

context *parallel-chunks-plan*
begin

The operational-fidelity discharge for offset coordinates: on the offsets the plan uses (all bounded by the committed history) the instance's own comparisons agree with the denotational $\sqsubseteq$, and the declared window of a bracket is exactly the $[LOW, HIGH)$ position segment of the log — the membership decision “the event's position lies in the consumed span” is the denotational window membership.

lemma *offset-fidelity*:
assumes $a \leq \text{length } L$ **and** $b \leq \text{length } L$
shows $\text{ocs.cle } a \ b \longleftrightarrow a \leq b$
using *assms* **by** (*simp add: ocs.cle-def offden-le-iff*)

lemma *offset-window*:
assumes $lo \leq hi$ **and** $hi \leq \text{length } L$
shows $\text{ocs.win } lo \ hi = \text{drop } lo \ (\text{take } hi \ L)$
proof –
have $lo\text{-len}: lo \leq \text{length } L$
using *assms* **by** *linarith*
show *?thesis*
by (*simp add: ocs.win-def offden-id[OF lo-len] offden-id[OF assms(2)]*)
qed

The plan guarantee: the parallel-chunks plan is a contract instance, so THE theorem applies at the frontier — tier T2. Note what the proof did NOT need: any premise coupling two chunks (beyond disjointness), any bound on how many chunks run concurrently, any relation between distinct chunks' brackets. Wall-clock overlap of brackets is invisible to the plan.

theorem *parallel-cut*:
assumes $k \in \text{scope}$
shows $\text{pc.sink-at } nf \ k = \text{state-after } \sigma 0 \ (\text{take } nf \ L) \ k$
proof –
have $\text{pc.sink-at } nf \ k = \text{state-after } \sigma 0 \ (\text{ocs.pfx } nf) \ k$
by (*rule pc.contract-cut[OF assms]*)
then show *?thesis*
by (*simp add: ocs.pfx-def offden-id[OF f-len]*)
qed

The mixing form is plan-level as well. The final logical ownership partition composes with any conforming plan over a disjoint scope at the shared frontier, without a streaming-handoff premise.

theorem *parallel-mix-any*:
assumes *capture-contract* $L \ (\text{offden } L) \ \sigma 0 \ us2 \ sc2 \ nf$
and $\text{scope} \cap sc2 = \{\}$
shows *capture-contract* $L \ (\text{offden } L) \ \sigma 0 \ (\text{punits } @ \ us2) \ (\text{scope} \cup sc2) \ nf$

by (rule ocs.mixing-union[OF pc.capture-contract-axioms assms])

end

context parallel-chunks

begin

The discipline half: the emitted stream the connector actually ships — merged chunks once at their HIGH edges, all as INSERT records, stream events above the HIGH edges only — replays to the canonical sink. The consumption-start premise is discharged by *A-handoff*: this is precisely where the named handoff assumption is load-bearing.

theorem parallel-replay:

assumes *rml*: *pc.rml-ok rml* and *k*: $k \in \text{scope}$

shows *stream-replay* (*pc.rm-emission rml s0off*) $k = \text{pc.sink-at nf } k$

proof (rule *pc.range-merge-replay*[OF *rml - k*])

fix *u* assume $u \in \text{set punits}$

then obtain *i* where $i: i < n \ u = \text{punit } i$

by (rule *punits-mem*)

show *ocs.cle s0off* (*u-hi u*)

using *A-handoff*[OF *i(1)*]

by (*simp add: i(2) punit-def ocs.cle-def offden-mono*)

qed

This is the direct final-output result. The exact range-merge enumeration, the modeled withholding gate, and the handoff premise first establish replay equivalence. The plan-level cut then yields equality with the source row-event replay at the configured frontier. The conclusion is final replay from the empty state, not physical prefix equivalence or an exactly-once delivery theorem.

theorem parallel-emitted-cut:

assumes *rml*: *pc.rml-ok rml* and *k*: $k \in \text{scope}$

shows *stream-replay* (*pc.rm-emission rml s0off*) $k = \text{state-after } \sigma 0 \ (\text{take nf } L) \ k$

using *parallel-replay*[OF *rml k*] *parallel-cut*[OF *k*] **by** *simp*

theorem parallel-monotone:

assumes *rml*: *pc.rml-ok rml* and *k*: $k \in \text{scope}$

shows *sorted* (*map fst* (*pc.titems-for k* (*pc.rm-emission-t rml s0off*))))

proof (rule *pc.range-merge-monotone*[OF *rml - k*])

fix *u* assume $u \in \text{set punits}$

then obtain *i* where $i: i < n \ u = \text{punit } i$

by (rule *punits-mem*)

show *ocs.cle s0off* (*u-hi u*)

using *A-handoff*[OF *i(1)*]

by (*simp add: i(2) punit-def ocs.cle-def offden-mono*)

qed

end

8.3 Constructed witness (non-vacuity)

Two chunks over a four-event log, brackets genuinely overlapping in coordinates (chunk 1's whole bracket lies inside chunk 0's — parallel wall-clock execution made visible), both windows non-trivial: chunk 0's window contains a delete-then-reinsert of key 1 (clause (b)'s present-again case: the LAST in-window event decides, so the merged unit re-emits the key at its reinserted value), chunk 1's refresh reads key 2 as absent BEFORE its in-window insert (the upsert overrides the absent read result). The emitted stream is two INSERT records — key 1's three log events produce zero stream emissions below its chunk's HIGH edge — and replays to the source state at the frontier on both keys.

definition $L\text{-}p :: (\text{nat}, \text{nat}) \text{ event list}$ **where**
 $L\text{-}p = [\text{Event } 1 \text{ (Some } 10), \text{Event } 2 \text{ (Some } 20),$
 $\text{Event } 1 \text{ None}, \text{Event } 1 \text{ (Some } 11)]$

definition $c\text{dom-}p :: \text{nat} \Rightarrow \text{nat set}$ **where**
 $c\text{dom-}p \ i = (\text{if } i = 0 \text{ then } \{1\} \text{ else } \{2\})$

definition $c\text{low-}p :: \text{nat} \Rightarrow \text{nat}$ **where**
 $c\text{low-}p \ i = (\text{if } i = 0 \text{ then } 0 \text{ else } 1)$

definition $c\text{high-}p :: \text{nat} \Rightarrow \text{nat}$ **where**
 $c\text{high-}p \ i = (\text{if } i = 0 \text{ then } 4 \text{ else } 2)$

definition $c\text{read-}p :: \text{nat} \Rightarrow (\text{nat}, \text{nat}) \text{ state}$ **where**
 $c\text{read-}p \ i = (\lambda k. \text{if } i = 0 \wedge k = 1 \text{ then Some } 10 \text{ else None})$

interpretation $p\text{w}$: parallel-chunks $L\text{-}p \ \lambda\text{-}$. None 2 $c\text{dom-}p \ c\text{low-}p \ c\text{high-}p$
 $c\text{read-}p \ \{1, 2\} \ 4 \ 0$

proof (*unfold-locales*)
fix $i :: \text{nat}$ **assume** $i < 2$
then show $c\text{low-}p \ i \leq c\text{high-}p \ i$
by (*auto simp: c\text{low-}p-def c\text{high-}p-def*)
next
fix $i :: \text{nat}$ **assume** $i < 2$
then show $c\text{high-}p \ i \leq 4$
by (*auto simp: c\text{high-}p-def*)
next
show $(4 :: \text{nat}) \leq \text{length } L\text{-}p$
by (*simp add: L\text{-}p-def*)
next
show $(\bigcup i \in \{0..<2\}. c\text{dom-}p \ i) = \{1, 2\}$
proof (*intro subset-antisym subsetI*)
fix x **assume** $x \in (\bigcup i \in \{0..<2\}. c\text{dom-}p \ i)$
then obtain i **where** $i \in \{0..<(2::\text{nat})\}$ **and** $x \in c\text{dom-}p \ i$
by *blast*
then show $x \in \{1, 2\}$

```

    by (auto simp: cdom-p-def split: if-splits)
next
  fix x assume x ∈ {1, (2::nat)}
  then consider (K1) x = 1 | (K2) x = 2
    by auto
  then show x ∈ (⋃ i∈{0..<2}. cdom-p i)
  proof cases
    case K1
    have (0::nat) ∈ {0..<2} by simp
    moreover have x ∈ cdom-p 0
      using K1 by (simp add: cdom-p-def)
    ultimately show ?thesis by blast
  next
    case K2
    have (1::nat) ∈ {0..<2} by simp
    moreover have x ∈ cdom-p 1
      using K2 by (simp add: cdom-p-def)
    ultimately show ?thesis by blast
  qed
qed
next
  fix i j :: nat
  assume i < 2 j < 2 i ≠ j
  then show cdom-p i ∩ cdom-p j = {}
    by (auto simp: cdom-p-def)
next
  fix i :: nat and k :: nat
  assume i: i < 2 and k: k ∈ cdom-p i
  show ∃ c. clow-p i ≤ c ∧ c ≤ chigh-p i ∧
    cread-p i k = state-after (λ-. None) (take c L-p) k
  proof (cases i = 0)
    case True
    then have k = 1 using k by (simp add: cdom-p-def)
    then show ?thesis
      by (intro exI[of - 1])
        (simp add: True clow-p-def chigh-p-def cread-p-def L-p-def
          state-after-def apply-ev-def)
  next
    case False
    then have k = 2 using k by (simp add: cdom-p-def)
    then show ?thesis
      using False
      by (intro exI[of - 1])
        (simp add: clow-p-def chigh-p-def cread-p-def L-p-def
          state-after-def apply-ev-def)
  qed
qed
next
  fix i :: nat assume i < 2
  then show (0 :: nat) ≤ chigh-p i

```

by *simp*
qed

lemma *wit-parallel-cut*:

pw.pc.sink-at 4 1 = *state-after* ($\lambda\cdot$. *None*) (*take* 4 *L-p*) 1
pw.pc.sink-at 4 2 = *state-after* ($\lambda\cdot$. *None*) (*take* 4 *L-p*) 2
 by (*rule pw.parallel-cut, simp*)+

lemma *wit-parallel-punit-ne*: *pw.punit* 0 $\neq$ *pw.punit* 1

proof

assume *pw.punit* 0 = *pw.punit* 1
 then have *u-lo* (*pw.punit* 0) = *u-lo* (*pw.punit* 1)
 by *simp*
 then show *False*
 by (*simp add: pw.punit-def clow-p-def*)

qed

lemma *wit-parallel-the-unit-1* [*simp*]: *pw.pc.the-unit* 1 = *pw.punit* 0

proof –

have *props*: *pw.pc.the-unit* 1 $\in$ *set pw.punits*
 (1 :: *nat*) $\in$ *u-dom* (*pw.pc.the-unit* 1)
 by (*rule pw.pc.the-unit-props, simp*) +
 from *props*(1) **obtain** *i* **where** *i* < 2 *pw.pc.the-unit* 1 = *pw.punit* *i*
 by (*rule pw.punits-mem*)
 have *i* $\neq$ 1
proof
 assume *i* = 1
 then have (1 :: *nat*) $\in$ *u-dom* (*pw.punit* 1)
 using *props*(2) *i*(2) **by** *simp*
 then show *False*
 by (*simp add: pw.punit-def cdom-p-def*)

qed

with *i*(1) **have** *i* = 0 **by** *linarith*

with *i*(2) **show** *?thesis* **by** *simp*

qed

lemma *wit-parallel-the-unit-2* [*simp*]: *pw.pc.the-unit* 2 = *pw.punit* 1

proof –

have *props*: *pw.pc.the-unit* 2 $\in$ *set pw.punits*
 (2 :: *nat*) $\in$ *u-dom* (*pw.pc.the-unit* 2)
 by (*rule pw.pc.the-unit-props, simp*) +
 from *props*(1) **obtain** *i* **where** *i* < 2 *pw.pc.the-unit* 2 = *pw.punit* *i*
 by (*rule pw.punits-mem*)
 have *i* $\neq$ 0
proof
 assume *i* = 0
 then have (2 :: *nat*) $\in$ *u-dom* (*pw.punit* 0)
 using *props*(2) *i*(2) **by** *simp*
 then show *False*

```

    by (simp add: pw.punit-def cdom-p-def)
  qed
  with i(1) have i = 1 by linarith
  with i(2) show ?thesis by simp
qed

lemma wit-parallel-merged-1: pw.pc.merged (pw.punit 0) 1 = Some 11
proof -
  have pw.ocs.win (u-lo (pw.punit 0)) (u-hi (pw.punit 0)) = L-p
    unfolding pw.ocs.win-def
  by (simp add: pw.punit-def clow-p-def chigh-p-def offden-def L-p-def)
  moreover have evs-for 1 L-p =
    [Event 1 (Some 10), Event 1 None, Event 1 (Some 11)]
  by (simp add: evs-for-def L-p-def)
  ultimately show ?thesis
    by (simp add: pw.pc.merged-def)
qed

lemma wit-parallel-merged-2: pw.pc.merged (pw.punit 1) 2 = Some 20
proof -
  have pw.ocs.win (u-lo (pw.punit 1)) (u-hi (pw.punit 1)) =
    [Event 2 (Some 20)]
    unfolding pw.ocs.win-def
  by (simp add: pw.punit-def clow-p-def chigh-p-def offden-def L-p-def)
  then show ?thesis
    by (simp add: pw.pc.merged-def evs-for-def)
qed

definition rml-p :: (nat, nat, nat) cunit  $\Rightarrow$  nat list where
  rml-p u = (if u = pw.punit 0 then [1] else [2])

lemma wit-parallel-rml-ok: pw.pc.rml-ok rml-p
proof -
  have p0: pw.pc.rm-present (pw.punit 0) = {1}
    using wit-parallel-merged-1
  by (auto simp: pw.pc.rm-present-def pw.punit-def cdom-p-def)
  have p1: pw.pc.rm-present (pw.punit 1) = {2}
    using wit-parallel-merged-2
  by (auto simp: pw.pc.rm-present-def pw.punit-def cdom-p-def)
  have distinct (rml-p u)  $\wedge$  set (rml-p u) = pw.pc.rm-present u
    if u: u  $\in$  set pw.punits for u
  proof -
    from u obtain i where i: i < 2 u = pw.punit i
    by (rule pw.punits-mem)
    consider (Z) i = 0 | (O) i = 1
    using i(1) by linarith
    then show ?thesis
  proof cases
    case Z

```

```

    then have  $rml-p\ u = [1]$ 
      using  $i(2)$  by (simp add:  $rml-p-def$ )
    then show ?thesis
      using  $i(2)$   $Z\ p0$  by simp
  next
  case  $O$ 
  then have  $u \neq pw.punit\ 0$ 
    using  $i(2)$  wit-parallel-punit-ne by simp
  then have  $rml-p\ u = [2]$ 
    by (simp add:  $rml-p-def$ )
  then show ?thesis
    using  $i(2)$   $O\ p1$  by simp
qed
qed
then show ?thesis
  by (simp add:  $pw.pc.rml-ok-def$ )
qed

```

The equivalence theorem at the witness, plus the concrete replay values: the merged-chunk stream replays key 1 to its reinserted value and key 2 to its inserted value — the source state at the frontier, per *wit-parallel-cut*.

lemma *wit-parallel-replay*:

```

  stream-replay (pw.pc.rm-emission  $rml-p\ 0$ ) 1 = pw.pc.sink-at 4 1
  stream-replay (pw.pc.rm-emission  $rml-p\ 0$ ) 2 = pw.pc.sink-at 4 2
  by (rule pw.parallel-replay[OF wit-parallel-rml-ok], simp)+

```

lemma *wit-parallel-values*:

```

  pw.pc.sink-at 4 1 = Some 11
  pw.pc.sink-at 4 2 = Some 20

```

proof –

```

  have  $e1$ :  $evs-for\ 1\ (pw.ocs.win\ (u-lo\ (pw.punit\ 0))\ 4) =$ 
    [ $Event\ 1\ (Some\ 10)$ ,  $Event\ 1\ None$ ,  $Event\ 1\ (Some\ 11)$ ]
  unfolding pw.ocs.win-def
  by (simp add: pw.punit-def clow-p-def offden-def L-p-def evs-for-def)
  show  $pw.pc.sink-at\ 4\ 1 = Some\ 11$ 
    unfolding pw.pc.sink-at-def wit-parallel-the-unit-1  $e1$  by simp
  have  $e2$ :  $evs-for\ 2\ (pw.ocs.win\ (u-lo\ (pw.punit\ 1))\ 4) =$ 
    [ $Event\ 2\ (Some\ 20)$ ]
  unfolding pw.ocs.win-def
  by (simp add: pw.punit-def clow-p-def offden-def L-p-def evs-for-def)
  show  $pw.pc.sink-at\ 4\ 2 = Some\ 20$ 
    unfolding pw.pc.sink-at-def wit-parallel-the-unit-2  $e2$  by simp
qed

```

end

```

theory Dump-Splice-Instance
  imports Range-Merge

```

begin

9 Native-dump splice and LSN-positioned selects

Native-dump splices and LSN-positioned selects share one formal object: a point-bracket plan ($lo = hi = c^*$) whose refresh is an engine-consistent read at a published coordinate. The plan signature is deliberately silent about refresh provenance, so the LSN-positioned select (MariaDB *START TRANSACTION WITH CONSISTENT SNAPSHOT* plus the binlog-snapshot status pair; PostgreSQL slot-exported snapshots) and the native dump (mysql-dump *--single-transaction --source-data*, mariadb-dump, *pg-dump* on an exported snapshot) instantiate the SAME locale — they differ in named-assumption pedigree and operational preconditions, not in formal content. For a coordinate-bound dump, the coordinate is captured at the snapshot instant (“In all cases, any action on logs happens at the exact moment of the dump”), no closing marker exists (the high edge is definitionally the low edge), and the consistency guarantee is engine-scoped (“only InnoDB tables are dumped in a consistent state”).

The named assumption register (O4: externally asserted clauses enter as named assumptions, never as proved facts). Three carry formal content and are locale assumptions below; two are external preconditions whose in-model shadow is structural, carried by this register and the witness rather than by a vacuous axiom:

- **A-cert** (*A-cert*): the snapshot’s read view and the engine-published coordinate coincide ($n\text{-view} = n\text{-pub}$). O4 records this coincidence as a named external assumption.
- **A-scope** (*A-scope*): the recorded splice coordinate is the published one ($n\text{-rec} = n\text{-pub}$) — the MariaDB binlog-snapshot status pair is Global-scope, so the read must be sequenced against concurrent snapshots overwriting it. The assumption names exactly that hazard.
- **A-engine** (*A-engine* with *A-engine-scope*): the dump is a consistent read at the view coordinate FOR ENGINE-BACKED KEYS ONLY (InnoDB-only; MyISAM/MEMORY excluded), and the dumped scope lies inside the engine-backed keys. The engine-scope caveat is a real premise of the derivation, not a footnote.
- **A-ddl** (DDL prohibition during the dump): a documented, UNENFORCED external precondition (“no other connection should use ALTER TABLE, CREATE TABLE, DROP TABLE, RENAME TABLE, or TRUNCATE TABLE”). Its in-model shadow is structural: this event model has a fixed key space and no schema transitions, so the model REPRESENTS the deployment only when the prohibition

holds. There is no in-model proposition to assume; the instance carries the precondition by this register.

- **Scope witness** (server-wide vs dumped-scope coordinate, the *gtid-purged* caveat): the coordinate denotes a prefix of the **WHOLE** server log L — covering transactions “even those that changed suppressed parts of the database” — while the plan’s scope is only *dumped*. The model expresses this directly: L ranges over all keys, the claim is restricted to *dumped*, and the witness log below contains a non-dumped key’s event to keep the distinction visible.

O5 (representation agreement) is this instance’s load-bearing obligation and is discharged **AT THE TYPED REFRESH**: the locale fixes R at the contract’s state type, which is exactly the point where a real deployment constructs typed rows from dump-file bytes. The dump-file-vs-log **RENDERING** half (charset, collation, numeric edge cases — where real splices break) is the operational clause of O5, cite-only, discharged per deployment when R is constructed.

Guarantee level: the point bracket puts the whole dedup machinery out of work (the window is empty — *point-window-vacuous*; §6 point-splice: nothing to prove), and the recorded coordinate is a shared state witness for the one-unit plan, so Corollary 1 applies: an exhibited common read-state anchor **AT** the splice coordinate (*splice-trajectory*) — tier T3 on the dumped scope, the fast-bootstrap headline. For this one point unit the splice coordinate is also the latest high, so T2 already implies the same trajectory interval. T3 additionally records why the refresh is one coherent source state. In a multi-unit plan a shared witness can also precede the latest high. Per-table splices at **DISTINCT** recorded points compose by Corollary 3 to global T2 (*splice-mix-any*). The composite has a conservative trajectory from its latest recorded splice, but the tablesync fixture shows that no common witness or earlier start follows automatically.

The degradation theorem: an **UNTRUSTED** recovery point — a restored physical backup whose asserted coordinate nobody certifies — widens to a genuine bracket [*backup-start*, *post-recovery*] and stays **INSIDE** the contract: the widened plan is a contract instance (*degraded-cut*, placed at tier T2 under the available recorded bounds), has a safe canonical trajectory from the bracket high (*degraded-trajectory-from-end*), and the §6 formal window-discard construction across the uncertainty window has final replay equal to the canonical sink (*degraded-replay*). A physical emission inherits that result only when it realizes the formal construction and its observation and close premises. Formally the latent true view coordinate still sits in the bracket — O2’s existential witness — and even remains a shared witness in the Corollary-1 equations. The locale does not encode what an operational capture knows or can exhibit. Under the available recorded evidence, the known

high edge remains a safe, conservative canonical start. The citable claim of the untrusted instance is therefore T2, while the trusted point carries the additional T3 witness evidence. The trusted point instance is literally the degenerate member of the bracket family (*trusted-point-is-degenerate-bracket*).

9.1 The trusted point splice

```

locale dump-splice =
  fixes  $L :: ('k, 'v)$  event list
    and  $\sigma 0 :: ('k, 'v)$  state
    and  $\text{dumped} :: 'k$  set
    and  $R :: ('k, 'v)$  state
    and  $n\text{-view } n\text{-pub } n\text{-rec } nf :: \text{nat}$ 
    and  $\text{engine-ok} :: 'k \Rightarrow \text{bool}$ 
  assumes  $A\text{-engine}: \bigwedge k. \text{engine-ok } k \implies$ 
     $R \ k = \text{state-after } \sigma 0 \ (\text{take } n\text{-view } L) \ k$ 
    and  $A\text{-engine-scope}: \bigwedge k. k \in \text{dumped} \implies \text{engine-ok } k$ 
    and  $A\text{-cert}: n\text{-view} = n\text{-pub}$ 
    and  $A\text{-scope}: n\text{-rec} = n\text{-pub}$ 
    and  $\text{rec-f}: n\text{-rec} \leq nf$ 
    and  $f\text{-len}: nf \leq \text{length } L$ 
begin

definition dunit  $:: ('k, 'v, \text{nat}) \text{ cunit}$  where
   $dunit = (\text{u-dom} = \text{dumped}, \text{u-lo} = n\text{-rec}, \text{u-hi} = n\text{-rec}, \text{u-refresh} = R)$ 

lemma view-eq-rec:  $n\text{-view} = n\text{-rec}$ 
  using  $A\text{-cert } A\text{-scope}$  by simp

lemma rec-len:  $n\text{-rec} \leq \text{length } L$ 
  using  $\text{rec-f } f\text{-len}$  by simp

  The assumption composition: the engine's consistent read at the view
  (A-engine), the view-published coincidence (A-cert), and the published-
  recorded identity (A-scope) chain into point validity at the RECORDED
  coordinate — O2 with a point witness.

lemma refresh-honest-at-rec:
  assumes  $k \in \text{dumped}$ 
  shows  $R \ k = \text{state-after } \sigma 0 \ (\text{take } n\text{-rec } L) \ k$ 
  using  $A\text{-engine}[OF \ A\text{-engine-scope}[OF \ \text{assms}]] \text{ view-eq-rec}$  by simp

end

sublocale dump-splice  $\subseteq dcs$ : coordinate-space  $L$  offden  $L$ 
  by unfold-locales (rule offden-le-len)

sublocale dump-splice  $\subseteq dc$ : capture-contract  $L$  offden  $L \ \sigma 0 \ [dunit] \ \text{dumped } nf$ 
proof (unfold-locales)

```

```

  show  $(\bigcup_{u \in \text{set } [dunit]} u\text{-dom } u) = \text{dumped}$ 
    by (simp add: dunit-def)
next
  fix i j
  assume  $i < \text{length } [dunit]$   $j < \text{length } [dunit]$   $i \neq j$ 
  then show  $u\text{-dom } ([dunit] ! i) \cap u\text{-dom } ([dunit] ! j) = \{\}$  by simp
next
  fix u assume  $u \in \text{set } [dunit]$ 
  then show  $\text{dcs.cle } (u\text{-lo } u) (u\text{-hi } u)$ 
    by (simp add: dunit-def dcs.cle-def)
next
  fix u assume  $u \in \text{set } [dunit]$ 
  then show  $\text{dcs.cle } (u\text{-hi } u) \text{ nf}$ 
    using rec-f by (simp add: dunit-def dcs.cle-def offden-mono)
next
  fix u k assume  $u \in \text{set } [dunit]$  and  $k: k \in u\text{-dom } u$ 
  then have  $u\text{-eq}: u = \text{dunit}$  and  $k\text{-dom}: k \in \text{dumped}$ 
    by (auto simp: dunit-def)
  show  $\exists c. \text{dcs.cle } (u\text{-lo } u) c \wedge \text{dcs.cle } c (u\text{-hi } u) \wedge$ 
     $u\text{-refresh } u k = \text{state-after } \sigma 0 (\text{dcs.pfx } c) k$ 
  proof (intro exI[of - n-rec] conjI)
    show  $\text{dcs.cle } (u\text{-lo } u) \text{ n-rec}$ 
      by (simp add: u-eq dunit-def dcs.cle-def)
    show  $\text{dcs.cle } \text{ n-rec } (u\text{-hi } u)$ 
      by (simp add: u-eq dunit-def dcs.cle-def)
    show  $u\text{-refresh } u k = \text{state-after } \sigma 0 (\text{dcs.pfx } \text{ n-rec}) k$ 
      using refresh-honest-at-rec[OF k-dom]
      by (simp add: u-eq dunit-def dcs.pfx-def offden-id[OF rec-len])
  qed
qed

```

```

context dump-splice
begin

```

§6 point-splice, made literal: the window of the point bracket is empty — there is nothing to dedup, and the merge machinery of the wider instances is vacuous here.

```

lemma point-window-vacuous:  $\text{dcs.win } \text{ n-rec } \text{ n-rec} = []$ 
  by (simp add: dcs.win-def)

```

```

theorem splice-cut:

```

```

  assumes  $k \in \text{dumped}$ 
  shows  $\text{dc.sink-at } \text{ nf } k = \text{state-after } \sigma 0 (\text{take } \text{ nf } L) k$ 
proof -
  have  $\text{dc.sink-at } \text{ nf } k = \text{state-after } \sigma 0 (\text{dcs.pfx } \text{ nf}) k$ 
    by (rule dc.contract-cut[OF assms])
  then show ?thesis
    by (simp add: dcs.pfx-def offden-id[OF f-len])
qed

```

A concrete point-splice emission uses the formal window-discard stream with consumption starting exactly at the splice coordinate. Because the unit closes at that same coordinate, the close block of present copied rows precedes the suffix event at that position and the rest of the observed suffix follows in log order. The theorem is replay equivalence for this modeled order, not entry-for-entry identity with another stream representation.

theorem *splice-replay*:

assumes *svl*: *dc.svl-ok svl* **and** *k*: *k* ∈ *dumped*
shows *stream-replay* (*dc.emission svl n-rec*) *k* = *dc.sink-at nf k*
proof (*rule dc.window-discard-replay[OF svl - k]*)
fix *u* **assume** *u* ∈ *set [dunit]*
then show *dcs.cle n-rec (u-lo u)*
by (*simp add: dunit-def dcs.cle-def*)
qed

theorem *splice-emitted-cut*:

assumes *svl*: *dc.svl-ok svl* **and** *k*: *k* ∈ *dumped*
shows *stream-replay* (*dc.emission svl n-rec*) *k* = *state-after σ 0 (take nf L) k*
using *splice-replay[OF svl k] splice-cut[OF k]* **by** *simp*

Corollary 1 at the instance — tier T3: the recorded splice coordinate is an exhibited shared state witness, and from it up to the frontier the replay tracks the source trajectory. For this point unit the witness equals the latest high. The additional T3 content is the exact view binding, not trajectory existence beyond what the latest-high theorem already supplies.

theorem *splice-trajectory*:

assumes *rg*: *n-rec ≤ g* **and** *gf*: *g ≤ nf* **and** *k*: *k* ∈ *dumped*
shows *dc.sink-at g k* = *state-after σ 0 (take g L) k*
proof –
have *g-len*: *g ≤ length L*
using *gf f-len* **by** *simp*
have *dc.sink-at g k* = *state-after σ 0 (dcs.pfx g) k*
proof (*rule dc.shared-witness-trajectory[where cstar = n-rec]*)
show $\bigwedge u. u \in \text{set } [dunit] \implies dcs.cle (u-lo u) n-rec \wedge dcs.cle n-rec (u-hi u)$
by (*simp add: dunit-def dcs.cle-def*)
show $\bigwedge u k. u \in \text{set } [dunit] \implies k \in u-dom u \implies$
 $u-refresh u k = state-after \sigma 0 (dcs.pfx n-rec) k$
proof –
fix *u kk* **assume** *u* ∈ *set [dunit]* **and** *kk* ∈ *u-dom u*
then have *u-eq*: *u = dunit* **and** *kk-dom*: *kk* ∈ *dumped*
by (*auto simp: dunit-def*)
show *u-refresh u kk* = *state-after σ 0 (dcs.pfx n-rec) kk*
using *refresh-honest-at-rec[OF kk-dom]*
by (*simp add: u-eq dunit-def dcs.pfx-def offden-id[OF rec-len]*)
qed
show *dcs.cle n-rec g*
using *rg* **by** (*simp add: dcs.cle-def offden-mono*)
show *dcs.cle g nf*

```

    using gf by (simp add: dcs.cle-def offden-mono)
  show  $k \in \text{dumped}$ 
    by (rule k)
qed
then show ?thesis
  by (simp add: dcs.pfx-def offden-id[OF g-len])
qed

```

Corollary 3 at the instance: the splice composes with ANY conforming plan over a disjoint scope at the shared frontier — per-table dumps, chunk fleets, point selects. Two splices at DISTINCT recorded points compose conservatively to T2. Their union has a trajectory from its latest recorded splice, but composition alone supplies neither a common in-bracket witness nor an earlier start. The sharpness is witnessed by *fix-sync-no-early-onset* in theory *Contract-Witnesses*. One shared snapshot across the members supplies the common witness required for T3 via Corollary 1.

theorem *splice-mix-any*:

```

  assumes capture-contract L (offden L)  $\sigma 0$  us2 sc2 nf
    and dumped  $\cap$  sc2 = {}
  shows capture-contract L (offden L)  $\sigma 0$  ([dunit] @ us2) (dumped  $\cup$  sc2) nf
  by (rule dcs.mixing-union[OF dc.capture-contract-axioms assms])

```

end

9.2 The uncertainty-bracket degradation

The untrusted variant: no A-cert, no A-scope — nobody certifies WHERE the recovery point sits. What survives is a BOUND: the true read view lies between the backup start and the post-recovery coordinate (*A-uncertainty-lo*/*A-uncertainty-hi*), and the engine read is consistent at that unknown view (*A-engine*). The plan widens its bracket to the whole uncertainty interval; O2 holds with the unknown view as the existential witness. Trust is exchanged for dedup work, not for membership.

locale *dump-splice-untrusted* =

```

  fixes L :: ('k, 'v) event list
    and  $\sigma 0$  :: ('k, 'v) state
    and dumped :: 'k set
    and R :: ('k, 'v) state
    and n-start n-view n-end nf :: nat
    and engine-ok :: 'k  $\Rightarrow$  bool
  assumes A-engine:  $\bigwedge k. \text{engine-ok } k \implies$ 
    R k = state-after  $\sigma 0$  (take n-view L) k
    and A-engine-scope:  $\bigwedge k. k \in \text{dumped} \implies \text{engine-ok } k$ 
    and A-uncertainty-lo: n-start  $\leq$  n-view
    and A-uncertainty-hi: n-view  $\leq$  n-end
    and end-f: n-end  $\leq$  nf
    and f-len: nf  $\leq$  length L

```

begin

definition $wunit :: ('k, 'v, nat) \text{ cunit}$ **where**

$wunit = (\lambda u. \text{dom} = \text{dumped}, u\text{-lo} = n\text{-start}, u\text{-hi} = n\text{-end}, u\text{-refresh} = R)$

lemma $\text{view-len}: n\text{-view} \leq \text{length } L$

using $A\text{-uncertainty-hi}$ end-f $f\text{-len}$ **by** simp

end

sublocale $\text{dump-splice-untrusted} \subseteq \text{ucs}: \text{coordinate-space } L \text{ offden } L$
by unfold-locales $(\text{rule } \text{offden-le-len})$

sublocale $\text{dump-splice-untrusted} \subseteq$

$uc: \text{capture-contract } L \text{ offden } L \sigma 0 [wunit] \text{ dumped } nf$

proof (unfold-locales)

show $(\bigcup u \in \text{set } [wunit]. u\text{-dom } u) = \text{dumped}$

by $(\text{simp add: wunit-def})$

next

fix $i \ j$

assume $i < \text{length } [wunit] \ j < \text{length } [wunit] \ i \neq j$

then show $u\text{-dom } ([wunit] ! i) \cap u\text{-dom } ([wunit] ! j) = \{\}$ **by** simp

next

fix u **assume** $u \in \text{set } [wunit]$

then show $\text{ucs.cle } (u\text{-lo } u) \ (u\text{-hi } u)$

using $A\text{-uncertainty-lo}$ $A\text{-uncertainty-hi}$

by $(\text{simp add: wunit-def ucs.cle-def offden-mono})$

next

fix u **assume** $u \in \text{set } [wunit]$

then show $\text{ucs.cle } (u\text{-hi } u) \ nf$

using end-f **by** $(\text{simp add: wunit-def ucs.cle-def offden-mono})$

next

fix $u \ k$ **assume** $u \in \text{set } [wunit]$ **and** $k: k \in u\text{-dom } u$

then have $u\text{-eq}: u = wunit$ **and** $k\text{-dom}: k \in \text{dumped}$

by $(\text{auto simp: wunit-def})$

show $\exists c. \text{ucs.cle } (u\text{-lo } u) \ c \wedge \text{ucs.cle } c \ (u\text{-hi } u) \wedge$

$u\text{-refresh } u \ k = \text{state-after } \sigma 0 \ (\text{ucs.pfx } c) \ k$

proof $(\text{intro exI[of } n\text{-view]} \text{ conjI})$

show $\text{ucs.cle } (u\text{-lo } u) \ n\text{-view}$

using $A\text{-uncertainty-lo}$

by $(\text{simp add: u-eq wunit-def ucs.cle-def offden-mono})$

show $\text{ucs.cle } n\text{-view} \ (u\text{-hi } u)$

using $A\text{-uncertainty-hi}$

by $(\text{simp add: u-eq wunit-def ucs.cle-def offden-mono})$

show $u\text{-refresh } u \ k = \text{state-after } \sigma 0 \ (\text{ucs.pfx } n\text{-view}) \ k$

using $A\text{-engine[OF } A\text{-engine-scope[OF } k\text{-dom}]$

by $(\text{simp add: u-eq wunit-def ucs.pfx-def offden-id[OF view-len]})$

qed

qed

context *dump-splice-untrusted*
begin

The widened-bracket plan is a T2 contract instance. The cut holds at the frontier, and the unit's known high edge is the conservative start supplied by the latest-high theorem.

theorem *degraded-cut*:
assumes $k \in \text{dumped}$
shows $uc.\text{sink-at } nf\ k = \text{state-after } \sigma 0\ (\text{take } nf\ L)\ k$
proof –
have $uc.\text{sink-at } nf\ k = \text{state-after } \sigma 0\ (ucs.\text{pfx } nf)\ k$
by (*rule* $uc.\text{contract-cut}[OF\ \text{assms}]$)
then show *?thesis*
by (*simp add: ucs.pfx-def offden-id*[*OF f-len*])
qed

theorem *degraded-trajectory-from-end*:
assumes $eg: n\text{-end} \leq g$ **and** $gf: g \leq nf$ **and** $k: k \in \text{dumped}$
shows $uc.\text{sink-at } g\ k = \text{state-after } \sigma 0\ (\text{take } g\ L)\ k$
proof –
have $g\text{-len}: g \leq \text{length } L$
using $gf\ f\text{-len}$ **by** *simp*
have $uc.\text{sink-at } g\ k = \text{state-after } \sigma 0\ (ucs.\text{pfx } g)\ k$
proof (*rule* $uc.\text{contract-cut-at}[OF\ -\ k]$)
fix u **assume** $u \in \text{set } [wunit]$
then show $ucs.\text{cle } (u\text{-hi } u)\ g$
using eg **by** (*simp add: wunit-def ucs.cle-def offden-mono*)
qed
then show *?thesis*
by (*simp add: ucs.pfx-def offden-id*[*OF g-len*])
qed

And the §6 window-discard discipline runs across the uncertainty window. With an exact survivor listing and consumption beginning at or before the backup start, *uc.emission* consumes the log through the frontier, closes the survivor block at the post-recovery edge after processing that window, and has final replay equal to the canonical sink. This theorem states final replay of that formal construction, not prefix equivalence of a physical shipped stream. The untrusted point costs dedup work over [*backup*–*start*, *post*–*recovery*]; it does not cost contract membership.

theorem *degraded-replay*:
assumes $svl: uc.\text{svl-ok } svl$
and $s0: s0 \leq n\text{-start}$
and $k: k \in \text{dumped}$
shows $\text{stream-replay } (uc.\text{emission } svl\ s0)\ k = uc.\text{sink-at } nf\ k$
proof (*rule* $uc.\text{window-discard-replay}[OF\ svl\ -\ k]$)
fix u **assume** $u \in \text{set } [wunit]$

```

    then show ucs.cle s0 (u-lo u)
    using s0 by (simp add: wunit-def ucs.cle-def offden-mono)
qed

theorem degraded-emitted-cut:
  assumes svl: uc.svl-ok svl
    and s0: s0 ≤ n-start
    and k: k ∈ dumped
  shows stream-replay (uc.emission svl s0) k = state-after σ0 (take nf L) k
  using degraded-replay[OF svl s0 k] degraded-cut[OF k] by simp

end

```

The trusted point instance is the degenerate member of the uncertainty-bracket family: certifying the coordinate collapses the bracket to a point. Both members remain inside one contract family. The point member additionally exhibits the true shared state witness, while the interval member retains its conservative start at the known high edge.

```

context dump-splice
begin

```

```

theorem trusted-point-is-degenerate-bracket:
  dump-splice-untrusted L σ0 dumped R n-rec n-view n-rec nf engine-ok
proof (unfold-locales)
  show  $\bigwedge k. \text{engine-ok } k \implies R\ k = \text{state-after } \sigma 0\ (\text{take } n\text{-view } L)\ k$ 
    by (rule A-engine)
  show  $\bigwedge k. k \in \text{dumped} \implies \text{engine-ok } k$ 
    by (rule A-engine-scope)
  show  $n\text{-rec} \leq n\text{-view}$ 
    using view-eq-rec by simp
  show  $n\text{-view} \leq n\text{-rec}$ 
    using view-eq-rec by simp
  show  $n\text{-rec} \leq nf$ 
    by (rule rec-f)
  show  $nf \leq \text{length } L$ 
    by (rule f-len)
qed

end

```

9.3 Constructed witness: the trusted splice (non-vacuity)

A three-event server log: the dumped key 1 is inserted, a NON-dumped key 9 is written (another database on the same server — the server-wide coordinate covers it, the dumped-scope claim says nothing about it: the *gtid-purged* scope caveat made concrete), and key 1 is deleted AFTER the splice coordinate. The dump reads at coordinate 2, the splice replays the post-splice delete, and the trajectory holds from the splice coordinate: at

the splice the key is present at its dumped value, at the frontier it is absent
— both equal to the source.

definition $L-d :: (nat, nat)$ event list **where**
 $L-d = [Event\ 1\ (Some\ 8),\ Event\ 9\ (Some\ 99),\ Event\ 1\ None]$

definition $R-d :: (nat, nat)$ state **where**
 $R-d = (\lambda k. \text{if } k = 1 \text{ then } Some\ 8 \text{ else } None)$

definition $eng-d :: nat \Rightarrow bool$ **where**
 $eng-d\ k \longleftrightarrow k = 1$

interpretation ds : dump-splice $L-d\ \lambda\cdot$. None $\{1\}\ R-d\ 2\ 2\ 2\ 3\ eng-d$

proof (unfold-locales)

fix $k :: nat$ **assume** $eng-d\ k$
then have $k = 1$ **by** (simp add: eng-d-def)
then show $R-d\ k = state-after\ (\lambda\cdot. None)\ (take\ 2\ L-d)\ k$
by (simp add: R-d-def L-d-def state-after-def apply-ev-def)
next
fix $k :: nat$ **assume** $k \in \{1\}$
then show $eng-d\ k$ **by** (simp add: eng-d-def)
qed (simp-all add: L-d-def)

lemma wit-dump-the-unit [simp]: $ds.dc.the-unit\ 1 = ds.dunit$
using $ds.dc.the-unit-props[of\ 1]$ **by** simp

lemma wit-dump-cut:
 $ds.dc.sink-at\ 3\ 1 = state-after\ (\lambda\cdot. None)\ (take\ 3\ L-d)\ 1$
by (rule ds.splice-cut) simp

lemma wit-dump-cut-value: $ds.dc.sink-at\ 3\ 1 = None$

proof —

have win : $evs-for\ 1\ (ds.dcs.win\ (u-lo\ ds.dunit)\ 3) = [Event\ 1\ None]$
unfolding $ds.dcs.win-def\ ds.dunit-def$
by (simp add: offden-def L-d-def evs-for-def)
show ?thesis
unfolding $ds.dc.sink-at-def\ wit-dump-the-unit\ win$ **by** simp
qed

lemma wit-dump-trajectory:

$ds.dc.sink-at\ 2\ 1 = state-after\ (\lambda\cdot. None)\ (take\ 2\ L-d)\ 1$
by (rule ds.splice-trajectory) simp-all

lemma wit-dump-trajectory-value: $ds.dc.sink-at\ 2\ 1 = Some\ 8$

proof —

have $win-e$: $evs-for\ 1\ (ds.dcs.win\ (u-lo\ ds.dunit)\ 2) = []$
unfolding $ds.dcs.win-def\ ds.dunit-def$
by (simp add: offden-def L-d-def evs-for-def)
have $ds.dc.sink-at\ 2\ 1 = u-refresh\ ds.dunit\ 1$
unfolding $ds.dc.sink-at-def\ wit-dump-the-unit\ win-e$ **by** simp

```

    also have ... = R-d 1
      unfolding ds.dunit-def by simp
    also have ... = Some 8
      by (simp add: R-d-def)
    finally show ?thesis .
qed

```

9.4 Constructed witness: the degraded bracket (non-vacuity)

The uncertainty bracket doing real work: the restored read is valid at the unknown view coordinate 1 (after the first insert), the bracket spans the whole log, and an in-window event supersedes the restored value — the §6 discard discipline drops the buffered row and the log wins. The replay lands on the source state at the frontier, per the degradation theorems.

definition $L-u :: (nat, nat)$ event list **where**
 $L-u = [Event\ 1\ (Some\ 4),\ Event\ 1\ (Some\ 6)]$

definition $R-u :: (nat, nat)$ state **where**
 $R-u = (\lambda k. \text{if } k = 1 \text{ then } Some\ 4 \text{ else } None)$

interpretation du : dump-splice-untrusted $L-u\ \lambda\cdot$. None {1} $R-u\ 0\ 1\ 2\ 2\ eng-d$

proof (unfold-locales)

```

  fix k :: nat assume eng-d k
  then have k = 1 by (simp add: eng-d-def)
  then show R-u k = state-after ( $\lambda\cdot$ . None) (take 1 L-u) k
    by (simp add: R-u-def L-u-def state-after-def apply-ev-def)
next
  fix k :: nat assume k ∈ {1}
  then show eng-d k by (simp add: eng-d-def)
next
  show (0 :: nat) ≤ 1 by simp
next
  show (1 :: nat) ≤ 2 by simp
next
  show (2 :: nat) ≤ 2 by simp
next
  show (2 :: nat) ≤ length L-u by (simp add: L-u-def)
qed

```

lemma wit-degraded-the-unit [simp]: $du.uc.the-unit\ 1 = du.wunit$
 using $du.uc.the-unit-props[of\ 1]$ by simp

lemma wit-degraded-cut:
 $du.uc.sink-at\ 2\ 1 = state-after\ (\lambda\cdot$. None) (take 2 L-u) 1
 by (rule $du.degraded-cut$) simp

lemma wit-degraded-win:
 $evs-for\ 1\ (du.ucs.win\ (u-lo\ du.wunit)\ (u-hi\ du.wunit)) =$

$[Event\ 1\ (Some\ 4),\ Event\ 1\ (Some\ 6)]$
unfolding $du.ucs.win-def\ du.wunit-def$
by ($simp\ add: offden-def\ L-u-def\ evs-for-def$)

lemma *wit-degraded-cut-value*: $du.uc.sink-at\ 2\ 1 = Some\ 6$
proof –
have $win: evs-for\ 1\ (du.ucs.win\ (u-lo\ du.wunit)\ 2) =$
 $[Event\ 1\ (Some\ 4),\ Event\ 1\ (Some\ 6)]$
unfolding $du.ucs.win-def\ du.wunit-def$
by ($simp\ add: offden-def\ L-u-def\ evs-for-def$)
show *?thesis*
unfolding $du.uc.sink-at-def\ wit-degraded-the-unit\ win$ **by** $simp$
qed

lemma *wit-degraded-surv-empty*: $du.uc.survivors\ du.wunit = \{\}$
proof ($rule\ ccontr$)
have $dom-eq: u-dom\ du.wunit = \{1\}$
unfolding $du.wunit-def$ **by** $simp$
assume $du.uc.survivors\ du.wunit \neq \{\}$
then obtain x **where** $x: x \in du.uc.survivors\ du.wunit$
by $blast$
have $x-dom: x \in u-dom\ du.wunit$
using $x\ du.uc.survivors-sub-dom$ **by** $blast$
have $x-quiet: evs-for\ x\ (du.ucs.win\ (u-lo\ du.wunit)\$
 $(u-hi\ du.wunit)) = []$
using x **unfolding** $du.uc.survivors-def$ **by** $blast$
have $x1: x = 1$
using $x-dom$ **unfolding** $dom-eq$ **by** $blast$
have $evs-for\ 1\ (du.ucs.win\ (u-lo\ du.wunit)\ (u-hi\ du.wunit)) = []$
using $x-quiet$ **unfolding** $x1$.
then show *False*
using *wit-degraded-win* **by** $simp$
qed

lemma *wit-degraded-svl-ok*: $du.uc.svl-ok\ (\lambda-. [])$
unfolding $du.uc.svl-ok-def$
proof
fix u **assume** $u \in set\ [du.wunit]$
then have $u = du.wunit$ **by** $simp$
then show $distinct\ ((\lambda-. [])\ u) \wedge$
 $set\ ((\lambda-. [])\ u) = du.uc.survivors\ u$
using *wit-degraded-surv-empty* **by** $simp$
qed

lemma *wit-degraded-replay*:
 $stream-replay\ (du.uc.emission\ (\lambda-. [])\ 0)\ 1 = du.uc.sink-at\ 2\ 1$
by ($rule\ du.degraded-replay[OF\ wit-degraded-svl-ok]$) $simp-all$

lemma *wit-degraded-replay-value*:

```

    stream-replay (du.uc.emission ( $\lambda$ -. []) 0) 1 = Some 6
    using wit-degraded-replay wit-degraded-cut-value by simp
end

```

References

- [1] A. Andreakis. DBLog_Virtual_Cuts: Isabelle/HOL formal development for “A Theoretical Study of DBLog”, 2026. Zenodo, version 2.1; concept [doi:10.5281/zenodo.20389696](https://doi.org/10.5281/zenodo.20389696) (resolves to the latest version).
- [2] A. Andreakis. A theoretical study of DBLog: Certified virtual cuts for a snapshot-equivalent replay of live databases, 2026. arXiv:2605.31475.
- [3] A. Andreakis and I. Papapanagiotou. DBLog: A watermark based change-data-capture framework, 2020. arXiv:2010.12597.